

Texas Instruments

TMS320DM270 System Specification

Version 0.2

Preliminary

Revision History

Revision N0.	Date	Modification List
Version 0.0	01/18/2002	First edition of DSC27 system spec
Version 0.1	05/11/2002	First release of DM270 system spec
Version 0.2	09/16/2002	<u>6.8.2 Serial boot-up mode</u> <u>7.6 SDRAM & External Memory I/F Pin Sharing</u> <u>10 ARM-DSP Communication</u> <u>11 Power Saving Modes</u>

Preliminary

Contents

1	FEATURES AND PERFORMANCE TARGET	9
1.1	Key features:.....	10
2	BLOCK DIAGRAM.....	12
3	SYSTEM DATA FLOWS	13
3.1	Preview Mode.....	13
3.2	Still Picture Capture Mode.....	13
3.3	Video Capture Mode.....	14
3.4	Still Picture and Video Playback	15
4	IMAGING PERIPHERALS.....	16
4.1	CCD Controller.....	16
4.1.1	Features.....	16
4.1.2	Signal Interface	17
4.1.3	The Typical CCDC Interface.....	18
4.1.4	CCD Controller Processing.....	18
4.1.5	Digital Clamp	19
4.1.6	Black Level Compensation.....	20
4.1.7	Median Filter.....	20
4.1.8	Gain and Offset.....	21
4.1.9	Output Formatter	21
4.1.10	CCDC Interrupts.....	22
4.1.11	Shutter Control.....	22
4.1.12	Timing Generator	22
4.1.13	YCbCr signal Processing	23
4.1.14	Input/Output port configuration	24
4.2	Preview Engine.....	26
4.2.1	Features.....	27
4.2.2	CFA Arrangements	28
4.2.3	Horizontal noise filter	28
4.2.4	White Balance.....	28
4.2.5	CFA Interpolation	29
4.2.6	Black Adjustment Sub-Module	30
4.2.7	RGB2RGB Blending Module.....	30
4.2.8	Gamma Correction Module.....	31
4.2.9	RGB2YCbCr Conversion Matrix.....	31
4.2.10	4:2:2 Conversion and Output Interface Modules	31
4.2.11	4:2:2 SDRAM Input mode	32
4.3	AF/AE/AWB Statistics Engine	33
4.3.1	Features.....	33
4.3.2	AF Engine.....	34
4.3.3	AE/AWB Engine.....	37
4.4	On Screen Display (OSD) and Graphics Acceleration	40
4.4.1	Features.....	40
4.4.2	OSD data storage	41
4.4.3	Setting Up an OSD Window.....	41
4.4.4	Location Register	41
4.4.5	Color Look Up Table.....	42

4.4.6	Blending and Transparency.....	42
4.4.7	Hardware Cursor.....	42
4.4.8	Window Magnification.....	43
4.5	NTCS/PAL Encoder	44
4.5.1	Composite analog video features.....	45
4.5.2	Digital video features	45
4.5.3	Signal Level	46
4.5.4	Signal calculation	47
4.5.5	Color signal cutoff frequency select.....	47
4.5.6	Video attribute insertion	47
4.5.7	Digital RGB/YUV Interface.....	47
5	DSP SUBSYSTEM.....	49
5.1	DSP core.....	49
5.1.1	Features.....	49
5.1.2	DSP peripherals.....	50
5.1.3	Multi-channel buffered serial ports	50
5.1.4	DSP subsystem memory	50
5.1.5	Program data and IO spaces.....	51
5.1.6	DSP Memory map.....	51
5.1.7	I/O memory space.....	52
5.2	Image Buffer	53
5.2.1	Access Switching.....	54
5.2.2	The Image Buffer's DMA Controller	55
5.3	Imaging Extension (iMX)	56
5.3.1	iMX Structure.....	56
5.3.2	DSP/iMX Shared Memory Interface	57
5.4	VLCD Engine	59
6	ARM SUBSYSTEM.....	61
6.1	Features.....	61
6.2	ARM7TDMI RISC Processor Core.....	62
6.3	Instruction cache (I-cache) system.....	62
6.4	Memory Map.....	63
6.4.1	Reset Vector ROM.....	64
6.4.2	ARM Internal RAM.....	64
6.4.3	Peripheral Registers.....	64
6.4.4	DSP Memory.....	65
6.4.5	External Memory.....	65
6.5	Interrupt Controller.....	66
6.5.1	INTC Methodology	67
6.5.2	Hardware Configurable Interrupt Entry Table.....	68
6.6	Timers, Watch-Dog Timer.....	69
6.7	PLL / Clock Controller	69
6.8	ARM Boot-up Sequence	72
6.8.1	Self boot-up mode.....	72
6.8.2	Serial boot-up mode.....	72
6.8.3	External CPU boot-up mode.....	73
6.8.4	Hardware configuration	73
7	MEMORY TRAFFIC CONTROLLER.....	74
7.1	Memory map.....	74

7.2	Blocks Diagram.....	74
7.2.1	Access to Memory Traffic Controller(MTC) Areas	75
7.2.2	Boundary Addresses and Sizes of the External Memory Regions.....	76
7.3	External Memory Interface.....	77
7.3.1	Hardware Connection.....	77
7.3.2	Timing control	77
7.3.3	External Memory Region 0: The FLASH Memory Interface.....	79
7.3.4	External Memory Region 1: The CompactFlash Interface.....	80
7.3.5	External Memory Region 2: The SmartMedia Interface.....	81
7.3.6	External Memory Regions 3 and 4.....	82
7.4	SDRAM controller.....	83
7.4.1	Features:.....	84
7.4.2	Hardware Connection.....	85
7.4.3	Memory Structure Setting.....	85
7.4.4	SDRAM command.....	85
7.4.5	SDRAM Initialization.....	86
7.5	EMIF DMA Controller	86
7.5.1	Sources and destinations devices.....	87
7.5.2	Sources/Destinations Addresses and Transfer Size.....	87
7.6	SDRAM & External Memory I/F Pin Sharing.....	88
8	COMMUNICATION INTERF ACE	92
8.1	USB Device Controller.....	92
8.1.1	Features.....	92
8.1.2	USB Functional Description.....	92
8.1.3	USB Device Configuration	93
8.1.4	USB Power Management	94
8.1.5	DMA Data Transfer	94
8.2	SD/MMC Interface.....	95
8.3	Memory Stick Interface.....	95
8.4	Serial Port.....	96
8.5	UART	96
8.6	I ² C Interface.....	97
8.7	GIO.....	97
8.7.1	Parallel IO mode.....	97
8.7.2	External Interrupt Settings.....	97
8.7.3	GIO Functional Settings.....	97
9	EXTERNAL CPU INTERFACE.....	99
9.1	Features.....	99
9.2	Memory Space.....	100
9.3	Access Time	100
9.4	SDRAM access.....	101
10	ARM-DSP COMMUNICATION.....	102
10.1	Introduction.....	102
10.2	HPI-16 mode	102
10.3	Memory Access	102
10.4	HPI Interrupt	103
10.4.1	Host Device Using DSPINT0 to Interrupt the DSP.....	103
10.4.2	DSP Using Bit 0 of 0xFFFF to Interrupt the Host Device.....	103
10.5	Boot loader	103

10.5.1	ARM boot sequence	103
10.5.2	DSP Boot Sequence	104
11	POWER SAVING MODES	105
11.1	DSP Idle modes	105
11.1.1	IDLE1 Mode	105
11.1.2	IDLE2 Mode	105
11.2	ARM Power Saving Modes	105
11.2.1	Sleep Mode:	105
11.2.2	Sleep by sleep mode register setting	106
11.2.3	Power-down mode	106
11.2.4	Wake up Method	106
11.2.5	PLL Bypass Mode	106
11.3	DAC Power	107
12	PIN ASSIGNMENT	108

Preliminary

Figures

Figure 1	DSC engine (DM270) system architecture	9
Figure 2	Function Block Diagram	12
Figure 3	Live View mode	13
Figure 4	Still Picture Capture mode	14
Figure 5	Motion Picture Capture mode	15
Figure 6	Play Back mode	15
Figure 7	CCD controller Block Diagram	17
Figure 8	CCD signal processing	19
Figure 9	Digital Clamp Block Diagram	19
Figure 10	Median Filter	20
Figure 11	Output Formatter Block Diagram	21
Figure 12	Frame Image format conversion	22
Figure 13	Frame Image Format	23
Figure 14	YCbCr signal processing	23
Figure 15	Video Port Configuration	24
Figure 16	Video System Configuration	25
Figure 17	Preview Engine block diagram	27
Figure 18	CFA arrangements supported by the Preview Engine	28
Figure 19	White Balancing in the Preview Engine	29
Figure 20	CFA Interpolation Module (Normal Mode)	30
Figure 21	CFA Interpolation Module (Resize Only Mode)	30
Figure 22	Pseudo-color Suppression Functional Mapping	31
Figure 23	Block Diagram of 3A Hardware Engine	34
Figure 24	Paxel Configuration	35
Figure 25	IIR Filter	36
Figure 26	Window configuration	38
Figure 27	OSD block diagram	41
Figure 28	OSD Window Example	41
Figure 29	Hardware Cursor in OSD	43
Figure 30	NTSC/PAL Encoder Block Diagram	44
Figure 31	Input Video Data	47
Figure 32	DSP subsystem block diagram	51
Figure 33	DSP Memory Map	52
Figure 34	Shared Memory Block	54
Figure 35	Two dimensional data transfer	55
Figure 36	Structure of four parallel MACs	57
Figure 37	Shared Memory Between iMX and DSP	58
Figure 38	VLCD Block Diagram	59
Figure 39	ARM Subsystem	62
Figure 40	ARM Memory Map	64
Figure 41	INTC Methodology	68
Figure 42	Interrupt Entry Table	69
Figure 43	Clock Controller Block	71
Figure 44	Serial data read timing chart	72
Figure 45	Serial data write timing chart	73
Figure 46	Memory Traffic Controller blocks diagram	75
Figure 47	External Memory Bus Timings	79
Figure 48	Flash Memory Interface Data Bus	80
Figure 49	CompactFlash Interface	81
Figure 50	SmartMedia Interface	82
Figure 51	External memory bus timing with hardware wait-states	83
Figure 52	SDRAM Controller Block	84

Figure 53 EMIF DMA Controller	87
Figure 54 Pin sharing	88
Figure 55 Memory connection on DM270	90
Figure 56 Block Diagram of USB	93
Figure 57 Device Configuration	94
Figure 58 Role of the MMC Controller	95
Figure 59 Host Interface mode	100
Figure 60 ARM-DSP Communication	102
Figure 61 Host Interface mode	104

Tables

Table 1 Data Input Formats	17
Table 2 CCDC Signal Interface	18
Table 3 SDRAM Data Format(CCD)	21
Table 4 SDRAM Data Format(YcbCr)	24
Table 5 YCbCr-4:2:2 Format	31
Table 6 Comparison of the two input mode	32
Table 7 SDRAM Data Format	32
Table 8 AF Engine Configuration Registers	34
Table 9 Configuration Registers	37
Table 10 Blending and transparency in OSD	42
Table 11 Magnification Ratio	43
Table 12 Video Encoder Signal Interface	47
Table 13 I/O space	52
Table 14 Memory Structure	54
Table 15 byte-to-word conversion	55
Table 16 Memory Structure	63
Table 17 Peripheral module memory map	65
Table 18 DM270 Interrupts	66
Table 19 ARM boot-up configuration	73
Table 20 ARM memory address map	74
Table 21 Initial values of offset	76
Table 22 Chip select signal priority order	76
Table 23 External Memory Bus Pins	77
Table 24 Configurable Timing Parameters for CS0/CS3/CS4	77
Table 25 Configurable Timing Parameters for CS1	78
Table 26 Configurable Timing Parameters for CS2	78
Table 27 FLASH_CE Bus Width	80
Table 28 Relationship Between Access Size and Flash Memory Data Bus Width	80
Table 29 CompactFlash Interface Memory Map	81
Table 30 SmartMedia Interface Memory Map	82
Table 31 SDRAM Bus Pins	85
Table 32 Correspondence between memory structure and address	85
Table 33 DMA transfer source, transfer destination devices	87
Table 34 Shared Memory Configuration	88
Table 35 External HOST CPU Configuration	89
Table 36 Memory Bus Connection	90
Table 37 GIO port selection output	97
Table 38 Memory Space	100
Table 39 ARM Memory Map	103
Table 40 External HOST Memory Map	103

1 Features and Performance Target

The DM270 is a highly integrated, programmable platform for the digital still camera and mobile system. Designed to offer camera manufacturers the ability to produce affordable DSC products with high picture quality, the DM270 combines programmable image processing capability with a highly integrated imaging peripheral set. The chip's programmability comes from a DSP-based imaging co-processor that enables camera manufacturers to implement their own proprietary image processing algorithms in software. The DM270 also enables seamless interface to most additional external devices required for a complete digital camera digital implementation. The interface is flexible enough to support various types of CCD and CMOS sensors, signal conditioning circuits, power management, SDRAM, shutter, Iris and auto-focus motor controls. The DM270 will allow camera manufacturers to meet customer demands by fulfilling both image quality and low cost expectations required in the volume point and shoot digital still camera segment.

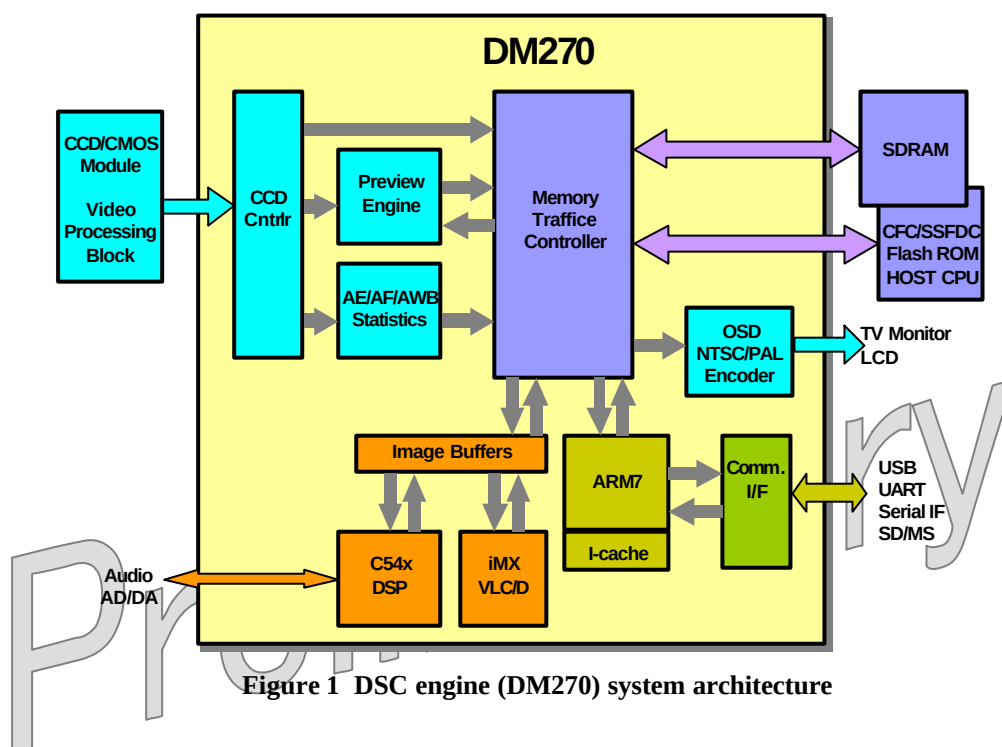


Figure 1 DSC engine (DM270) system architecture

1.1 Key features:

- ❑ 288-pin micro star BGA(μ *BGA) 16x16mm (0.8mm pitch)
- ❑ Supports 120 MHz SDRAM up to 64Mbytes with 32/16-bit interface
- ❑ Supports up to 14-bit AFE (Analog Front End)
- ❑ Supports 8/16bit CCIR601/656 video format
- ❑ Supports Motion JPEG compression/decompression video at VGA resolution up to 30fps
- ❑ Supports MPEG4 I+P frame video at CIF resolution up to 30fps
- ❑ On-chip 90MHz ARM7 32 bit RISC microprocessor with I-cache (8 Kbytes) and on-chip program/data memory (32 Kbytes)
- ❑ On-chip 100MHz Texas Instruments C54X Digital Signal Processor(DSP) core with on-chip program/data memory (128Kbytes)
- ❑ On-chip image processing buffers (2x 8Kbytes)
- ❑ On-chip 180MHz programmable SIMD image processing engine (iMX) for programmable image processing
- ❑ Variable Length Coding and Decoding coprocessor (VLC/VLD) for image/video compression and decompression.
- ❑ Preview engine for real-time live-view and digital zoom
- ❑ Versatile On Screen Display(OSD)
- ❑ Digital interface for color LCD
- ❑ Seamless interface Compact Flash, Smart Media, SD, and Memory Stick cards
- ❑ Supports flash memory up to 16MB
- ❑ Supports 27MHz, 48MHz (USB) crystal input to generate all clocks including.
- ❑ Four Timers + One WDT
- ❑ Two UARTs
- ❑ 1-channel 10-bit DAC for NTSC/PAL composite video output
- ❑ 2-channel serial audio codec interface (McBSP)
- ❑ Two sets of SPI serial interface with master/slave mode
- ❑ Up to 34 General-purpose IO pins available
- ❑ Limited I²C bus master capability
- ❑ On-chip USB 1.1 Function Controller
- ❑ 2 channel programmable PWM (pulse width modulation) outputs.
- ❑ Externally downloadable using SIF (serial interface) or HOST bus interface.

Preliminary

2 Block Diagram

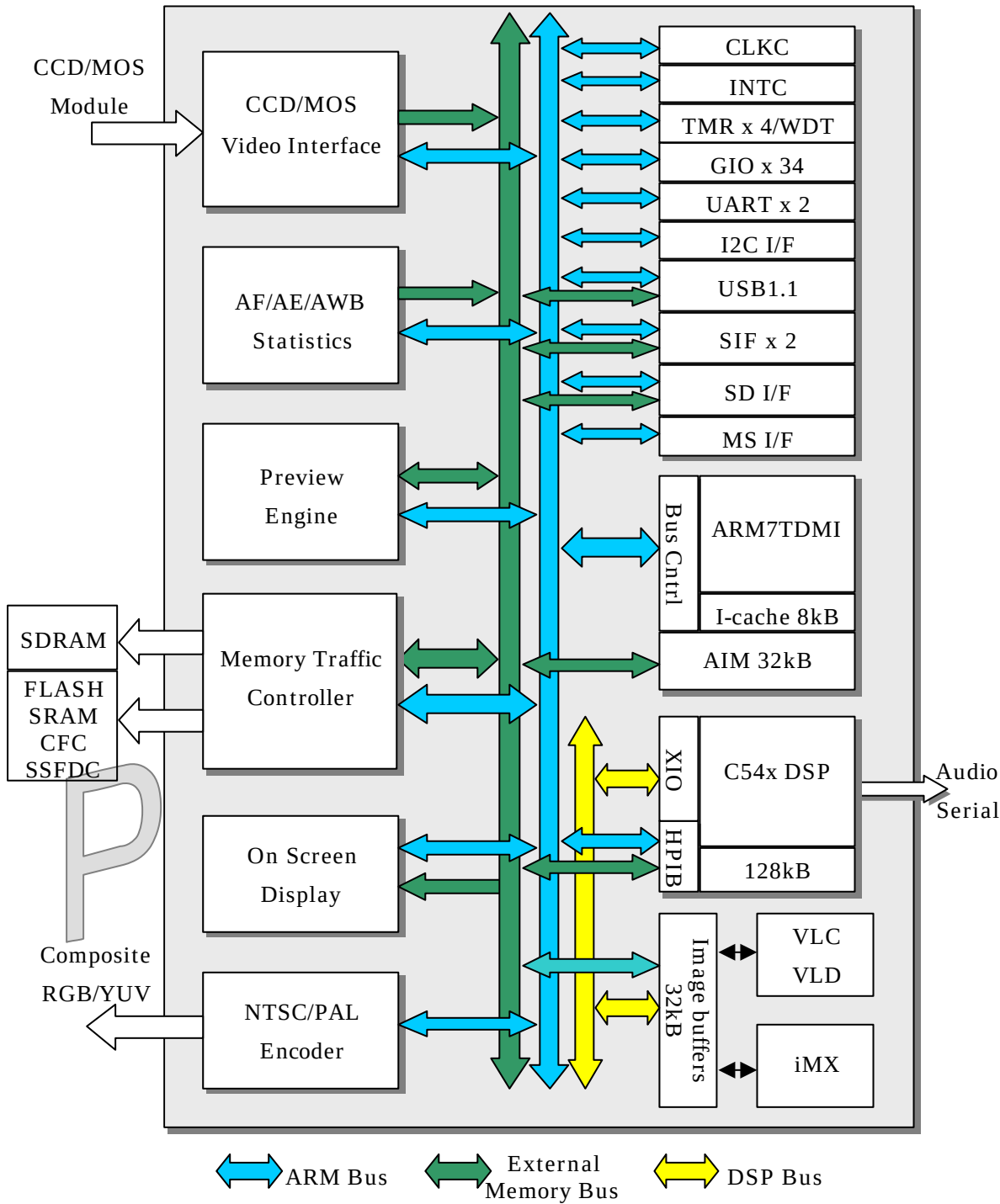


Figure 2 Function Block Diagram

3 System Data Flows

3.1 Preview Mode

In the preview mode, CCD or CMOS imager data comes in through the CCD controller to the preview engine. CCD controller has fault pixel correction, optical black clamping, and required timing control and interface. Preview engine contains a simplified image processing pipeline for AE/AWB, CFA interpolation, down sampling, and gamma lookups. The CCD/CMOS sensor should be configured to draft mode output, and preview engine handles further down-sampling, if necessary, to produce LCD/video resolution. Preview output is written to SDRAM.

DM270 has hardwired 3A (AF/AE/AWB) calculator. Each result are stored in SDRAM, ARM can read its data and controls AFE.

The display data is read out by the OSD module, and merged/blended with the OSD data. The display data is sent to video encoder (VENC) or LCD controller, which sends composite analog or digital LCD data to the external display device.

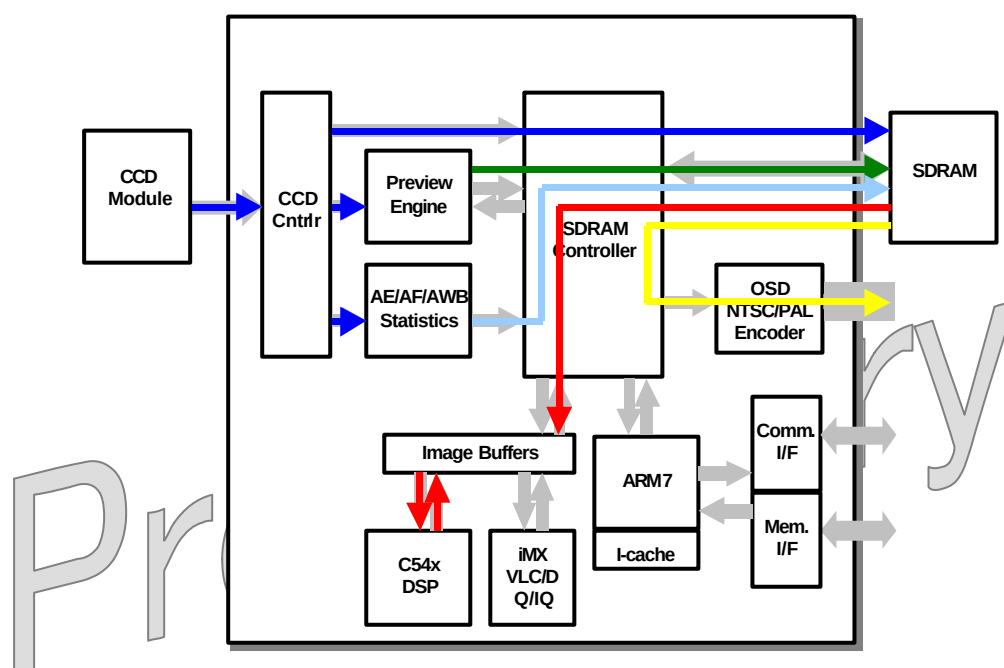


Figure 3 Live View mode

3.2 Still Picture Capture Mode

In the still picture capture mode, CCD or CMOS imager data comes in through the CCD controller and SDRAM controller directly to SDRAM first.

When there is a sufficient amount of data to start processing, which could be 20-24 lines with non-interlaced data and slightly over half-frame for interlaced data, ARM is notified via an interrupt signal from CCD controller. ARM then coordinates with iMX, VLCD, and DMA controller to transfer in the data from SDRAM into image buffer and process the data. The processing flow is software programmable and customer-specific. Typically compressed JPEG bitstream is the final output, and it's written back into SDRAM.

ARM then handles media storage card file system setup, sector header setup, and triggers DMA to write data from SDRAM into the card.

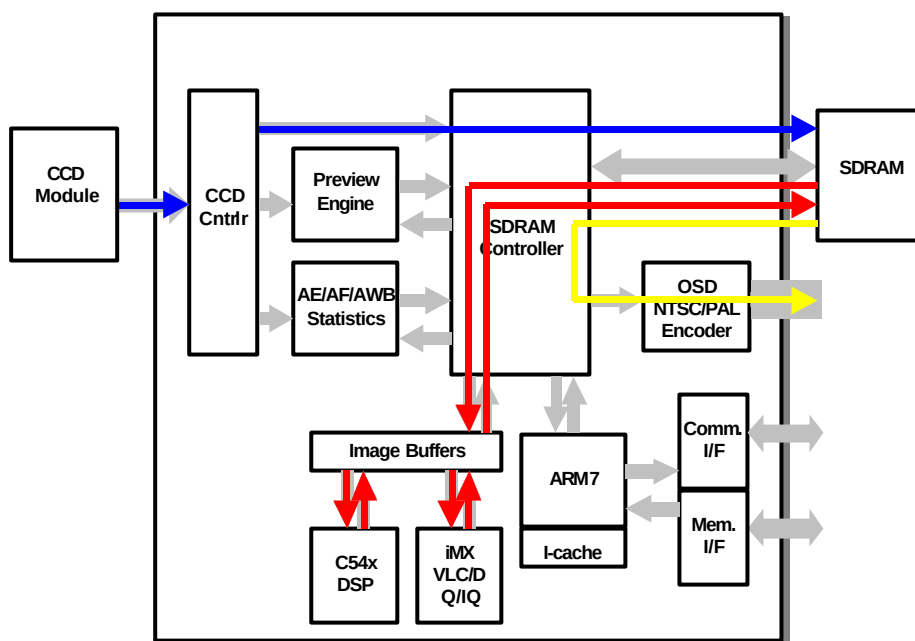


Figure 4 Still Picture Capture mode

3.3 Video Capture Mode

Video capture is similar to still picture capture, except for the input stage. For video capture, we use Preview Engine to down sample the CCD/CMOS imager data to the video capture size (VGA, CIF, QCIF etc) before writing into SDRAM. This offloads much processing in the programmable engines and reduces power consumption due to processing and traffic.

Once sufficient amount of the video data is written into SDRAM, again ARM gets an interrupt from CCD controller and triggers the processing steps. MPEG-1 or MPEG-4 compression is performed with the coordination of ARM, iMX, VLCD and the DMA controller. The compressed MPEG bitstream is written back to SDRAM.

The video data produced by Preview Engine are also pulled out by OSD module to produce display data for composite analog video or for digital LCD interface.

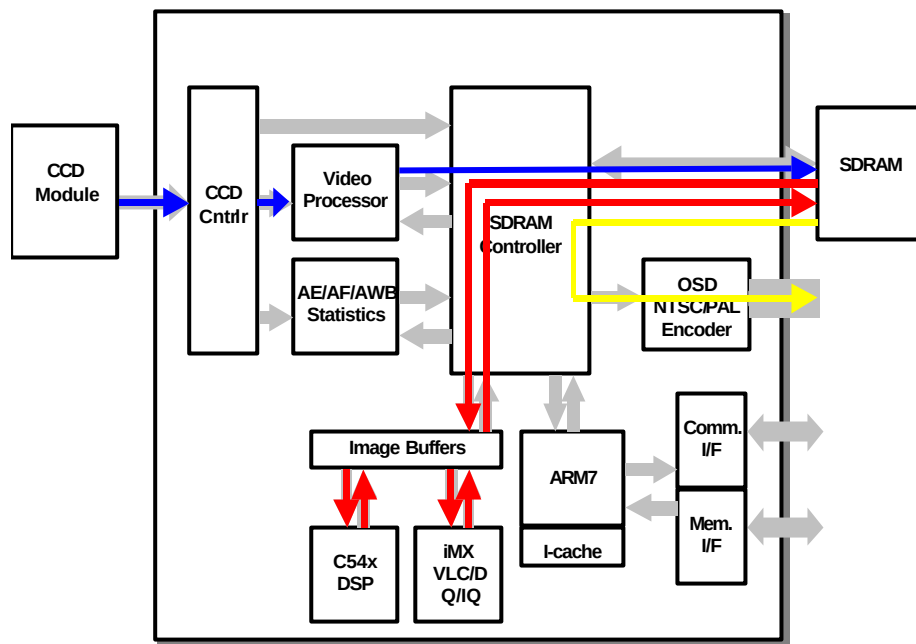


Figure 5 Motion Picture Capture mode

3.4 Still Picture and Video Playback

Still pictures or video clips can be played back on the digital camera. Typically the compressed data is first read out of the media card into SDRAM. Next the compressed data is processed and transported by ARM, coprocessors and DMA to uncompressed and, if necessary, resized into display resolution, and stored back into SDRAM.

The display data is then pulled out by OSD to supply VENC or LCD controller for analog composite video or digital LCD interface.

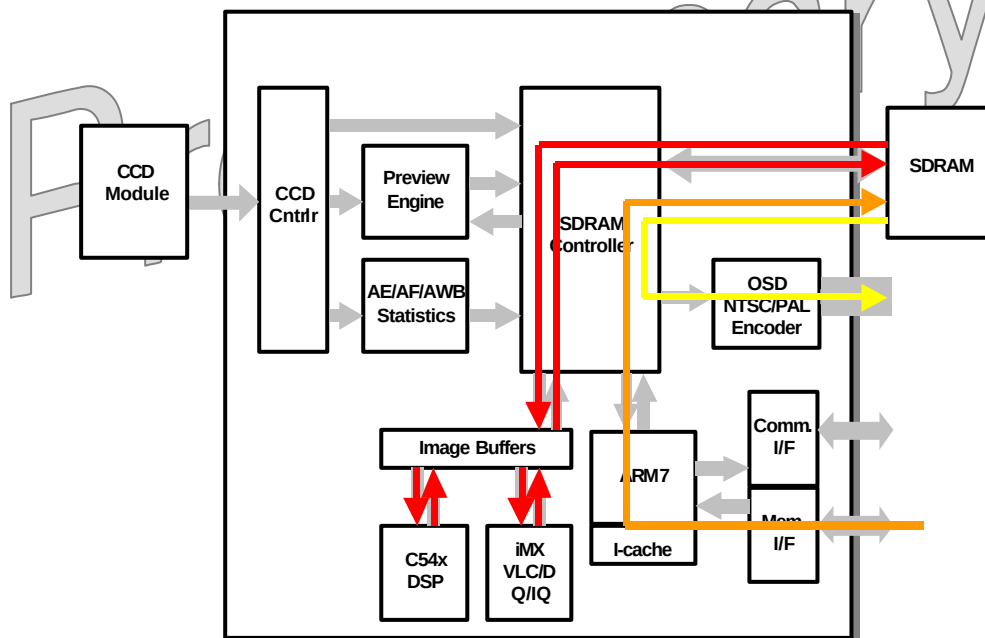


Figure 6 Play Back mode

4 Imaging Peripherals

4.1 CCD Controller

The CCD Controller captures and processes the physical data from the CCD sensor. The A/D converter and Timing Generator for CCD sensor are external to DM270 chip. The CCD controller can provide a HD/VD to external timing generator or synchronize to HD/VD given from external timing generator. The other function of CCD controller is a digital clamping and decimation for physical image capturing.

4.1.1 Features

- ❑ Generates timing signals VD/HD and field ID
- ❑ Can synchronize on externally generated HD/VD signals
- ❑ Supports progressive scan and interlaced CCD
- ❑ Generates optical black clamping control signals
- ❑ Programmable culling pattern
- ❑ Maximal supported pixel clock is 40 MHz
- ❑ Maximal supported CCD imager size 4096 X 4096
- ❑ Shutter signal control
- ❑ Digital clamping
- ❑ Black level compensation
- ❑ Median filtering
- ❑ Gain and offset control
- ❑ 10-to-8 bit A-law compression table inside
- ❑ Supports CCIR601/656 Digital YCbCr 4:2:2 (8/16bit) interface

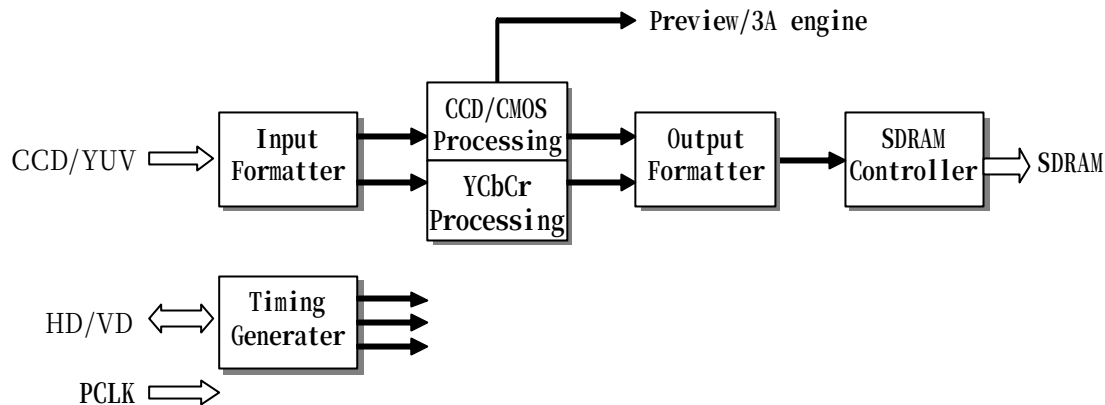


Figure 7 CCD controller Block Diagram

4.1.2 Signal Interface

The CCD Controller accepts pixel data via the CI7-CI0, YI7-YI0 input pins. The definition of each pixel input is determined by the input format selected in the input formatter. The definition for each mode is shown in Table 1. The interface consists of a set of signals used to transfer raw CCD data from an imager to the CCD Controller. Additionally, the CCD controller can be configured to operate in a mode that adheres to the CCIR 601/656 interface specification. The CCIR 601/656 specification provides a standard method, to transfer 8/16-bit, YCbCr-4:2:2 formatted video data.

Table 1 Data Input Formats

Pin Name	CCD	16-bit YCbCr	8-bit YCbCr
CI7		Cb7,Cr7	Y7,Cb7,Cr7
CI6		Cb6,Cr6	Y6,Cb6,Cr6
CI5	C_DATA13	Cb5,Cr5	Y5,Cb5,Cr5
CI4	C_DATA12	Cb4,Cr4	Y4,Cb4,Cr4
CI3	C_DATA11	Cb3,Cr3	Y3,Cb3,Cr3
CI2	C_DATA10	Cb2,Cr2	Y2,Cb2,Cr2
CI1	C_DATA9	Cb1,Cr1	Y1,Cb1,Cr1
CI0	C_DATA8	Cb0,Cr0	Y0,Cb0,Cr0
YI7	C_DATA7	Y7	Y7,Cb7,Cr7
YI6	C_DATA6	Y6	Y6,Cb6,Cr6
YI5	C_DATA5	Y5	Y5,Cb5,Cr5
YI4	C_DATA4	Y4	Y4,Cb4,Cr4
YI3	C_DATA3	Y3	Y3,Cb3,Cr3
YI2	C_DATA2	Y2	Y2,Cb2,Cr2
YI1	C_DATA1	Y1	Y1,Cb1,Cr1
YI0	C_DATA0	Y0	Y0,Cb0,Cr0

The CCDC supports 8 to 14-bit wide data signals. When the CCD has fewer than 14 data lines, connect the CCD to the lower or upper data lines C_DATA[13]-C_DATA[0], and ground the upper or lower, unused lines.

4.1.3 The Typical CCDC Interface

Timing and control of the interface is done using the HD, VD, field ID, pixel clock, and write enable signals. As shown in Table 2, the CCDC controller can either provide HD, VD, and field signals to the CCD imager, or it can use the HD, VD, and field ID signals provided by the CCD imager. The pixel clock clocks data into the CCDC at a maximum rate of 40MHz. When the CCDC is configured to write data to SDRAM, the write enable signal allows an external device to control whether-or-not the current data will be written to SDRAM.

Table 2 CCDC Signal Interface

Name	I/O	Function
C_DATA[13]-C_DATA[0]	I	Image data loaded by CCD. Bit width can be configured to 8 to 14 bits. The polarity of the input image data can be reversed.
C_VSYNC (VD)	I/O	VSYSN. Vertical synch signal. This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the VD signal. When configured as an output the DM270 will supply the VD signal. The polarity of VD can be reversed.
C_HSYNC (HD)	I/O	HSYSN. Horizontal synch signal. This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the HD signal. When configured as an output the DM270 will supply the HD signal. The polarity of HD can be reversed.
C_FIELD	I/O	Field identification signal. This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the field identification signal. When configured as an output the DM270 will supply the field identification signal. When the field identification signal is set to be an input to the DM270, this signal can be configured to be latched by the VD signal. The polarity of the field identification signal can be reversed.
C_PCLK	I	Pixel clock. This signal is the pixel clock used to load image data into the CCDC. The Clock controller can configure to trigger on the rising or falling edge of the PCLK signal. The maximum pixel clock rate is 40MHz.

4.1.4 CCD Controller Processing

As shown in Figure 8, the CCDC consists of several data processing modules that format and enhance raw CCD data. There are two destinations for the processed output data, SDRAM and the Preview Engine. The following sections describe the modules that perform the following functions.

- ❑ Digital clamping
- ❑ Black level compensation
- ❑ Median Filtering
- ❑ Gain & Offset
- ❑ Gamma Correction
- ❑ Output Formatter

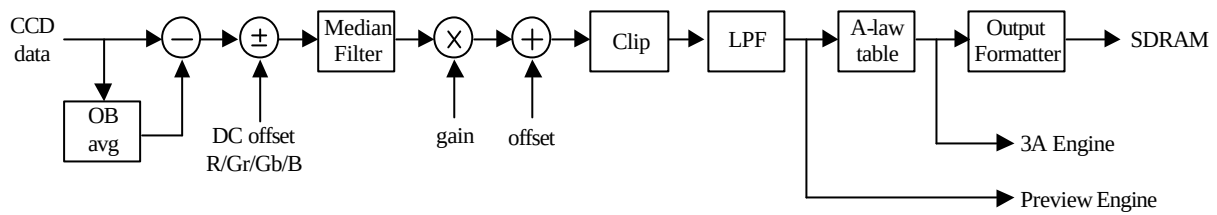


Figure 8 CCD signal processing

4.1.5 Digital Clamp

The first sub-module is the digital clamp block. The image data is latched at the rising or falling edge of the input pixel clock (PCLK). The averaging circuit takes an average of masked (black) pixel values from the image sensor and this value is subtracted from the image data. The user controls the position of the black pixels, the number of pixels (8 or 16) in each line averaged, and the number of lines (8 or 16) averaged.

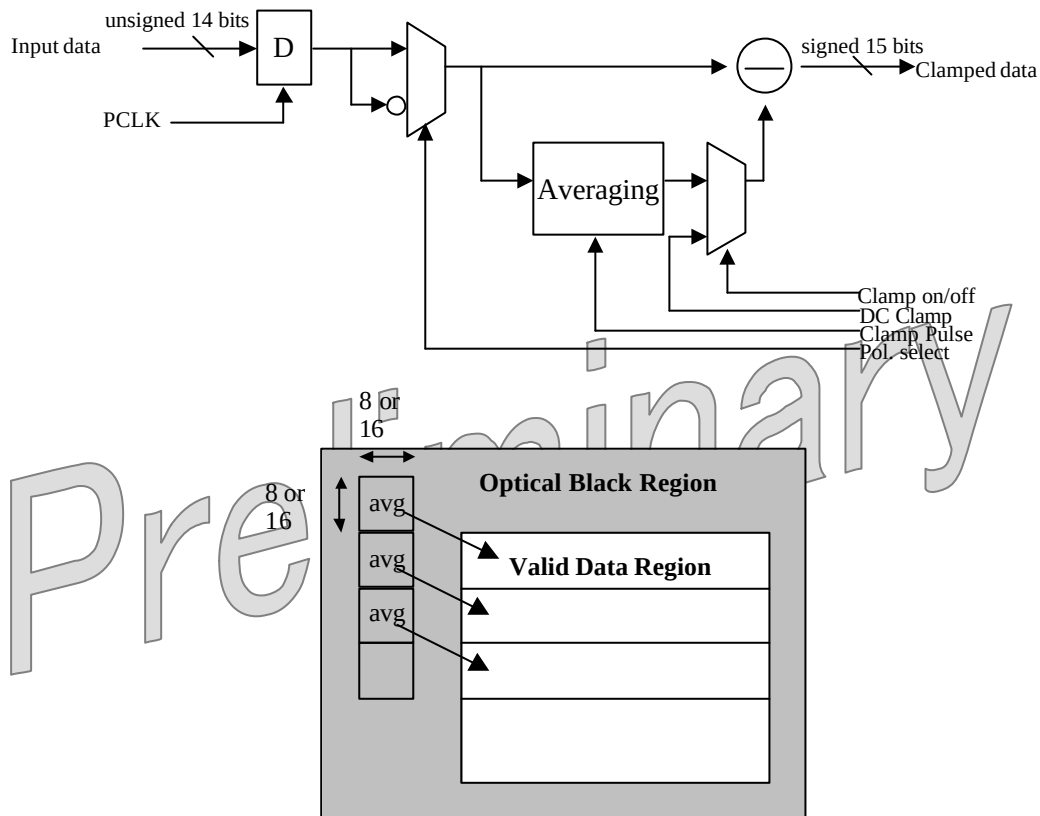


Figure 9 Digital Clamp Block Diagram

Alternately, the user can disable black clamp averaging and select a constant black value for subtraction, instead of using the calculated average value for subtraction.

4.1.6 Black Level Compensation

After the Digital Clamp is applied to the data, Black Level Compensation is applied to the R/Ye, Gr/Cy, Gb/G, and B/Mg signals. The offset applied to the current data sample is selected according to the even/odd phase and the color specified for each phase.

4.1.7 Median Filter

The Median Filter, shown in Figure 10 calculates the absolute difference between the current pixel (i) and pixel (i-2) and between the current pixel (i) and pixel (i+2). If the absolute differences exceeds a threshold, the average of pixel (i-2) and pixel (i+2) replaces pixel (i). A line of CCD data is input into the Media Filter, so by operating on pixels (i-2), (i), and (i+2) the filter is applied to like pixels in a CCD data CFA pattern. The Median Filter's threshold is configurable and the Median Filter can be enabled or disabled.

```
for (i=0, i< length_pixel, i++){
    sub1 = input_data(i) - input_data(i-2);
    sub2 = input_data(i) - input_data(i+2);
    if ((abs(sub1) > THRESHOLD && abs(sub2) > THRESHOLD) &&
        ((sub1<0 && sub2<0) || (sub1>0 && sub2>0)))
        output_data(i) = (input_data(i-2) + input_data(i+2))/2;
    else
        output_data(i) = input_data(i);
}
```

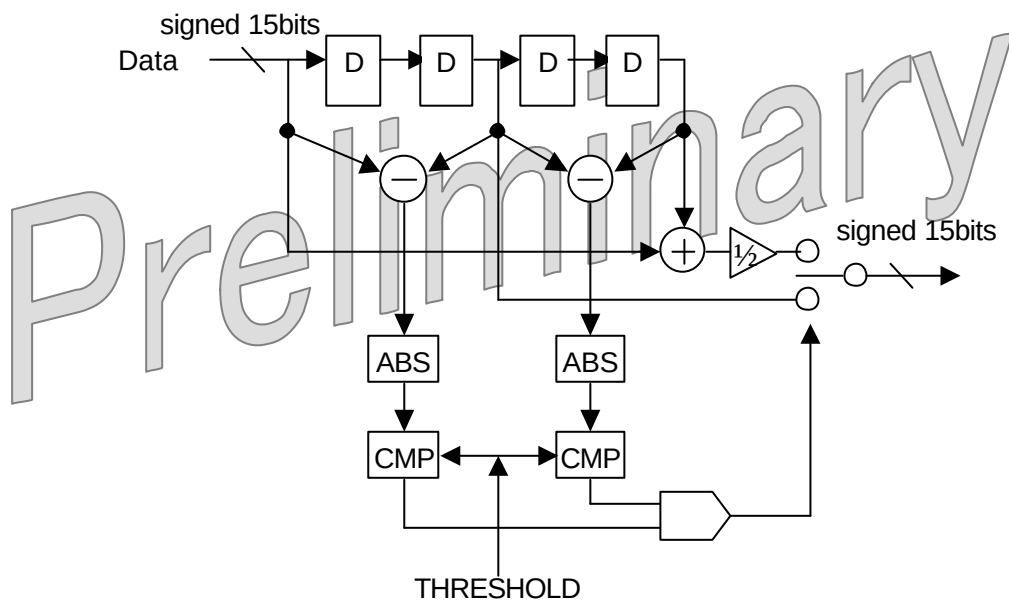


Figure 10 Median Filter

4.1.8 Gain and Offset

A gain factor from 0 to 7.9921875 can be applied to the output of the Median Filter. After multiplying the data by the gain factor an offset can be added to the result. The offset can range from 0 to 1023. The result CCD data of gain control is clipped to signed 15bit data, and finally clipped to unsigned 14bit data after offset control.

4.1.9 Output Formatter

The output formatter block provides options for applying an anti-aliasing filter for horizontal culling. The low-pass filter consists of a simple three-tap ($\frac{1}{4}$, $\frac{1}{2}$, $\frac{1}{4}$) filter. In the data compression pass, any 10 bits of 14bit CCD data are compressed to 8 bits and every four pixels are stored to SDRAM at the same time. This format can secure the less SDRAM capacity. The A-law table has a same characteristic as Voice codec.

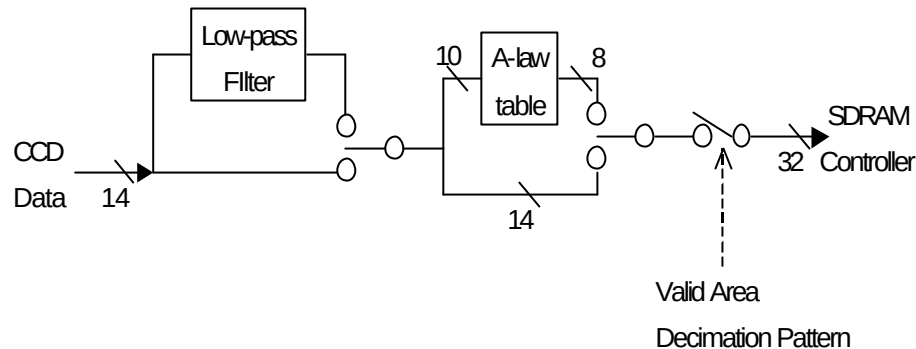


Figure 11 Output Formatter Block Diagram

The final stage is a programmable decimation function along both the horizontal and vertical directions.

Two registers control the horizontal and vertical decimation of CCD Data. The 16-bit CULH register sets the horizontal culling for even and odd lines. The 8-bit CULV register sets the one along the vertical direction. The LSB of CULV represent the top line of the CCD, the MSB is the 8th line. The bits of data from each pixel are stored in the upper bits of a 16-bit SDRAM word, and the unused bit positions are filled with zeros. The SDRAM data format is shown in Table 3.

Table 3 SDRAM Data Format(CCD)

	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
14bit	Pixel0	0	Pixel1	0
13bit	Pixel0	0	Pixel1	0
12bit	Pixel0	0	Pixel1	0
11bit	Pixel0	0	Pixel1	0
10bit	Pixel0	0	Pixel1	0
9bit	Pixel0	0	Pixel1	0
8bit	Pixel0	0	Pixel1	0
8bit pack	Pixel0	Pixel1	Pixel2	Pixel3

The processed CCD data is transferred to the SDRC in units of 32 bytes. The CCDC has an internal, 16-bit by 16-bytes FIFO. Data is transferred from the FIFO to the SDRC in 32-byte bursts.

Data is only written to SDRAM when WEN in SYNCEN is set to "1".

Furthermore, the output formatter can configure to any image format by using SDRAM line offset register, and offset control registers.

Figure 12 shows how to construct a frame format in SDRAM.

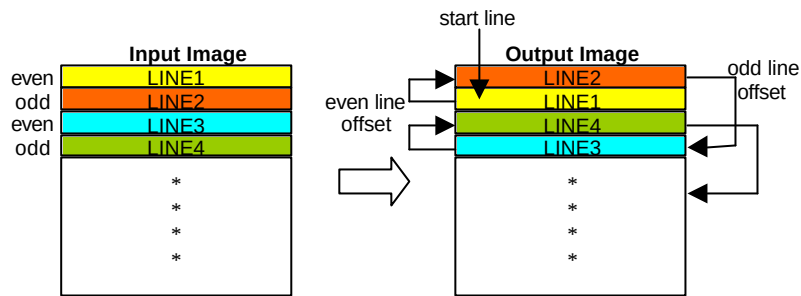
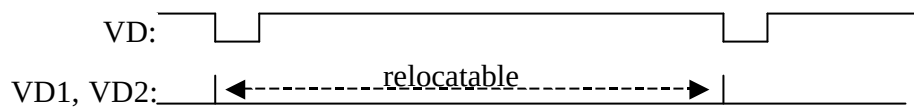


Figure 12 Frame Image format conversion

4.1.10 CCDC Interrupts

The CCDC can generate two different interrupts: VD1 and VD2. The VD1 & VD2 interrupts can be set at any position in VD interval.

CCDC interrupts



4.1.11 Shutter Control

The CCD Shutter signal is synchronized with the VD2 interrupt. The CCD Shutter signal is immediately set high when VD2 becomes active. The ARM can set the CCD Shutter signal low by writing to the SHUT register. The CCD Shutter signal is connected to one of GPIO pins, so the CCD Shutter signal can be connected to an imager's external shutter control logic through GPIO.



4.1.12 Timing Generator

The timing generator provides the use of external sync signals (HD/VD) or internal generated timing signals. The ARM controls width, polarity, position and direction of internal generated signals. 0 shows various CCDC register settings related to the timing. The shaded area is the physical imager size and the gray area is the valid data area. The image data in this area will be processed and stored to external SDRAM. The vertical start position for even and odd fields can be configured independently.

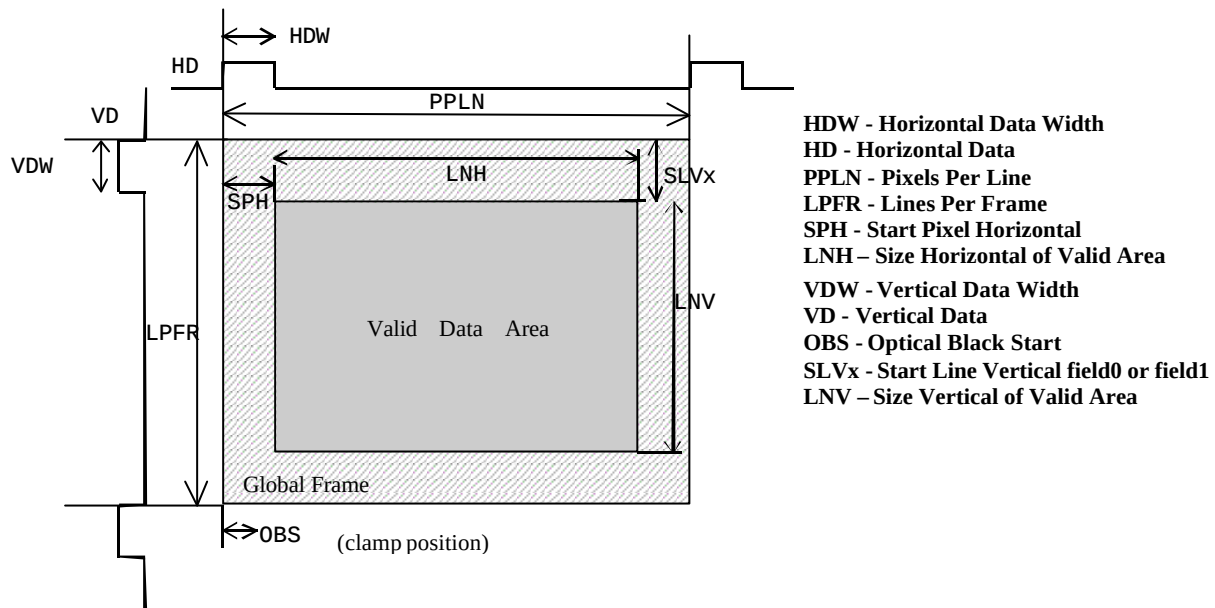


Figure 13 Frame Image Format

4.1.13 YCbCr signal Processing

Figure 14 shows a YCbCr signal processing flow in the CCD controller block. The CCD controller accepts 4:2:2 sampled YCbCr input data. The luminance and color difference signals are each 8 bits, scaled 0 to 255. The color difference signals are multiplexed into one 8-bit bus beginning with a Cb sample. The Y and CbCr busses may be input parallel (16-bit mode) or may be time multiplexed and input as a single bus (8-bit mode). The single bus may also contain SAV and EAV video timing reference codes (ITU-R BT.656 mode). In CCIR656 mode, the CCD controller is controlled by the SAV (Start Active Video) time code in the 8-bit pixel data. All timing information is transmitted using a 4-byte synchronization pattern. A synchronization pattern is sent immediately before and after each line during active picture and retrace. DM270 decodes SAV code and synchronized at this timing.

The 16-bit or 8-bit YCbCr data is stored in SDRAM as 4:2:2 format. Table 4 shows data format in SDRAM. Y data typically has a range of 16 to 235, however, it is possible to subscribe a DC value to Y signal.

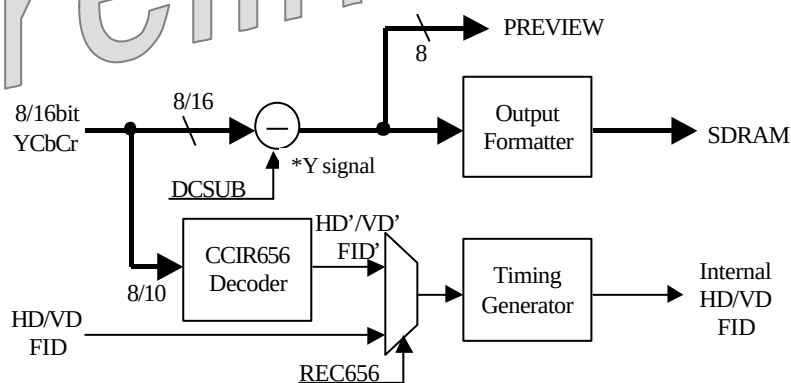


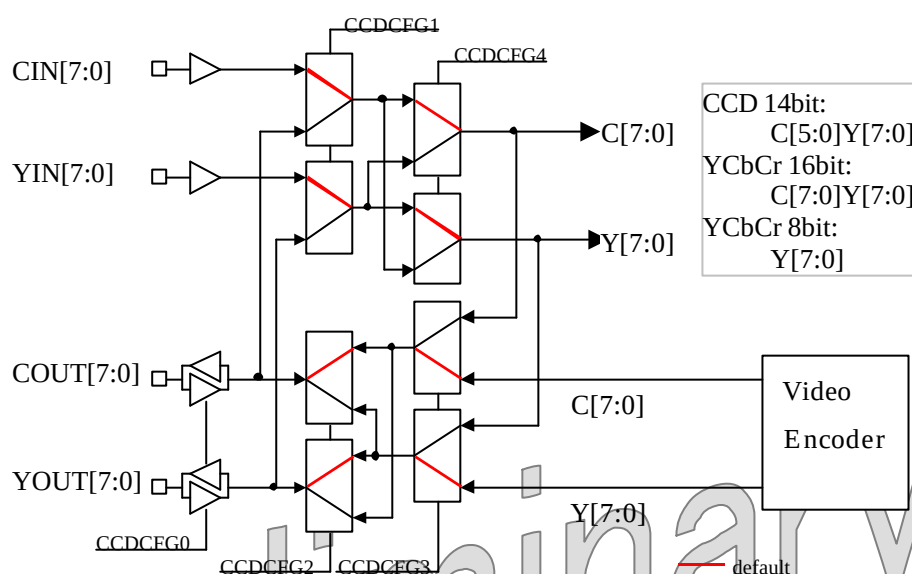
Figure 14 YCbCr signal processing

Table 4 SDRAM Data Format(YCbCr)

SDRAM Address	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
N	Cb0	Y0	Cr0	Y1
N+1	Cb2	Y2	Cr2	Y3
N+2	Cb4	Y4	Cr4	Y5

4.1.14 Input/Output port configuration

DM270 allows to have several port configuration as shown in Figure 15. Byte swapping of input/output(see Table 12 Video Encoder Signal Interface) port or input data through mode is configurable by internal CPU registers. Figure 16 shows a example of system construction.

**Figure 15 Video Port Configuration**

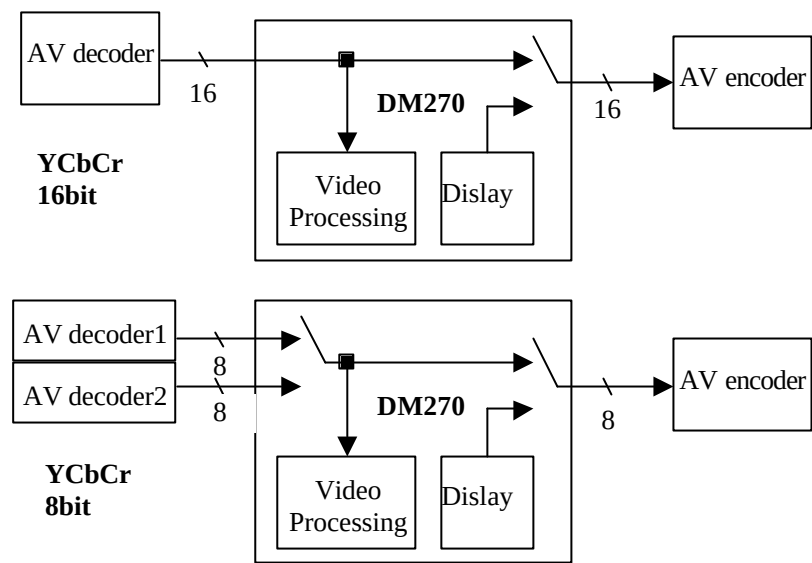


Figure 16 Video System Configuration

Preliminary

4.2 Preview Engine

The Preview Engine is a programmable hardware video processing module that generates image data in YCbCr-4:2:2 format from raw CCD data. The Preview Engine can also be configured to operate in a resize only mode, which allows the raw CCD data to be resized and converted to YCbCr-4:2:2 without applying the processing of every module in the Preview Engine. As shown in Figure 17, several internal modules are used to process CCD data into YCbCr-4:2:2 data. These modules are briefly mentioned in this section and described in detail in the sections that follow.

The input interface module has two input ports, which are connected to the CCD Controller (CCDC) and the SDRAM Controller (SDRC).

The input interface extracts valid raw data from the CCD raw data, and then various modules in the Preview Engine process the raw CCD data.

The noise filter module reduces impulse noise in the raw data.

The white balance module applies two gain adjustments to the data: a digital gain and a white balance gain.

The CFA interpolation module implements CFA interpolation, as well as smoothing and a resizing. The output from the CFA interpolation module is always RGB formatted data regardless of the type of color space used by the CCD.

The black adjustment module independently adds an offset value to each RGB component. The offset adjusts the black level of image data.

The RGB2RGB blending module applies a 3x3 matrix transform to the RGB data generated by the CFA interpolation module.

The gamma correction module independently applies gamma correction to each RGB component. This module can be configured to use one of two gamma tables, which are stored in the Preview Engine's internal ROM.

The RGB2YCbCr conversion module applies 3x3 matrix transformation to the RGB data to convert it to YCbCr data. This module also implements offset, contrast, brightness, and pseudo color suppression.

The 4:2:2 conversion module down samples Cb and Cr, so that the Preview Engine output data is in YCbCr-4:2:2 format.

The output interface module transfers data from the Preview Engine to SDRAM. Data is transferred from the Preview Engine in burst of 16, 32 bit words.

Preview Engine has two different processing paths. Case 1: The CCD raw data directly leads to Preview Engine and stores the YCbCr data to SDRAM. Case 2: The Preview Engine reads CCD raw data from SDRAM and stores the YCbCr data to SDRAM.

4.2.1 Features

- ❑ Real-time hardware processing with complete software control by the ARM
- ❑ Gain control for white balancing at each component
- ❑ Horizontal /Vertical noise reduction filter
- ❑ CFA interpolation for good quality CFA interpolation with reduced false color artifacts
- ❑ Programmable coefficients for RGB to YCbCr conversion
- ❑ Separate lookup tables for gamma correction on each of R, G and B components for display
- ❑ Programmable down or up-sampling filter for both horizontal and vertical decimation Range up to x1/64 to x4
- ❑ Support for RGB Bayer and complementary CCD
- ❑ Inverse A-law table capability

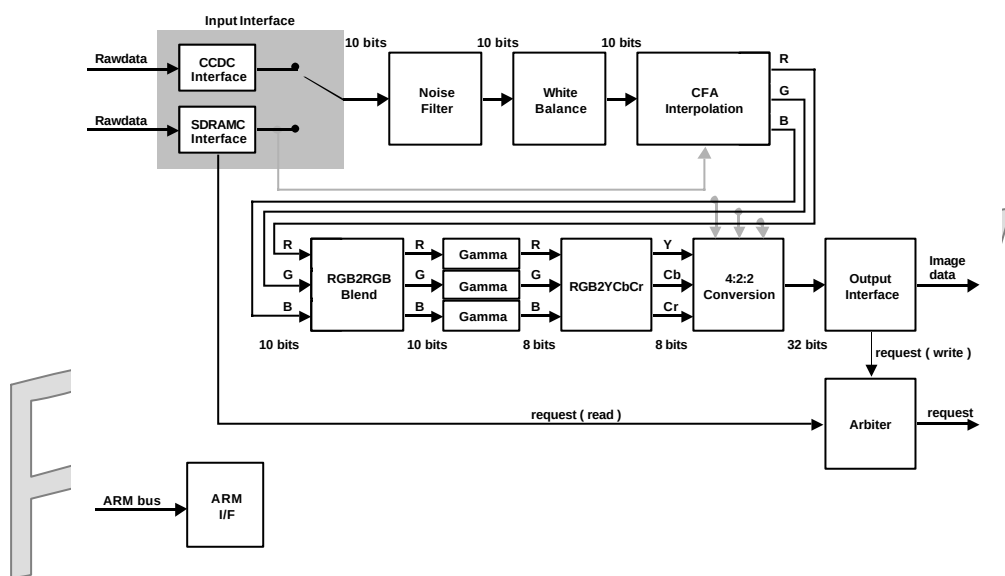


Figure 17 Preview Engine block diagram

4.2.2 CFA Arrangements

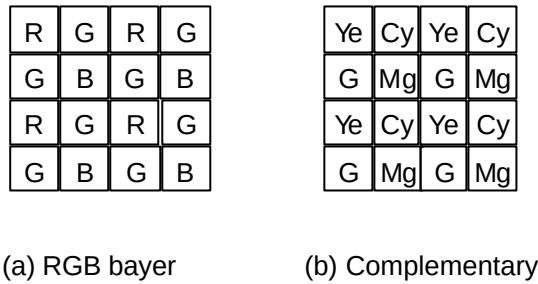


Figure 18 CFA arrangements supported by the Preview Engine

4.2.3 Horizontal noise filter

The noise filter is the first stage of the Preview Engine's image-processing pipeline. The noise filter is a high pass filter that reduces noise in raw CCD data. The filter equation is shown below. In the formula, X_0 , is the current pixel and a is a configurable constant. The noise filter operates on pixels of the same color component, i.e., X_{-2} , X_0 and X_2 . The value of the constant a is defined by the NFRT field in the NFILT register.

$$x_0 = X_0 - a \frac{X_{-2} - 2X_0 + X_2}{4}$$

$$\left(a = \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1 \right)$$

An on/off switching of this filter can be controlled by register setting.

4.2.4 White Balance

White balance module executes digital gain adjustment and white balance to CCD raw data. Digital gain is for adjusting a total brightness of image and white balance for adjusting a ratio of each color existing in a CFA pattern.

Figure 19 shows a block diagram of this white balance module. There are two multipliers for the two gain adjustment and clip circuits to reduce size of circuits. A digital gain named PVGAIN in this figure is used for gain control of CCD raw data. And WBGAIN is for white balance, which is selected automatically by setting register of the CFA pattern.

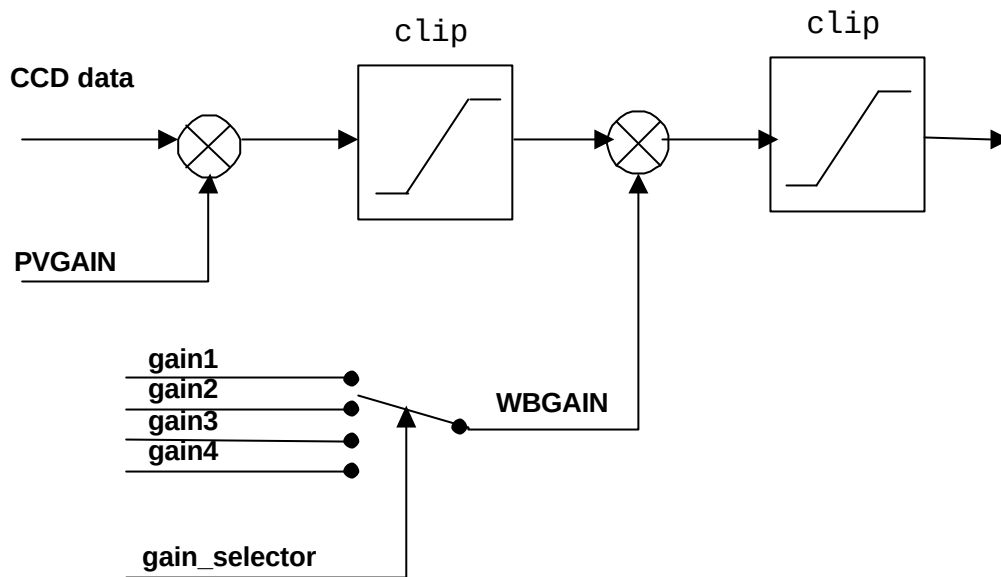


Figure 19 White Balancing in the Preview Engine

4.2.5 CFA Interpolation

The Preview Engine's CFA interpolation module includes many important sub-modules. The sub-modules perform functions such as, horizontal and vertical interpolation, smoothing, horizontal and vertical resizing, color adjustment, and conversion from complementary to RGB. Figure 20 shows a block diagram of the CFA interpolation module. In Figure 17, the data path taken during normal mode is shown by the dark lines, and the data path taken in resize only mode is shown by the light lines.

When the Preview Engine is operating in resize only mode, some of the CFA interpolation modules are bypassed. In resize only mode, only smoothing and horizontal and vertical resizing are applied to the data. The data path taken in resize only mode is shown in shown by the dark lines in Figure 21.

The horizontal interpolation sub-module applies horizontal interpolation using a two tap or a five-tap interpolation filter.

The smoother sub-module applies a low pass filter.

The horizontal resample sub-module applies a seamless horizontal down-sampling or up-sampling (zoom) by x4 over-sampling.

The vertical interpolation and resample sub-module performs a vertical interpolation and a seamless vertical down-sampling or up-sampling by x4 over-sampling using two line memories.

The color selector sub-module extracts the appropriate color from the 4 inputs of the vertical interpolation and resamples sub-module.

The color adjustment sub-module executes color adjustment on the interpolated data. This module is bypassed when the Preview Engine is in RGB CCD mode.

The comp2RGB conversion sub-module converts the data from the complementary color space to the RGB color space. The output of the CFA interpolation module RGB data regardless of a type of the color space of used by the CCD imager.

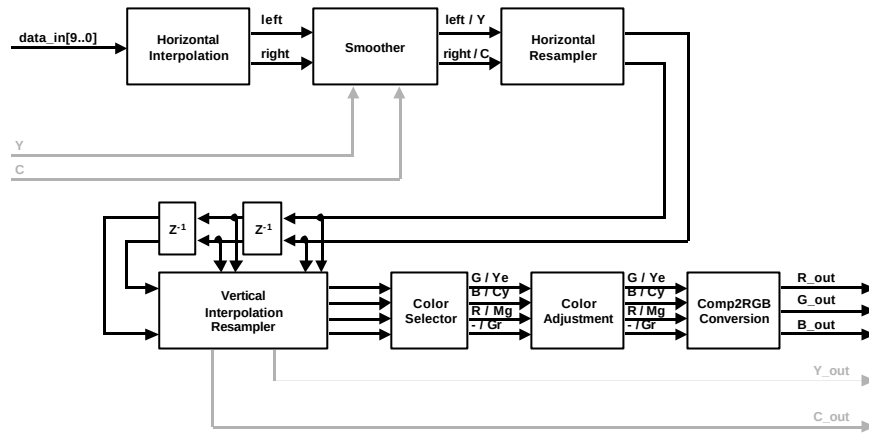


Figure 20 CFA Interpolation Module (Normal Mode)

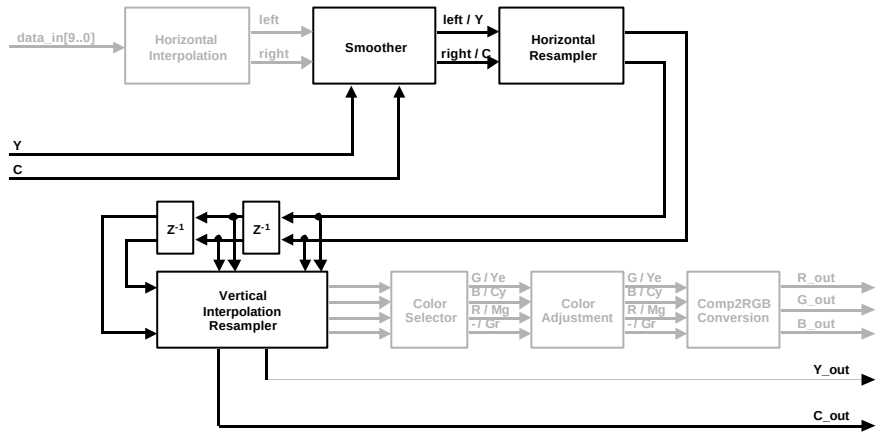


Figure 21 CFA Interpolation Module (Resize Only Mode)

4.2.6 Black Adjustment Sub-Module

After CFA interpolation, black adjustment can be applied to each color component, R, G, and B. A different offset can be applied to each component. The color adjustment sub-module performs the following calculation on each color component.

$$data_out = data_in + bl_offset$$

4.2.7 RGB2RGB Blending Module

The RGB2RGB blending module transforms the RGB data generated by the CFA interpolation module using a 3x3 square matrix transformation in combination with an added offset. The RGB2RGB blending module computes the following calculation.

$$\begin{bmatrix} R_{out} \\ G_{out} \\ B_{out} \end{bmatrix} = \begin{bmatrix} MTX_RR & MTX_GR & MTX_BR \\ MTX_RG & MTX_GG & MTX_BG \\ MTX_RB & MTX_GB & MTX_BB \end{bmatrix} \begin{bmatrix} R_{in} \\ G_{in} \\ B_{in} \end{bmatrix} + \begin{bmatrix} R_offset \\ G_offset \\ B_offset \end{bmatrix}$$

4.2.8 Gamma Correction Module

The gamma correction module applies gamma correction to each color component, R, G, and B. Two gamma tables are stored in the Preview Engine's internal ROM. Set TBSEL in GAMMA to select gamma table 1, and clear TBSEL to select gamma table 0. The gamma tables convert 10bit input data to 8 bit output data. Set the bypass bits (BYPR, BYPG, or BYPB) in GAMMA to bypass the gamma tables. The gamma tables can be bypassed for each color component, independently.

When the gamma table is bypassed the input data is divided by four.

4.2.9 RGB2YCbCr Conversion Matrix

The RGB to YCbCr conversion module converts RGB formatted data to YCbCr formatted data and adjusts Cb and Cr with offset values. The following matrix calculation is used to calculate Y, Cb, and Cr.

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} \text{COEF1} & \text{COEF2} & \text{COEF3} \\ \text{COEF4} & \text{COEF5} & \text{COEF6} \\ \text{COEF7} & \text{COEF8} & \text{COEF9} \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} \text{OFFSET_Y} \\ \text{OFFSET_Cb} \\ \text{OFFSET_Cr} \end{bmatrix}$$

In addition to the color space operation, contrast (scaling/multiplication) and brightness (offset/addition) adjustments can be made on the luminance data and pseudo-color suppression can be performed for chrominance data.

The gain factor applied to Cb and Cr during pseudo-color suppression is determined using a configurable function mapping. The functional mapping is shown in Figure 22. The pseudo-color suppression threshold and gain can be set in the CSUP1 register. Use SUPTH to set the threshold constant and use SUPG to set the gain constant.

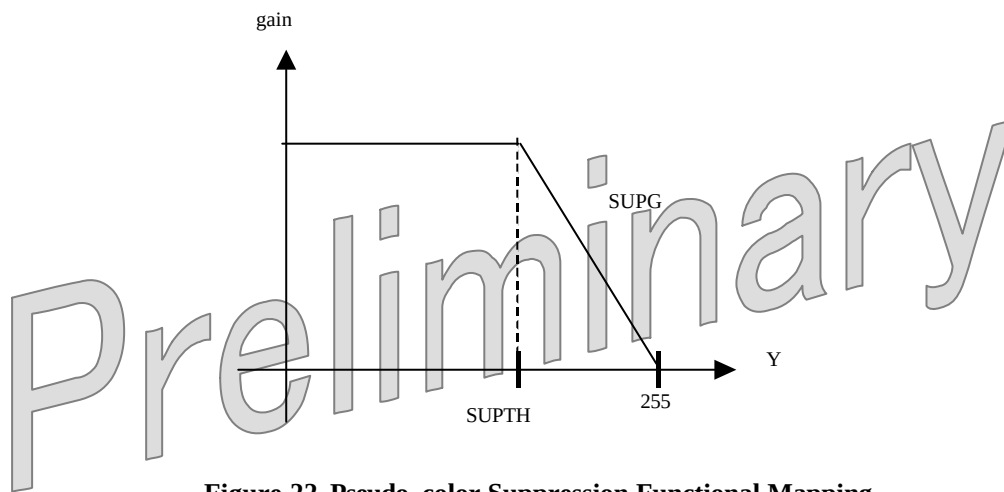


Figure 22 Pseudo-color Suppression Functional Mapping

4.2.10 4:2:2 Conversion and Output Interface Modules

The 4:2:2 conversion module converts the image data to YCbCr-4:2:2 format by discarding every other Cb and Cr component in the horizontal direction. After converting the data to YCbCr-4:2:2, the image data is transferred to the SDRC in units of 32 bytes. The SDRC manages the transfer of the image data to SDRAM. The Preview Engine has an internal 16-bit by 16 FIFO that is used buffer data. Data is transferred from the FIFO to the SDRC in, 32-byte bursts. Table 3 shows how the YCbCr image data is formatted in SDRAM.

Table 5 YCbCr-4:2:2 Format

MSB				LSB			
31	24	23	16	15	8	7	0

Y1	Cr0	Y0	Cb0
----	-----	----	-----

4.2.11 4:2:2 SDRAM Input mode

The preview engine has two selectable input ports, which are connected to the CCD controller (CCDC) and the SDRAM controller (SDRAMC). There is some difference of available functions between in the CCDC input mode and in the SDRAMC input mode. Table 6 shows the main difference of available functions between the two input modes.

Table 6 Comparison of the two input mode

Functions	CCDC input mode		SDRAMC input mode	
	Normal	Resize Only	Normal	Resize Only
Noise filter	O	X	O	X
Digital gain	O	X	O	X
White balance	O	X	O	X
Smoothing	O	O	O	O
Down sample	O	O	O	O
Zoom	X	X	O	O
RGB blend	O	X	O	X
Gamma	O	X	O	X
Chroma suppress	O	X	O	X
Chroma offset	O	X	O	X

Table 7 shows SDRAM data format which preview engine can perform. In 8bit pack mode, the preview engine also has an inverse A-law table function. It can decompress the 8bit data, which is compressed by A-law table in CCD controller. The 8bit input data is extended to 10bit data. In this table Pixel0 data is processed ahead of Pixel1 data.

Table 7 SDRAM Data Format

Lower word				Upper word			
LSB(0)		MSB(15)		LSB(16)		MSB(31)	
Pixel0		0		Pixel1		0	
Pixel0		Pixel1		Pixel2		Pixel3	
Pixel0(Cb0)		Pixel0(Y0)		Pixel0(Cr0)		Pixel1(Y1)	

10bit
8bit pack
YCbCr

4.3 AF/AE/AWB Statistics Engine

DM270 provides real-time 3A (AE/AWB/AF) statistics function. 3A statistics function can be performed either before WB or after gamma conversion. When 3A statistics calculation is completed, an interrupt will be generated to inform ARM of its completion. ARM then uses the statistics to compute the optimal 3A control parameters.

AE/AWB statistics is calculated using 192 windows. The windows are the same size and organized as an m-by-n matrix. The organization of the matrix, the size of the windows and the method of calculation are user-selectable.

AF statistics is calculated by up to 36 windows. The position and the size of each window can be independently set by software.

4.3.1 Features

1. AE, AWB
 - a. Total number of windows is 192.
 - b. The base position for AE/AWB statistics window matrix can be configured by software
 - c. Number of windows in horizontal/vertical direction is from 1 to 12(H),8(V)
 - d. Total number of pixels in horizontal/vertical direction is from 2 to 256(H),64(V)
 - e. Sampling step for calculation in horizontal/vertical direction is from 2 to 32. Total number of 2x2 blocks in a window is up to 256.
 - f. Outputs for each window are the accumulated R, Gr, Gb and B. The results are 16-bit integer stored in SDRAM readable by ARM
2. AF
 - a. Total number of windows is up to 36
 - b. The calculation can be performed on Green pixels only data in the window.
 - c. The position, size of each window can be independently set by software.
 - d. Two biquad filters are used to filter the green or Y pixel data. The coefficients of both filters are 10-bit wide and can be set by software.

Preliminary

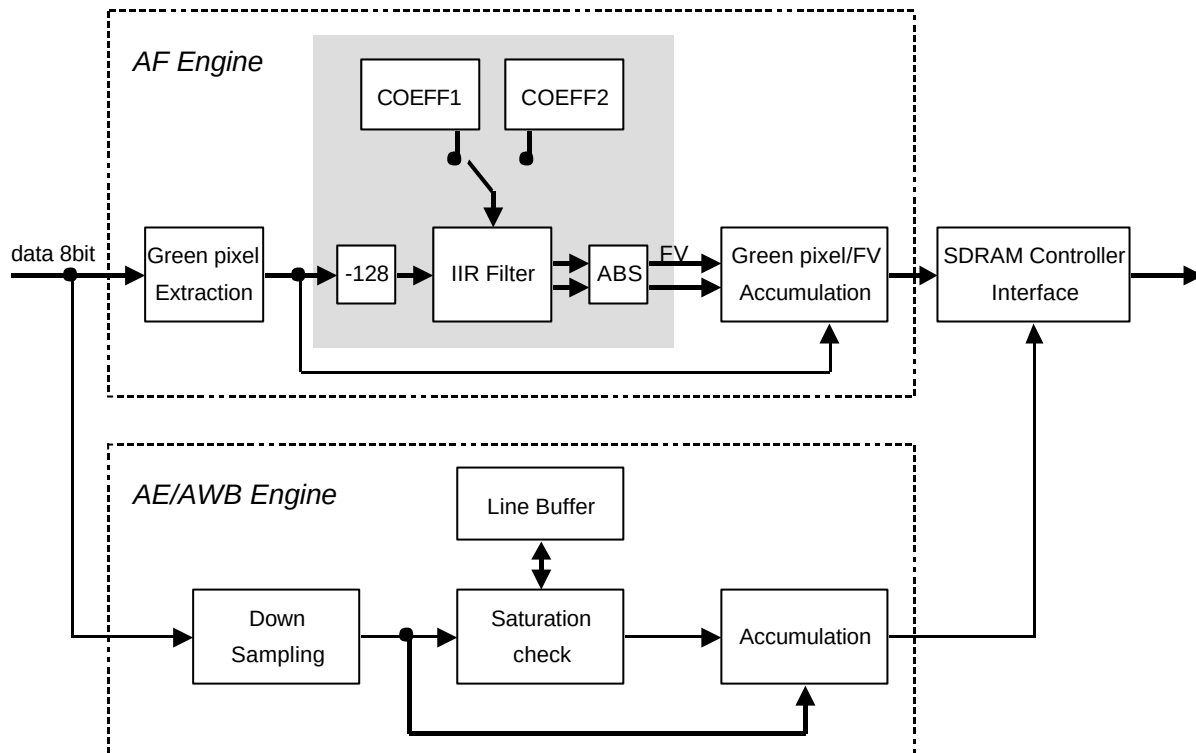


Figure 23 Block Diagram of 3A Hardware Engine

AF Engine

- Green pixel accumulation for each Poxel.
- Focus Value (FV) accumulation for each Poxel.

[Peak mode] Accumulate the maximum FV of each line in a Poxel.
Accumulate the peak position of each line in a Poxel.

[Sum mode] Accumulate FV in a Poxel.

AE/AWB Engine

- Accumulate the pixels in the sub-sampled 2x2 blocks for each Window (AVE1).
- Accumulate the pixels which are clipped to the limit value in the sub-sampled 2x2 blocks (AVE2).
- Count the number of the sub-sampled 2x2 blocks which are not saturating (CNT2).

4.3.2 AF Engine

Extracted green data are subtracted by 128, and fed to the filter block to calculate the Focus Value, which is the absolute value of the IIR filter output. The maximum FV of each line in a Poxel is acquired if FV mode is set to 'Peak mode'. Value of the green pixels and either the FV or the maximum FV are accumulated in the Poxel, and are stored in SDRAM.

The window configuration of the AF Engine is based on a frame formed by the extracted green pixels.

Table 8 AF Engine Configuration Registers

Register	width	Description
AFEN	1	AF Engine enable 0:disable 1:enable

GPOS	4	Green pixel position in 2x2 block
PAXW	8	Paxel Width (PAXW+1) (1 – 256)
PAXH	6	Paxel Height (PAXH+1) (1 – 64)
PAXHC	4	Paxel Count for H direction (PAXHC+1) (1 – 9)
PAXVC	2	Paxel Count for V direction (PAXVC+1) (1 – 4)
PAXSH	12	Paxel start position (H) (0 – 4,095)
PAXSV	12	Paxel start position (V) (0 – 4,095)
IIRSH	12	IIR filter start position (H) (0 – 4,095)
AFINCV	4	Line increments in a Paxel (AFINCV+1) (1-16)
FVMODE	1	Focus Value accumulation mode 0:Sum mode 1:Peak mode
COEFF1	11x11	Coefficient table1 for IIR filter (decimal 6bit)
COEFF2	11x11	Coefficient table2 for IIR filter (decimal 6bit)
AFSDRA	22	SDRAM destination address to store the results of AF engine. It is automatically incremented until the two continuous buffers for the two successive frames become full.
AFSDRFLG	1	SDRAM buffer flag to indicate the buffer at which the results are stored.

- PAXSH must be equal to or greater than (IIRSH + 1).
- Paxels cannot overlap the last pixel in a line. PAXSH, PAXW and PAXHC must be set accordingly.
- Paxels can overlap either the first or the last line in a frame, but not both of them. PAXSV, PAXH and PAXVC must be set accordingly.

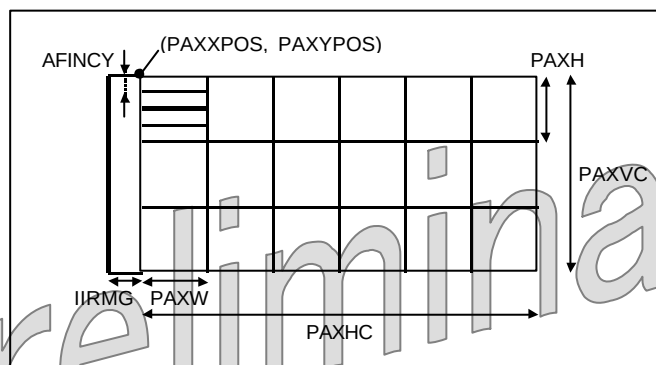


Figure 24 Paxel Configuration

IIR Filter

There are 2 sets of programmable coefficient tables. FV calculation is performed on both tables. Shift registers of the IIR filter are cleared to zero once in a line, at the next position of IIRSH.

Filter Input	[Value of a green pixel (8bit)] – 128	$-128 \leq \text{filter_in} \leq 127$
Coefficient	Signed 11bit (decimal 6 bit)	$-16 \leq \text{coeff} \leq 15 + 63/64$
Filter Output	signed 12bit (decimal 4 bit)	$-128 \leq \text{filter_out} \leq 127 + 15/16$

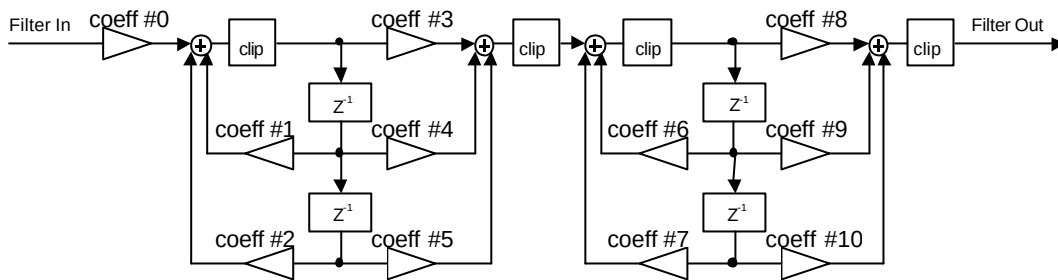


Figure 25 IIR Filter

AF Engine Output

1. Green Sum: 32bit(22bit)
2. FV sum: 32bit(26bit) (decimal 4bit) x 2

Data transfer to SDRAM

Results of the 2 adjacent Paxels in a row are gathered and transferred to the SDRAM at the same time, therefore, if PAXHC is set to 8 and PAXVC is set to 3, there will be 20 sets (= 5 x 4) of data transfer within an image. The results of the first Poxel in an image are stored to the address specified in the register AFSDRA. The address is automatically incremented until the two continuous buffers for the two successive frames become full. SDRAM buffer flag (AFSDRFLG) indicates the buffer at which the results are stored.

Example of data transfer

- Peak mode
- (PAXHC, PAXVC) = (2,1)

First Poxel

0	1	2
3	4	5

SDRAM address (byte address)	31	16	15	0	
AFSDRA					1 st transfer start
				Green sum	(Poxel 0)
				FV sum -1	(Poxel 0)
				FV sum -2	(Poxel 0)
				All zero	
				Green sum	(Poxel 1)
				FV sum -1	(Poxel 1)
				FV sum -2	(Poxel 1)
				All zero	
AFSDRA + 32				Green sum	(Poxel 2)
				FV sum -1	(Poxel 2)
				FV sum -2	(Poxel 2)
				All zero	
				Green sum	(Poxel 1)
				FV sum -1	(Poxel 1)
				FV sum -2	(Poxel 1)
				All zero	
AFSDRA + 64				Green sum	(Poxel 3)
				FV sum -1	(Poxel 3)
				FV sum -2	(Poxel 3)
				All zero	

1st transfer start1st transfer end2nd transfer start2nd transfer end3rd transfer start

AFSDRA + 96	Green sum	(Paxel 4)	
	FV sum -1	(Paxel 4)	
	FV sum -2	(Paxel 4)	
			3 rd transfer end
	Green sum	(Paxel 5)	4 th transfer start
	FV sum -1	(Paxel 5)	
	FV sum -2	(Paxel 5)	
	Green sum	(Paxel 4)	
	FV sum -1	(Paxel 4)	
	FV sum -2	(Paxel 4)	
	All zero		4 th transfer end

4.3.3 AE/AWB Engine

The pixels in the sub-sampled 2x2 blocks are accumulated in the window as AVE1. If any of the pixels in a 2x2 block are equal to or greater than the limit value(AVE2LMT), the block is treated as a saturated block. Only the blocks which are not saturating are counted into CNT2. The pixels which are clipped to AVE2LMT in the sub-sampled 2x2 blocks are accumulated in the window as AVE2, and are stored in SDRAM with AVE1.

Table 9 Configuration Registers

Register	width	Description
AEWEN	1	AE/AWB Engine enable 0:disable 1:enable
WINW	7	Window Width (even number) ((WINW +1) x2) (2 – 256)
WINH	5	Window Height (even number) ((WINH +1) x2) (2 – 64)
WINHC	4	Window Count for H direction (WINWHC+1) (1 – 12)
WINVC	3	Window Count for V direction (WINWVC+1) (1 – 8)
WINSH	11	X position of the Window start point (WINSH x2) (even number)
WINSV	11	Y position of the Window start point (WINSV x2) (even number)
AEWINCH	4	Sampling point increments (X) in a Window (even number) ((AEWINCH+1) x2) (2-32)
AEWINC V	4	Sampling point increments (Y) in a Window (even number) ((AEWINC V+1) x2) (2-32)
AVE2LMT	8	AVE2 limit value
AEWSDRA	22	SDRAM destination address to store the results of AE/AWB engine. It is automatically incremented until the two continuous buffers for the two successive frames become full.
AEWSDRFLG	1	SDRAM buffer flag to indicate the buffer at which the results are stored.

- The number of 2x2 blocks in a Window cannot exceed 256. WINW, WINH, AEWINCH and AEWINC V must be set accordingly.
- Windows can overlap either the first or the last line in a frame, but not both of them. WINSV, WINH and WINVC must be set accordingly.

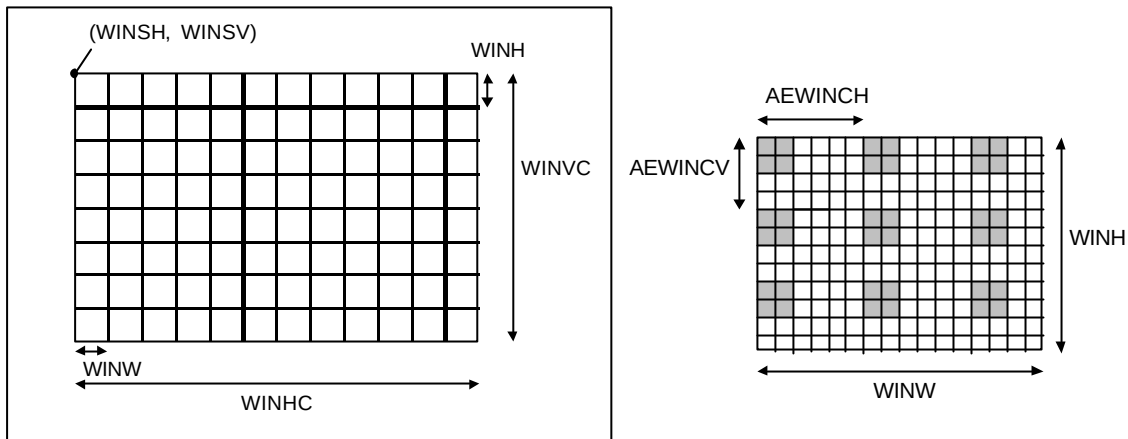


Figure 26 Window configuration

AE/AWB Engine Output

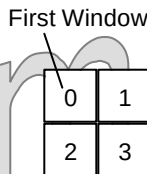
1. AVE1[i][j] – 16bit (Accumulated sub-sampled data clipped to 65,535)
2. AVE2[i][j] – 16bit (Accumulated sub-sampled data which are not saturated clipped to 65,535)
3. CNT2[i] – 16bit(9bit) (Number of sub-sampled data which are not saturated)
 - i: window number
 - j: pixel position in 2x2 blocks (0: upper left, 1: upper right, 2: lower left, 3: lower right)

Data transfer to SDRAM

Results of each window are transferred to the SDRAM at the same time, therefore, if WINHC is set to 8 and WINVC is set to 3, there will be 36 sets (= 9 x 4) of data transfer within an image. The results of the first Window in an image are stored to the address specified in the register AEWSDBA. The address is automatically incremented until the two continuous buffers for the two successive frames become full. SDRAM buffer flag (AEWSDBFLG) indicates the buffer at which the results are stored.

Example of data transfer

- (WINHC, WINVC) = (1,1)



SDRAM address (byte address) 31 16 15 0

AEWSDBA

AEWSDBA + 32

AVE1[0][1]	AVE1[0][0]	1 st transfer start
AVE1[0][3]	AVE1[0][2]	
AVE2[0][1]	AVE2[0][0]	
AVE2[0][3]	AVE2[0][2]	
All zero	CNT2[0]	
All zero		1 st transfer end
All zero		
All zero		
All zero		
All zero		
AVE1[1][1]	AVE1[1][0]	2 nd transfer start
AVE1[1][3]	AVE1[1][2]	
AVE2[1][1]	AVE2[1][0]	
AVE2[1][3]	AVE2[1][2]	

AEWSDRA + 64	All zero		CNT2[1]	2 nd transfer end 3 rd transfer start
	All zero			
	All zero			
	All zero			
	AVE1[2][1]		AVE1[2][0]	
	AVE1[2][3]		AVE1[2][2]	
	AVE2[2][1]		AVE2[2][0]	
	AVE2[2][3]		AVE2[2][2]	
	All zero		CNT2[2]	3 rd transfer end 4 th transfer start
	All zero			
AEWSDRA + 96	All zero			
	All zero			
	All zero			
	AVE1[3][1]		AVE1[3][0]	
	AVE1[3][3]		AVE1[3][2]	
	AVE2[3][1]		AVE2[3][0]	
	AVE2[3][3]		AVE2[3][2]	
	All zero		CNT2[3]	4 th transfer end
	All zero			
	All zero			
	All zero			

Preliminary

4.4 On Screen Display (OSD) and Graphics Acceleration

The OSD graphic accelerator is responsible for managing OSD data from different OSD windows and blending it with the video. It reads OSD data from SDRAM, and outputs to the on-chip PAL/NTSC Encoder. The OSD module defaults to standby mode, in which it simply sends video to the PAL/NTSC Encoder. After being configured and activated by the ARM CPU, the OSD reads OSD data and mixes it with the video output. The ARM is responsible for turning on and off OSD operations and writing to the OSD data to the SDRAM.

4.4.1 Features

- ❑ Can be configured for two video display windows and two OSD display windows, or can be configured for two video display windows, 1 OSD display window, and 1 attribute window
- ❑ Configurable rectangular hardware cursor can be displayed, its size can be specified, and its color can be specified by a look-up table.
- ❑ Attribute window to provide pixel level blinking and blending
- ❑ The four windows and the one cursor can be displayed simultaneously
- ❑ The background color can be specified
- ❑ Video-Window0 and Video-Window1 are use YCbCr data, and OSD-Window0, OSD-Window1
- ❑ Supports 1,2,4,8 bit wide bitmap data
- ❑ Five blending ratios for the OSD-window can be selected
- ❑ Transparency for the OSD-windows
- ❑ Default bit map look-up table in ROM available
- ❑ Programmable bit map look-up table in RAM available (24 bits x 256 words)
- ❑ Interlaced and non-interlaced modes supported
- ❑ Zoom horizontal and vertical with factor x2 and x4 available
- ❑ VGA->NTSC/PAL conversion circuit built in

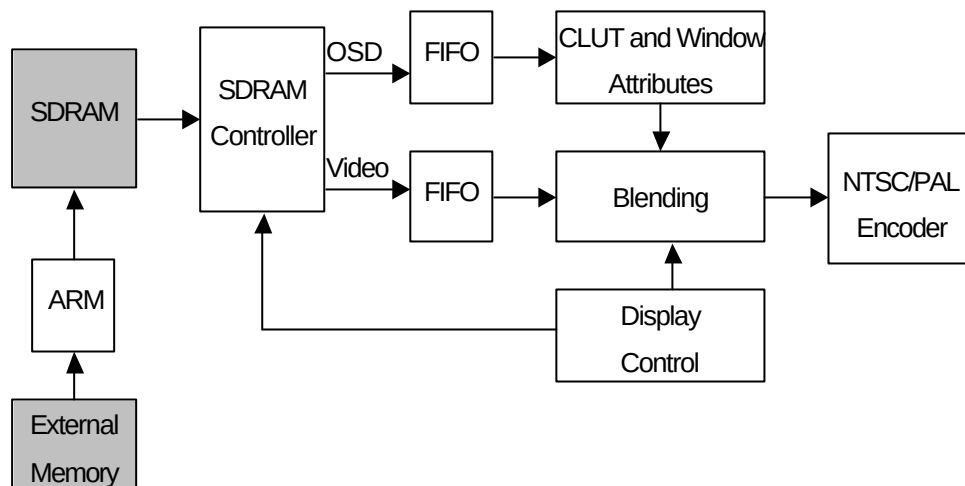


Figure 27 OSD block diagram

4.4.2 OSD data storage

The OSD data has variable size. In the bitmap window, each pixel can be 1, 2, 4, or 8 bits wide. In the YCbCr 4:2:2 window, it takes 8-bit per components, and the components are arranged according to 4:2:2 (Cb/Y/Cr/Y ...) format. In the case where RGB graphics data needs to be used as OSD, the application should perform software conversion to Y/Cr/Cb before storing it. The OSD data is always packed into 32-bit words and left justified. Starting from the upper left corner of the OSD window, all data will be packed into adjacent 32-bit words.

4.4.3 Setting Up an OSD Window

An OSD window is defined by its attributes. Besides storing OSD data for a window into SDRAM by ARM CPU, the application program also needs to update window attributes and other setup in the OSD module as described in the following subsections.

4.4.4 Location Register

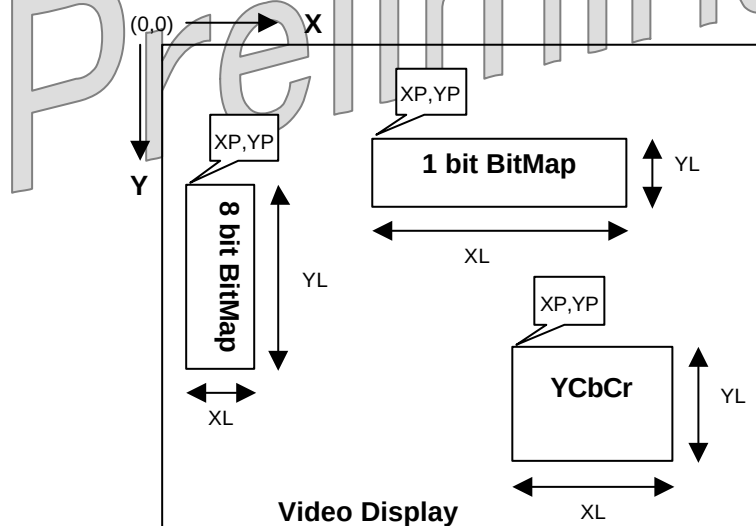


Figure 28 OSD Window Example

The Location register contains X and Y axis locations of the upper left and lower right corners of each window. The application program needs to set up the position of left-upper corner, the length of X and Y axis direction and enable selected OSD windows.

4.4.5 Color Look Up Table

The OSD has the fixed and programmable 256 entry color look up table (CLUT). The CLUT is used to convert bitmap data into Y/Cr/Cb components. In the case of 1, 2 or 4 bitmap pixels, the CLUT can be determined by CLUT registers.

4.4.6 Blending and Transparency

Color blending on the pixel level is also supported. This feature is available for the bitmap displays only (Window1, 2). If the window color blending is enabled, the amount of blending of each pixel is determined by the blending factor. As shown in Table 10 Blending and transparency in OSD, the window blending supports 8 different levels, according to the selected blending factor.

The hardware also supports a transparency mode with bitmap. If transparency is enabled, then any pixel on the bitmap display that has a value of 0 will allow video to be displayed. Essentially, 0 valued pixels are considered the transparent color, i.e. the background color will show through the bitmap. Table 10 shows the connection between transparency and blending on the same window.

Table 10 Blending and transparency in OSD

Transpare ncy	Blend Factor	OSD window contribution	Video contribution
OFF	0	0	1
	1	1/8	7/8
	2	2/8	6/8
	3	3/8	5/8
	4	4/8	4/8
	5	5/8	3/8
	6	6/8	2/8
	7	1	0
ON		if pixel value = 0	if pixel value = 0
	0	0	1
	1	1/8	7/8
	2	2/8	6/8
	3	3/8	5/8
	4	4/8	4/8
	5	5/8	3/8
	6	6/8	2/8
	7	1	0

4.4.7 Hardware Cursor

A rectangular shape is provided using video window1. The cursor always appears on top of other OSD Windows. The user can specify the position (XP, YP), size (XL, YL) or color of the shape. When video window1 is designated as the cursor, only three windows are available for the OSD application. If a hardware cursor is not used, then the application can use video window 1 as a regular hardware window. Figure 29 shows an example usage of the hardware cursor.

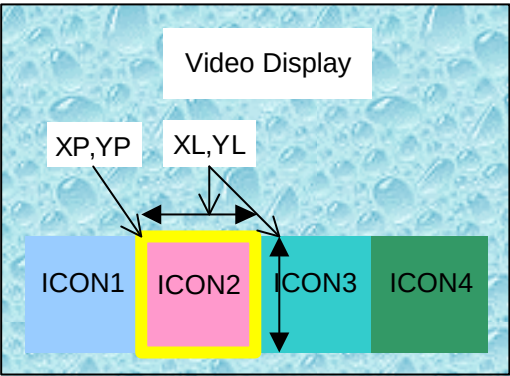


Figure 29 Hardware Cursor in OSD

4.4.8 Window Magnification

The OSD graphic accelerator has a function of magnification horizontally and vertically respectively. Each magnification ratio is bellow.

Table 11 Magnification Ratio

	Horizontal	Vertical
Video Window	$x1 / x2 / x4 / x9/8$	$x1 / x2 / x4 \ x6/5$
Bitmap Window	$x1 / x2 / x4$	$x1 / x2 / x4$

Preliminary

4.5 NTSC/PAL Encoder

The Video Encoder is used to display images and video on TV monitor and various LCD displays. The Video Encoder converts digital YCbCr signals to analog and digital video signals. The Video Encoder can generate composite NTSC or PAL signals, and 8-bit digital LCD interface signals. To generate these signals, the Video Encoder has digital to analog converter, a digital LCD interface, and a set of signals to perform timing and synchronization. The Video Encoder is shown in Figure 30.

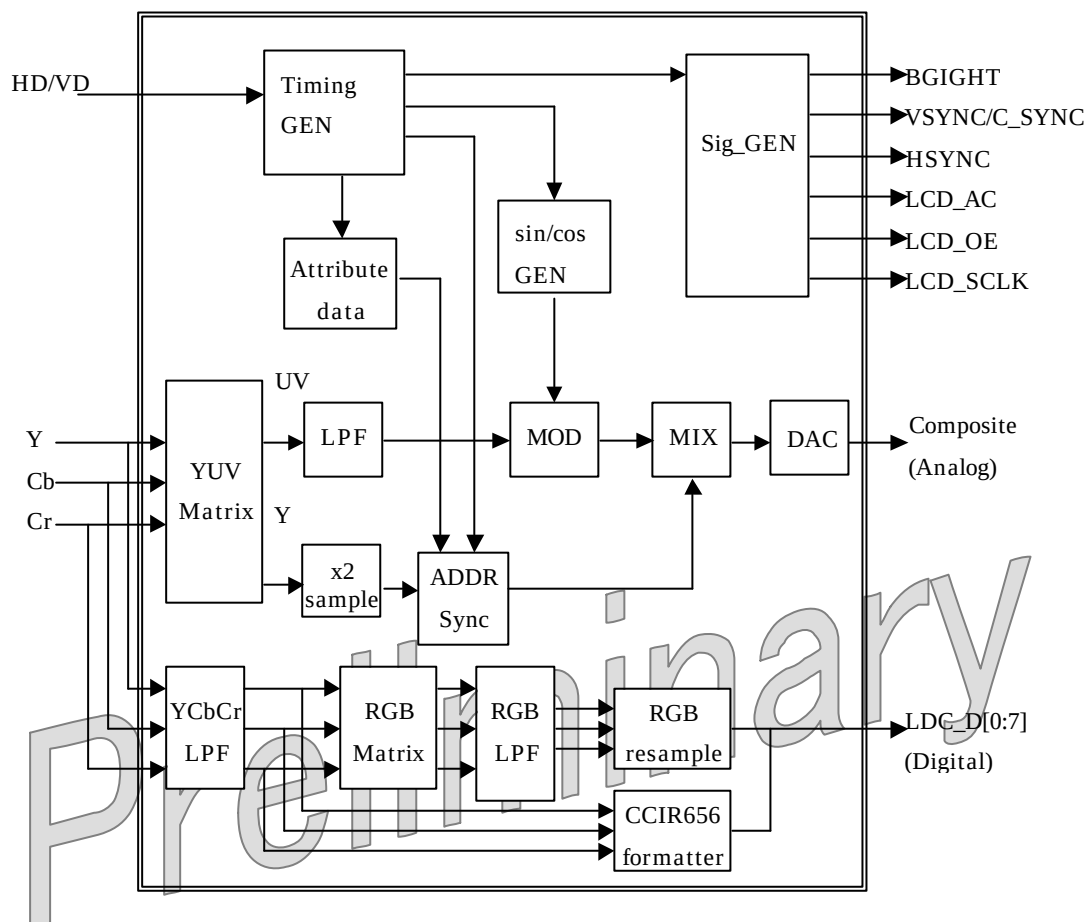


Figure 30 NTSC/PAL Encoder Block Diagram

4.5.1 Composite analog video features

- ❑ Master Clock Input single 27MHz
- ❑ 10-bit D/A Output Interface with output control
- ❑ NTSC/PAL Output Selectable
- ❑ Line 20 Video Attribute Insertion
- ❑ Interlace/Non-Interlace selected output
- ❑ Setup Selection for NTSC
- ❑ VSYNC/ HSYNC Output
- ❑ VSYNC/Composite SYNC Output Selectable
- ❑ 16-235/0-255 Input Amplitude Selectable
- ❑ Video Output Blanking

4.5.2 Digital video features

- ❑ Programmable DCLK out for Digital-RGB Output
- ❑ YCbCr or RGB digital output selectable
- ❑ 8-bit Digital RGB Output for Digital LCD
- ❑ Chrominance Low Pass Filter 1.5MHz/3MHz
- ❑ Internal Color Bar Generation (100/75%)
- ❑ Digital Low Pass Filter for RGB Output
- ❑ Digital VSYNC/HSYNC output
- ❑ Programmable HSYNC/VSYNC Width
- ❑ Programmable HSYNC/VSYNC Interval
- ❑ Programmable VSYNC/HSYNC Interval
- ❑ VSYNC Output Delay from Input VSYNC at DCLK
- ❑ HSYNC Output Delay from Input HSYNC at DCLK
- ❑ Programmable RGB Output Format of ODD/ Even Line (Digital RGB)
- ❑ Programmable Picture Start Position Horizontal (Digital RGB Output)
- ❑ Programmable Picture Start Position Vertical (Digital RGB Output)
- ❑ Programmable Picture Valid Length Horizontal (Digital RGB Output)
- ❑ Programmable Picture Valid Length Vertical (Digital RGB Output)
- ❑ Back Light Control Signal Output

4.5.3 Signal Level

The NTSC/PAL Encoder accepts digital input signals in YCbCr format. The NTSC/PAL Encoder has a selection register determines the manner in which input signal levels are processed. When register value is 1, a value of 16 is processed as black level, and a value of 235 is processed as 100% white level. For chroma signal also, a value of 16 and a value of 240 is processed as 100% chroma level.

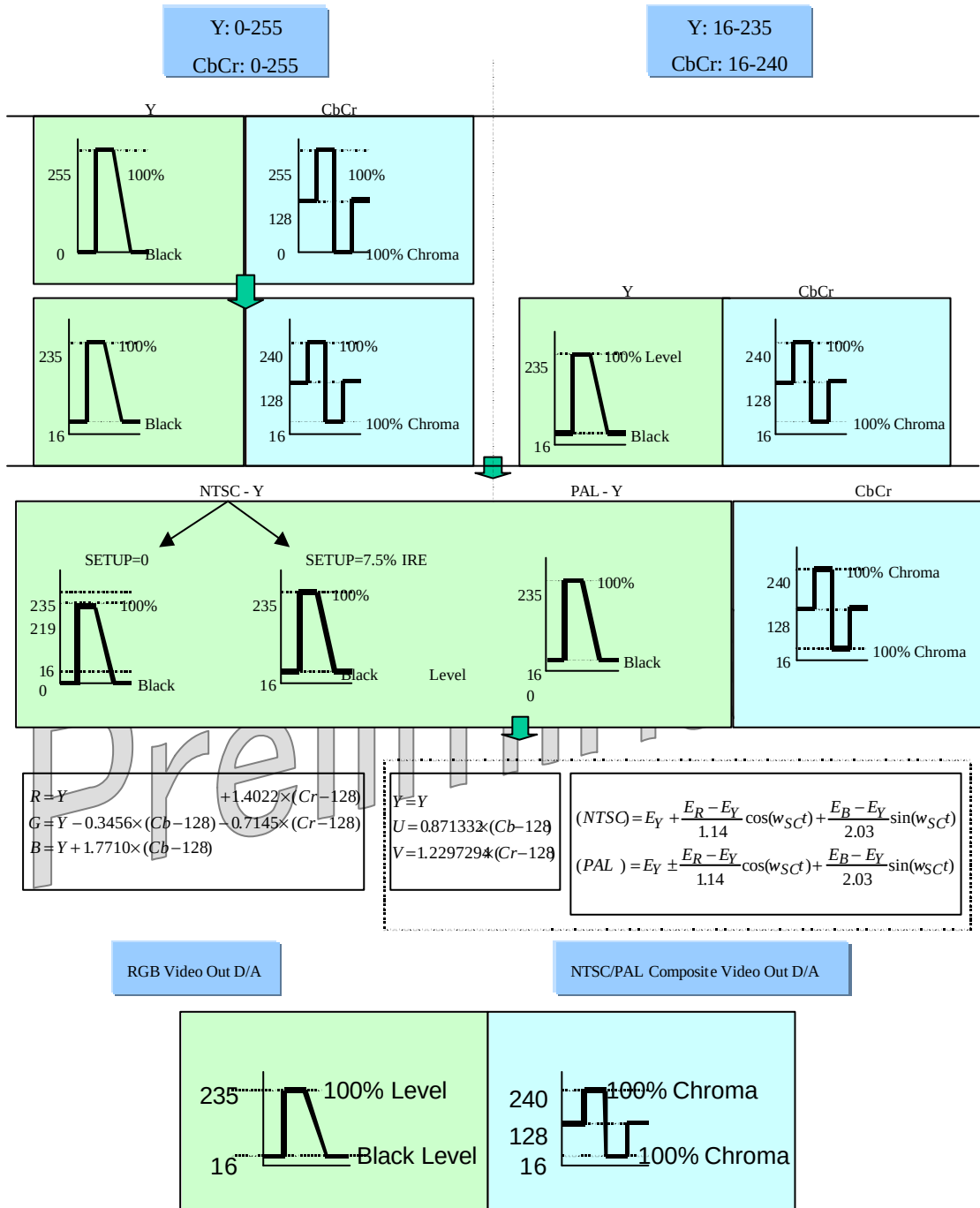


Figure 31 Input Video Data**4.5.4 Signal calculation**

Each RGB or Composite video output is transformed by using following equations.

[RGB output]

$$R = Y + 1.4022 \times (Cr - 128)$$

$$G = Y - 0.3456 \times (Cb - 128) - 0.7145(Cr - 128)$$

$$B = Y + 1.7710 \times (Cb - 128)$$

[Y/C, Composite output]

$$Y = Y$$

$$U = 0.871332 \times (Cb - 128)$$

$$V = 1.2297294 \times (Cr - 128)$$

$$NTSC = E_Y + \frac{E_R - E_Y}{1.14} \cos(?_{sc}t) + \frac{E_B - E_Y}{2.03} \sin(?_{sc}t)$$

$$PAL = E_Y + \frac{E_B - E_Y}{2.03} \cos(?_{sc}t) + \frac{E_R - E_Y}{2.03} \sin(?_{sc}t)$$

4.5.5 Color signal cutoff frequency select

When output is Y/C or Composite, the chroma signal is restricted to the frequency band by the low-pass filter. On the NTSC/PAL encoder, cutoff frequency is selected using the ARM register. When 0, cutoff frequency is 1.5MHz; When 1, cutoff frequency is 3MHz.

4.5.6 Video attribute insertion

The NTSC/PAL encoder has capability to insert video information into the vertical blanking period. For example, the NTSC/PAL encoder can insert a video attribute which indicates the proper aspect ratio to the video receiver. For NTSC mode, the NTSC/PAL encoder can insert 14-bit video information on line 20 of every field to conform to the EIAJ CPX-1024 Video Aspect Ratio ID specification. Attribute information should be set using ATR1 and ATR0 registers. The ATR2 register should be set with the 6bit CRC data that is calculated by the following equation:

$$G(X) = X^6 + X + 1 \text{ where } X^6, X \text{ are preset to } 1$$

For PAL mode, the NTSC/PAL encoder can insert 14-bit video information on line 20 of every frame to conform to the ETS300-294 Wide Screen Signaling specification. Attribute information should be set in the ATR1 register and ATR2 register.

4.5.7 Digital RGB/YUV Interface

The Video Encoder has an 8-bit digital RGB/YUV interface that can be configured to interface many different types of digital LCD's. DM270 supports various (EPSON/CASIO/SANYO/KODAK etc.) LCD panels.

Table 12 Video Encoder Signal Interface

Pin Name	YCbCr16	YCbCr8	RGB	EPSON	CASIO
HSYNC	H-sync	H-sync	H-sync	H-sync	H-sync
VSYNC	V-sync	V-sync	V-sync	V-sync	V-sync

VCLK	13.5/27MHz	27MHz	DCLK	XSDL	CLK
Y7	Y7	Y/C7	Data7	Data5	Data5
Y6	Y6	Y/C6	Data6	Data4	Data4
Y5	Y5	Y/C5	Data5	Data3	Data3
Y4	Y4	Y/C4	Data4	Data2	Data2
Y3	Y3	Y/C3	Data3	Data1	Data1
Y2	Y2	Y/C2	Data2	Data0	Data0
Y1	Y1	Y/C1	Data1	GCP	STBYB
Y0	Y0	Y/C0	Data0	LP	RIT
C7	C7	LCD_AC	LCD_AC	RES	CP
C6	C6	LCD_OE	LCD_OE	XINH	STH
C5	C5	Bright	Bright	YSCL	STB
C4	C4	CSYNC	CSYNC	YSCLD	GPCK
C3	C3	PWM_OUT	PWM_OUT	FRYS	GRES
C2	C2			FRYP	POL
C1	C1			FRX	
C0	C0			DY	GSRT

Preliminary

5 DSP Subsystem

The DSP subsystem is based on the TMS320C5409 DSP from TI catalog. This DSP core and its peripherals are fully code-compatible with other C54x products. The DSP software development tools for the C54x are very mature and can be used to develop, compile and debug the DSP code.

The DSP subsystem includes the C54x core, several blocks of volatile memories, an imaging extension coprocessor (iMX), and a variable length coding/decoding accelerator (VLCD).

This chapter gives only an overview of the DSP core and focuses more on its peripherals. For detailed information on the architecture of this 16-bit, DSP core and/or on the associated tools, refer to the C54x user's guides and tools documentation.

To realize image processing, image compression and voice processing, the DSP controls SDRAM DMA transfers to/from external memory and the iMX and VLCD accelerators. Since programming of the DSP is flexible and easy, users can improve the image processing pipeline and add differentiated functions to a system.

5.1 DSP core

C54x devices are fixed-point digital signal processors (DSPs) from the Texas Instruments TMS320 family. The C54x CPU, with its advanced modified Harvard architecture, features minimized power consumption and a high degree of parallelism.

This processor has one program memory bus and three data memory buses. It also provides an arithmetic logic unit (ALU) that has a high degree of parallelism, application-specific hardware logic, on-chip memory, and additional on-chip peripherals. The basis of the operational flexibility and speed of this DSP is a highly specialized instruction set.

Separate program and data spaces allow simultaneous access to program instructions and data, providing the high degree of parallelism. Two read operations and one write operation can be performed in a single cycle. Instructions with parallel store and application-specific instructions can fully utilize this architecture. In addition, data can be transferred between data and program spaces. Such parallelism supports a powerful set of arithmetic, logic, and bit manipulation operations that can all be performed in a single machine cycle. In addition, this processor includes the control mechanisms to manage interrupts, repeated operations, and function calls.

5.1.1 Features

- ❑ Low-power C54x DSP CPU
- ❑ Software-programmable wait-state generator with bank-switching wait state logic
 - ❑ External memory interface
 - ❑ Program space
 - ❑ Data space
 - ❑ I/O space
 - ❑ Scan-based emulation logic

In the DM270, the wait state generator and the external memory interface are used to connect the DSP core to the expansion memory of the DSP subsystem and to the image buffers.

5.1.2 DSP peripherals

The DSP CPU core is associated with an HPI interface, an interrupt handler, a parallel interface XIO, two multi-channel buffered serial ports (McBSPs), and a JTAG interface.

The JTAG interface is connected to the JTAG interface of the ARM core through some logic. This logic and the interface are described in more detail in the appendix.

The parallel interface is used to connect the core to the expanded memory, the shared blocks of memory, but as well, this interface is used to control the VLCD engine and iMX coprocessor. Refer to the appropriate chapter to get more details on each one of these points.

The host port interface (HPI) of the DSP is connected to the DSP Controller of the ARM subsystem. This connection cannot be modified. The HPI cannot serve any other purpose than to communicate back and forth with the ARM. The HPI bridge is described in the ARM-DSP Communication chapter.

The DSP core features as well a DMA controller that can be used to transfer data between the image buffers, the expanded memory block, the internal memory and the McBSPs.

The PLL that can normally be found on a standard C54x device has been removed. The DSP PLL is now part of the clock controller module and fully controlled by the ARM.

The only peripherals that can be used by the DSP to communicate directly with external devices to the DM270 are the two multi-channel buffered serial ports and the XF pin. The DSP can control the XF signal by writing into the proper memory map register. This signal is multiplexed with the ARM GIO pin. This signal can be read as well by the ARM through the DSP controller MMRs.

5.1.3 Multi-channel buffered serial ports

The DM270 has two high-speed, full-duplex multi-channel buffered serial ports (McBSPs), one of which (McBSP1) has its pins multiplexed with the ARM GIO pins. The McBSP ports allow direct interface to other C54x devices, codecs, and other devices in a system. The McBSPs used in the DSP are based on the standard serial port interface found on other TMS320C54x devices. The two McBSPs that are accessible to the DSP are named McBSP0 and McBSP1.

A brief description of the McBSP included in the DM270's DSP core is given in this section. For detailed information on the DSP's McBSP, refer to the C54x peripheral manual: *The C54x reference set volume 1 (spru131)*.

5.1.4 DSP subsystem memory

Figure 32 shows the DSP subsystem. Shared memory for image processing (Image Buffer, 16 KW) is connected to the DSP expansion memory bus (XIO 16-bit data bus). The DSP has 64 KW of DSP program and data space memory. In addition to the DSP, the VLCD, iMX, image processing hardware accelerator, and DMA controller (in the Image Buffer module) can access shared memory (Image Buffer). Since the shared buffer is mapped to DSP memory addresses, it can be accessed directly. DSP memory is constructed from three independent spaces: program, data, and I/O. Control and configuration registers for the Image Buffer, iMX and VLCD are mapped to I/O space.

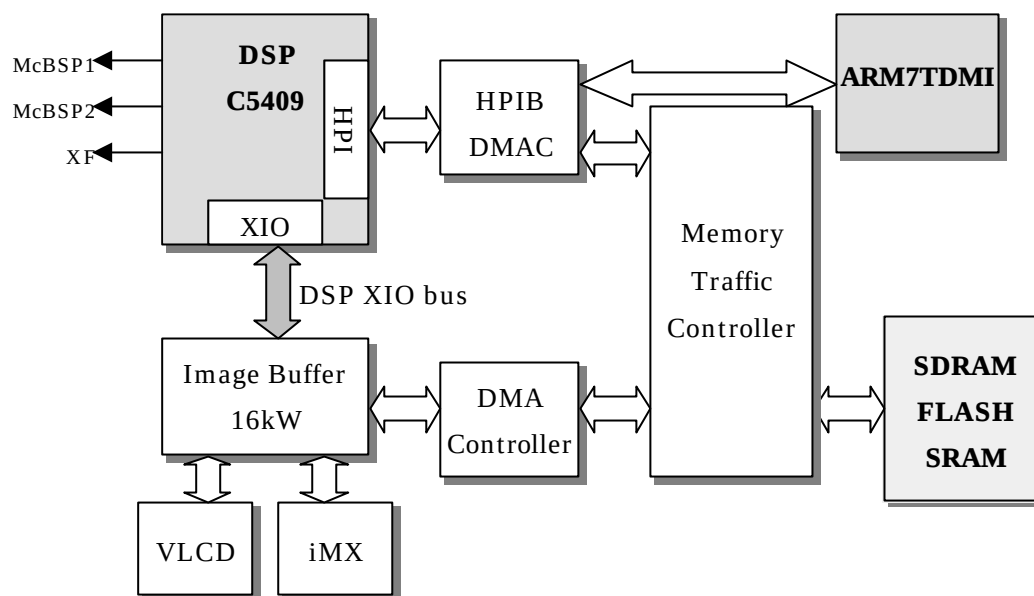


Figure 32 DSP subsystem block diagram

5.1.5 Program data and IO spaces

The DSP memory is organized into three individually selectable spaces: program, data, and I/O. Within any of these spaces, DARAM, ROM, SARAM, or memory-mapped peripherals can reside either within the DSP core or off the core.

The program memory space contains the instructions to execute, as well as tables used in execution. The data-memory space stores data used by instructions. The I/O memory space interfaces to the iMX, VLCD engines and to the image buffer controller.

There are three CPU status register bits that affect memory configuration. The MP/MC, OVLY, and DROM bits are located in the processor mode status register (PMST).

5.1.6 DSP Memory map

The DSP subsystem offers 32 K words (1 word = 16 bits) of DSP internal DARAM memory, 32 K words of DSP internal SARAM memory, 32 K words of ROM memory, and 16 K words of shared RAM memory. Figure 33 shows the DSP memory map after reset. The program space and the data memory spaces span the addresses 0x0h to 0x1FFFh. Each block of memory has a width of 16 bits. Some blocks are available for both program and data spaces, such as the DARAM block when OVLY=1.

	DATA	PROGRAM (PAGE0)	PROGRAM (PAGE1)
0000	MMR		
	Scratched-Pad	Reserved	Reserved
007F 0080			
	Data/Program RAM (OVLY=1)	Data/Program RAM (OVLY=1)	Data/Program RAM (OVLY=1)
7FFF 8000			
	Data RAM	Program ROM	Reserved
BFFF C000			
	ROM (DROM=1) or Image Buffer (DROM=0)	ROM (MP/MC=0)	
FFFF FF00			
	Reserved (DROM=1) or Image Buffer (DROM=0)	Interrupts & Reserved ROM (MP/PC=0)	Program RAM
FFFF			

Figure 33 DSP Memory Map

5.1.7 I/O memory space

The C54x devices offer an I/O-memory space in addition to the program memory and data-memory spaces. The I/O-memory space is a 64K-word address space (0000h–FFFFh) and exists only external to the DSP device. Two instructions, PORTR and PORTW, are used to access this space. Read timings vary from those of the program-memory and data-memory spaces to facilitate access to individual I/O-mapped devices rather than to memories.

On the DM270, only some addresses from this space are used. Image Buffer registers, iMX registers, and VLCD registers are mapped to I/O spaces. Signals to control the Image buffer clock, iMX buffer clock, and VLCD buffer clock are mapped to an I/O space. A signal to interrupt the ARM from the DSP is also mapped to an I/O space. Table 13 shows the address allocation of each module.

Table 13 I/O space

I/O address	Module name
0x0000h-0x0009h	Image Buffer
0x0010h-0x0017h	iMX
0x0020h-0x005Fh	VLCD
0x0080h-0x0087h	HPIB
0xFFFFh	CLOCK Control
0xFFFFh	DSP to ARM Interrupt

5.2 Image Buffer

The coprocessor subsystem is part of the DSP subsystem. It is designed to enhance DSP's capability in image processing and in other computation-intensive applications. The coprocessor subsystem consists of the following components:

- ❑ Coprocessors (iMX and VLCD)
- ❑ DMA controller (built into the Image Buffer module)
- ❑ Shared memory
- ❑ Shared memory access control
- ❑ The DSP's Interrupt controller
- ❑ Clock and coprocessor manager

All components reside in DSP's data page and/or I/O space, and all components are controlled by the DSP. Figure 32 shows the architecture of the Coprocessor Subsystem. There are two coprocessors in the system, Image Extension (iMX) and Variable Length Coder/Decoder (VLCD). The iMX accelerates computation-intensive imaging processing functions such as filtering, interpolation and DCT/IDCT. The VLCD provides acceleration for Huffman coding and encoding as well as quantization and inverse quantization required in various still image and motion picture compression standards.

The DMA controller provides a pathway to the ARM subsystem's SDRAM, allowing high-speed data transfer between the ARM subsystem and the DSP Subsystem. The DMA controller is part of the image buffer module. The DMA controller referred to in this section is the DMA controller built into the image buffer module not the DSP's DMA controller. However, the DSP's DMA controller can be used to transfer data between the DSP and each of the shared memories. The DSP's DMA controller cannot be used to transfer data to SDRAM, this can only be done by the image buffers DMA controller.

Each coprocessor and the DMA controller have their own private memory to store data structures and instructions unique to each functional unit. In addition, the DSP, iMX, VLCD and DMA controller also communicate to each other in a cooperative manner in order to perform useful functions. The communication is made possible by a shared memory, shared memory access control, and DSP interrupts.

Figure 34 shows the shared memory blocks. There are three types of coprocessor memory: Image buffers, iMX coefficient/command buffer, and VLCD table buffer. Image buffers consist of Buffer A and Buffer B. Buffer A and Buffer B are shared memory among the coprocessors and the DMA controller. The iMX coefficient/command buffer is used to store the coefficients and command for the iMX. The VLCD buffer is used to store Huffman tables, quantization coefficients, and other data structures for the VLCD. As mentioned in the previous section, user need to configure the device to add 1 wait state when accessing these memory blocks (Image Buffer A, Image Buffer B, iMX coefficient/command buffer, and VLCD table buffer).

The DSP software through registers in the Image Buffer module controls shared memory access. When a coprocessor, the DMA controller, or the DSP is given access to a shared memory buffer that module has a dedicated link to the buffer.

Each coprocessor and the image buffer's DMA controller can interrupt the DSP.

The Clock and Coprocessor Manager manages the clocks and provides top-level control to coprocessors and other functional units in the Coprocessor Subsystem.

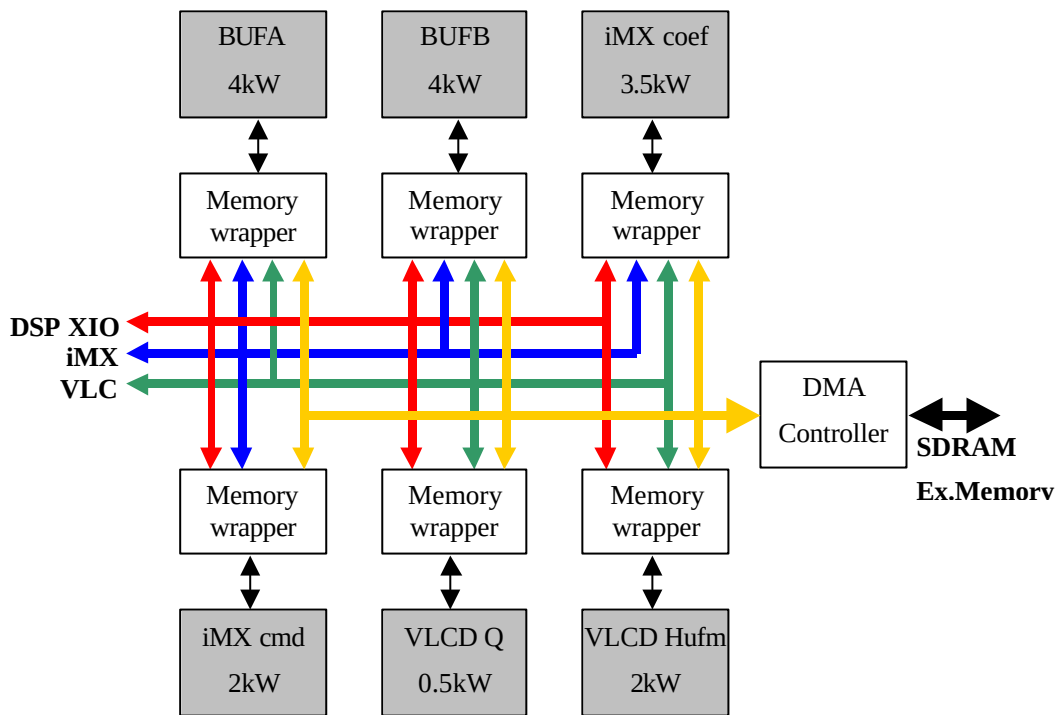


Figure 34 Shared Memory Block

BUFA and **BUFB** are used as the working memory for the image processing, and are shared either by DSP, iMX or VLCD. The memory wrapper is doing this switch logic for the shared memory. Note: a buffer can't be shared by all of access unit at same time, and access switch is controlled by DSP. **iMX coefficient memory** is used as the coefficient and temporary working space for iMX. DSP needs to initialize the coefficient of filter into this memory. The buffer switch is controlled by DSP. **iMX command memory** is used as the program command for the iMX. Before iMX is active, DSP needs to download the program into this memory. The buffer switch is control by DSP. **VLCD quantization table memory** is used as the quantization table for the VLCD engine. Table can be updated by DSP. **VLCD Huffman table memory** is used as the Huffman table for the VLCD engine. Table can be updated by DSP. The buffer switch is control by DSP.

The image buffer module manages access to each of the shared memory blocks and manages DMA transfers from the image buffers to SDRAM or External Memory.

5.2.1 Access Switching

Table 14 shows each memory size and which modules can access each buffer. The addresses, in the table, correspond to the DSP memory map. The DSP controls which module has access to each buffer by configuring register.

Table 14 Memory Structure

Memory	Access module				DSP Address	SIZE (WORDS)
	DSP	iMX	VLCD	DMAC		
Image BUFA	?	?	?	?	C000H-CFFFH	4k
Image BUFB	?	?	?	?	D000H-DFFFH	4k
iMX coefficient	?	?	?	?	E000H-EDFFH	3.5k
iMX command	?	?	-	?	EE00H-F5FFH	2k
VLCD Q table	?	-	?	?	F600H-F7FFH	512
VLCD Huffman	?	-	?	?	F800H-FFFFH	2k

5.2.2 The Image Buffer's DMA Controller

The Image Buffer module has a DMA controller that allows the DSP to initiate data transfers between the image buffers (Buffer A and Buffer B) and SDRAM or external memory or between the iMX coefficient buffer and SDRAM or external memory. It can also transfer the data to/from iMX cmd, VLCD Q and VLCD Huffman memory. The DMA controller will handle the handshaking sequence to/from SDRAM or the External Memory controller automatically.

The DMA controller supports 2-dimensional data transfers. As shown in Figure 35, DMA_XNUM specifies the number of pixels in the horizontal direction, and DMA_YNUM specifies the number of lines in the vertical direction. By setting each memory lead address (BUF_ADDR, SDR_ADDR) and line offset value (BUF_LOFST, SDR_LOFST), block transfer of second-order image data is possible. This function is effective in cases where processing in MACRO block units is required, such as in JPEG compression.

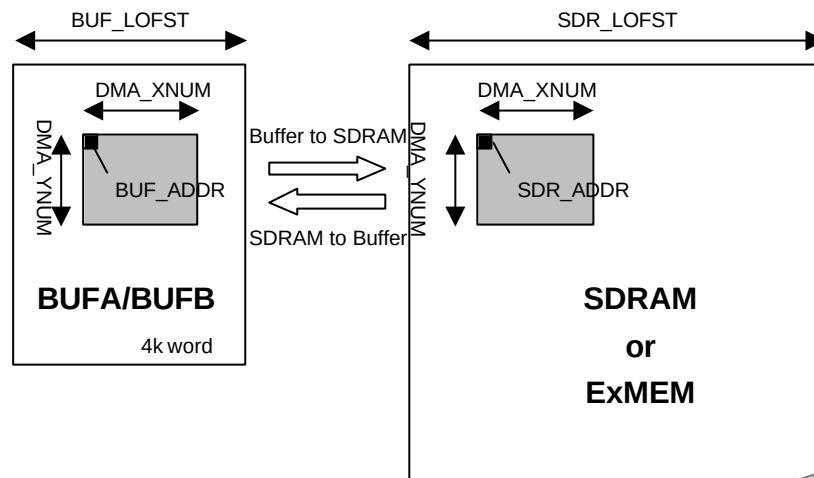


Figure 35 Two dimensional data transfer

DMA controller also has a function mode of byte-to-word conversion. In this mode, the 32bit(4 bytes) read data from SDRAM is converted to 64bit(4 words), and stored into image buffer. Table 15 shows data format in image buffer when byte-to-word conversion is enabled.

Table 15 byte-to word conversion

SDRAM read data	0(LSB) 31(MSB)			
	byte0	byte1	byte2	byte3
Image Buffer Address	4n (word)		4n+1	
write data	byte0	0	byte1	0
			byte2	0
			byte3	0

5.3 Imaging Extension (iMX)

Details describing the internal architecture of the iMX co-processor are generally not available to the user. The description of the iMX given in this chapter is very limited. To use the iMX, you should use the C callable iMX API provided by Texas Instruments. Refer to the documentation provided with the C callable API library for more information on these APIs.

The iMX is a parallel MAC engine for increasing DSP image processing performance. The iMX hardware module consists of a flexible control and memory interface and four MACs. The iMX works well in a shared memory configuration with the DSP, so that regular block-based image processing tasks can be accelerated by the iMX, and the DSP can handle irregular processing as well as task-level control.

The iMX can be used to increase the computation speed of many computations, such as the following computations:

- ❑ 2-D FIR filtering
- ❑ CFA interpolation
- ❑ Color space conversion
- ❑ Chroma down-sampling
- ❑ Edge enhancement
- ❑ False color suppression
- ❑ DCT/ IDCT
- ❑ Table lookup
- ❑ Motion estimation
- ❑ Generic matrix multiplication
- ❑ Generic linear transform

DM270 iMX is a 4-MAC version based on DSC25 iMX, with the following enhancements:

- Logical AND/OR/XOR function to support extracting lower bits, for post-lookup interpolation
- MIN-ID and MAX-ID functions. Basically preserves the pointer value to which the minimal value resides. This helps motion estimation.
- New “CALL” command which enables re-use a iMX command module
- Sequential access to image buffer A and B for non concurrent process

5.3.1 iMX Structure

As shown in Figure 36, the iMX consists of 4 parallel multiply-accumulate units (MACs). For most of the calculations mentioned above, the iMX processes 720(180x4) million MACs/sec. Each MAC can perform multiplication, addition, subtraction, absolute value, and accumulate.

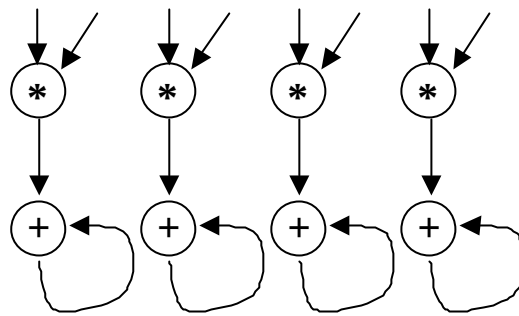


Figure 36 Structure of four parallel MACs

5.3.2 DSP/iMX Shared Memory Interface

Figure 37 shows the shared memory configuration in the DSP and iMX memory maps. The iMX co-processor shares iMX coefficient memory, iMX command memory, image buffer A, and image buffer B. All shared memory blocks shown are single-ported, and the DSP software must set which module has access to the shared memory block (no cycle-by-cycle arbitration is implemented). The DSP sets which module has access to each shared memory block by setting register.

In a typical application, the DSP places input data in buffer A and buffer B and iMX commands, filter coefficients, DCT/IDCT cosine constants, and lookup tables in the iMX coefficient/command memory. After placing data and commands in buffer A, buffer B, coefficient memory and command memory, the DSP configures BUF_CTL so that the iMX has access to each of these memory blocks, and then initiates iMX execution. The iMX accesses the memory blocks, computes the commands specified in the command buffer, and writes the result back to the data buffer.

The two general-purpose data buffers, buffer A and buffer B, can be used for efficient concurrent processing. For example, while the iMX is accessing data in buffer A the data in buffer B can be transferred to SDRAM through the image buffer's DMA controller. When the iMX finishes computation and the result computation result is stored into buffer A, buffer A and buffer B can be swapped so that the iMX can access data in buffer B and the processed data in buffer A can be sent to SDRAM through the image buffer's DMA controller.

To simplify the software that implements buffer swapping, the shared memory interface is designed such that iMX cannot access both buffer A and B at the same time. Buffer A and buffer B have the same memory map with respect to the iMX, and the DSP controls which buffer the iMX can access.

Buffer A, buffer B, the coefficient buffer, and the command buffer are 16-bit wide from DSP's point of view. Also the iMX can access data of type short or byte from buffer A, buffer B, the coefficient buffer, and the command buffer. Short accesses must be aligned to even 16-bit alignments.

Although the two iMX buffers are named coefficient and command, each buffer can be used to store input data, input coefficients, and output data. This provides flexibility in the DSP subsystem storage organization. Commands can only be stored in iMX command memory. DM270 has another memory interface to shared memory. It enables a sequential memory access to buffer A and B. In this mode iMX can access to image buffer A and B directly. Also DSP has to switch both buffer A and B to iMX access.

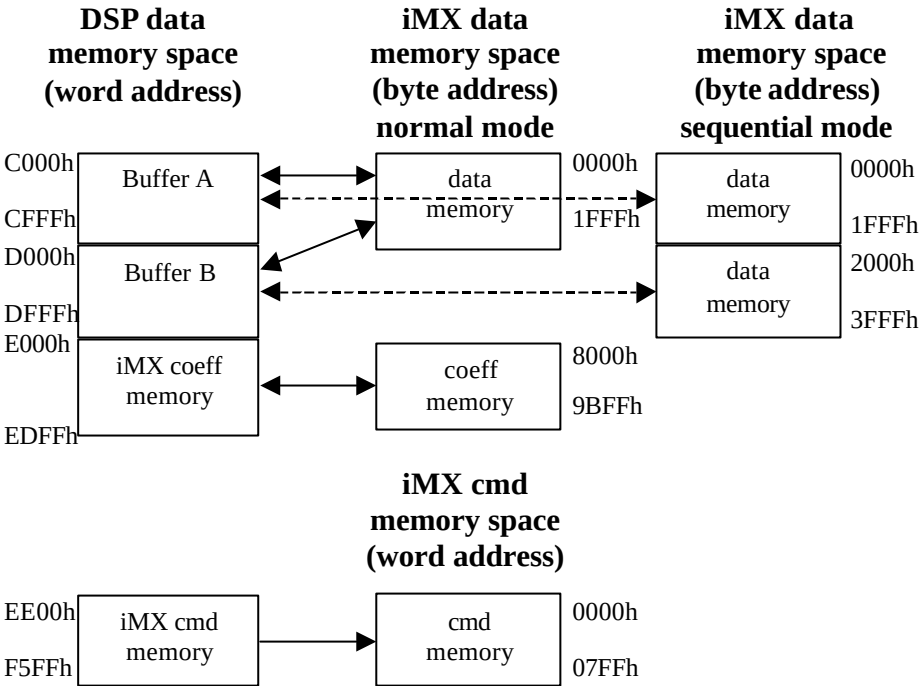


Figure 37 Shared Memory Between iMX and DSP

Preliminary

5.4 VLCD Engine

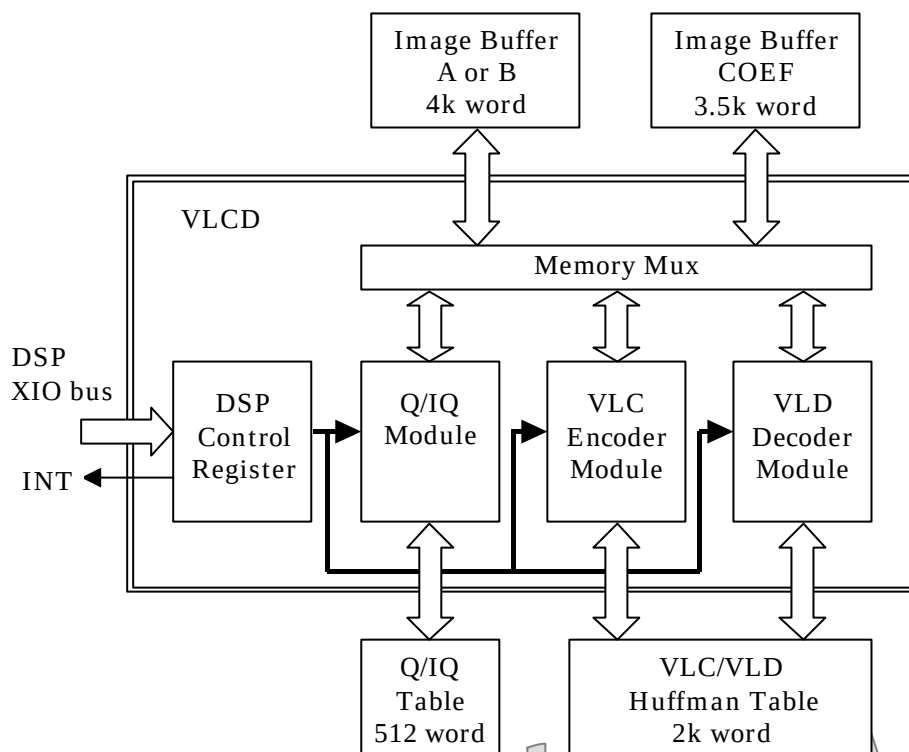


Figure 38 VLCD Block Diagram

DM270 VLCD module has QIQ,VLC and VLD functions for JPEG,MPEG standard. Quantizer and Inverse Quantizer (Q/IQ) module of the DM270 DSP subsystem is a high-performance, intelligent and highly programmable hardware accelerator for the quantization and de-quantization of DCT coefficients. The Q/IQ is highly autonomous and is capable of carrying out almost all the steps of quantization and de-quantization process with little or no help from the DSP.

VLCD module supports the quantization and de-quantization of the following standards:

- ❑ Quantization, zigzag(& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ Inverse Quantization, zigzag (& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263.
- ❑ Quantization, zigzag(& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ Huffman encode for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ Huffman decode for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ Quantization, zigzag scan for JPEG, MPEG1/2, MPEG4 and H.263
- ❑ CBP (Coded Block Pattern) detection
- ❑ Auto block skip for un-coded block (CBP)
- ❑ Auto insertion of Macro Block Header or SYNC Markers

With highly pipelined architecture, at its peak speed, Q/IQ module is capable of processing 180M DCT coefficients per second.

Q/IQ interfaces with the rest of the DSP subsystem via Image Buffers(A/B), Coefficient memory, Q/IQ Table and a set of control registers.

Q/IQ can access the Image Buffers(A/B) or Coefficient memory. Q/IQ can also transfer data to and from the Image Buffer(A/B) or Coefficient memory. Input DCT data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the quantization or de-quantization process. Q/IQ reads the input data from and writes output data to the Image Buffer(A/B) or Coefficient memory.

Q/IQ Table contains more configuration parameters in addition to what is already specified in the control registers. During the Q/IQ process, Q/IQ engine reads the parameters from the Q/IQ Table. The additional parameters provide more flexible control over the Q/IQ process. The parameters include quantization/de-quantization coefficients, DC predictor data, zigzag scan path and block operators.

The VLCD coprocessor handles Huffman encode and decode operation for image compression algorithms such as JPEG, MPEG-1/2/4, and H.263. It operates with Huffman tables preloaded by DSP, via shared memory blocks. The VLCD is highly autonomous and is capable of carrying out all the steps of Huffman coding and decoding process with little or no help from the DSP.

VLC/VLD interfaces with the rest of the DSP subsystem via Image Buffers(A/B), Coefficient memory, VLC/VLD Huffman Table and a set of control registers. VLC/VLD can access the Image Buffer(A/B) or Coefficient memory. VLC/VLD can also transfer data to and from the Image Buffer(A/B) or Coefficient memory. The quantized DCT data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the VLC. The JPEG/MPEG bitstream data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the VLD. VLC/VLD reads the input data from and writes output data to the Image Buffer(A/B) or Coefficient memory. VLC/VLD Huffman Table contains more configuration parameters in addition to what is already specified in the control registers. During the VLC/VLD process, VLC/VLD engine reads the parameters from the VLCD Huffman Table.

DSP configures and controls VLCD module via the control registers, which are mapped into DSP's IO space. The control registers allow the DSP to program Q/IQ and VLC/VLD or various industrial or user-defined standards, start the quantization or/and de-quantization process and VLC or VLD process and monitor its progress. The control registers also contain a set of address pointers to identify the locations of input data and output data in the Image buffers(A/B) or coefficient memory and the location of additional configuration parameters in the Q/IQ Table and VLC/VLD Huffman Table.

6 ARM Subsystem

The ARM subsystem is made up of the ARM7TMDI RISC processor core, Instruction cache, static RAM memory, and peripherals. The ARM controls the various peripheral modules including the timers, interrupt controller, video interface, burst compression engine, on-screen display, USB, and serial ports. The user interface should run on this processor.

In a system, the ARM is responsible for handling many system functions such as system-level initialization, configuration, user interface, user command execution, connectivity functions, and overall system control. The general purpose ARM handles these functions because it has a larger program memory space and better context switching capability, and is thus more suitable for complex, multi-tasking, and general purpose control tasks compared to the on-chip DSP.

The ARM subsystem of the DM270 offers the following features:

6.1 Features

- ❑ 32-bit RISC processor
- ❑ Maximum operating frequency 80 MHz
- ❑ 32-Kbyte built-in RAM
- ❑ 8-K byte Instruction cache
- ❑ 32/16-bit command sets
- ❑ Memory format: fixed Little Endian Format
- ❑ 2 interrupt systems (FIQ, IRQ)
- ❑ 16 external terminal interrupt systems
- ❑ x4 hardware timers
- ❑ Watchdog timer
- ❑ JTAG base emulation
- ❑ External memory interface

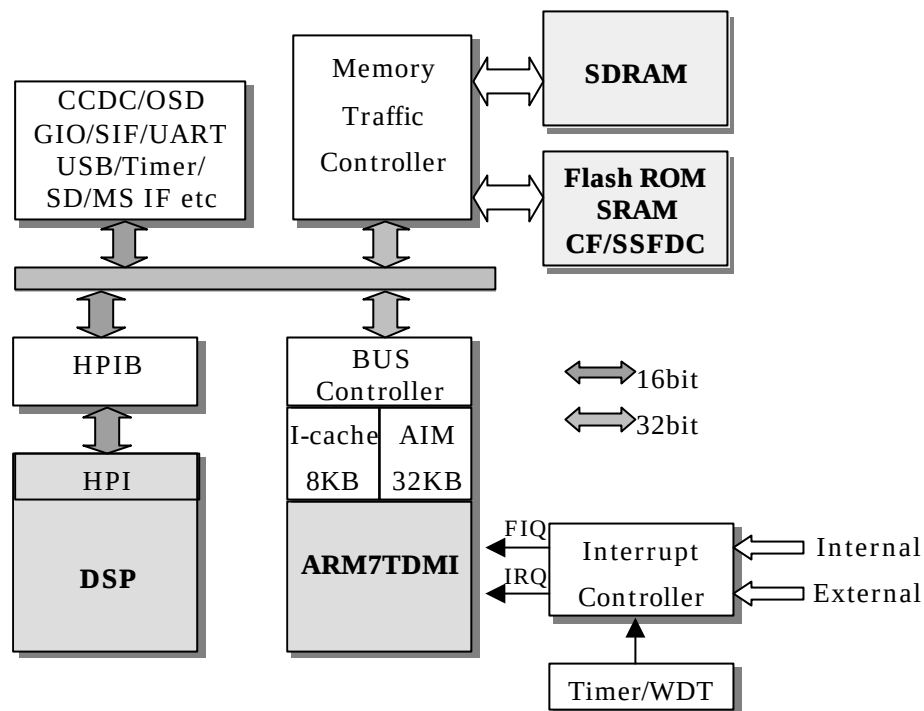


Figure 39 ARM Subsystem

6.2 ARM7TDMI RISC Processor Core

The ARM7TDMI core is integrated in the DM270 chip. The ARM7 core that you can find in the DM270 is based on the TMS470R1x microcontroller from Texas Instruments. The ARM7 core is specified to run at up to about 80 MHz. The ARM subsystem has its own 32 Kbytes of local static RAM.

The ARM7 processor employs a unique architectural strategy known as the 16-Bit Instruction Set (16-BIS), which makes it ideally suited to high-volume applications with memory restrictions, or applications where code density is an issue. A 32 bits instruction set (32-BIS) mode is also available as well to run complex algorithms.

6.3 Instruction cache (I-cache) system

I-cache in DM270 contains 8kbyte RAM for caching instruction. I-cache selects only instruction mapped external memory (0x100000-0x80FFFF). Below are I-cache features.

- ❑ 4-word (128bit) line size
- ❑ Total 512-line (256-line x 2way)
- ❑ Update data by LRU(Least Recently Used) algorithm
- ❑ Check tag-RAM according to address from ARM and execute below action in one wait cycle.
- ❑ When HIT, ARM reads the data stored in RAM in I-cache
- ❑ When MISS, external memory access starts.
- ❑ One valid bit per 2byte data in tag-RAM
- ❑ No Dirty bit in tag-RAM (no support write)
- ❑ Support I-cache disable(bypass) mode
- ❑ Selectable 1word/4word access (4word burst is supported only SDRAM access)
- ❑ Support cache clear function

6.4 Memory Map

ARM can access all peripherals registers, memory area. The BUS Controller is responsible to automatically generate the number of wait states required according to the peripheral module accessed by the ARM. The internal RAM (AIM) of the ARM can be access thru a 32 bits width data bus. It is recommended to store the main RTOS within this memory to get the best throughput. All registers of peripheral modules, except for the USB/SD/MS modules, are accessed thru a 16 bits width data bus with no wait-state.

The BUSC module supports three type of data access mode, e.g. a byte, a half-word and a word which is 8 bits, 16 bits and 32 bits respectively during data or instruction access to the external memories like SDRAM, Flash Memory and so on. It supports as well dynamic allocation of external memory area. The maximum number of partitioning area is six. This feature is explained in the memory controller chapter.

In addition, DSP memory can be also accessed by ARM through HPI Bridge.

The characteristics of the different memory spaces accessible by the ARM are summarized in the Table 16 and Figure 40 below.

Table 16 Memory Structure

Area Name	Start address	End address	Size(byte)
Reset vector ROM	0x0000:0000	0x0000:0003	4
ARM Internal RAM	0x0000:0004	0x0000:7FFF	32 K
Internal Peripheral	0x0003:0000	0x0003:0FFF	4 K
DSP Memory	0x0004:0000	0x0005:FFFF	128 K
External Memory	0x0010:0000	0x080F:FFFF	128 M

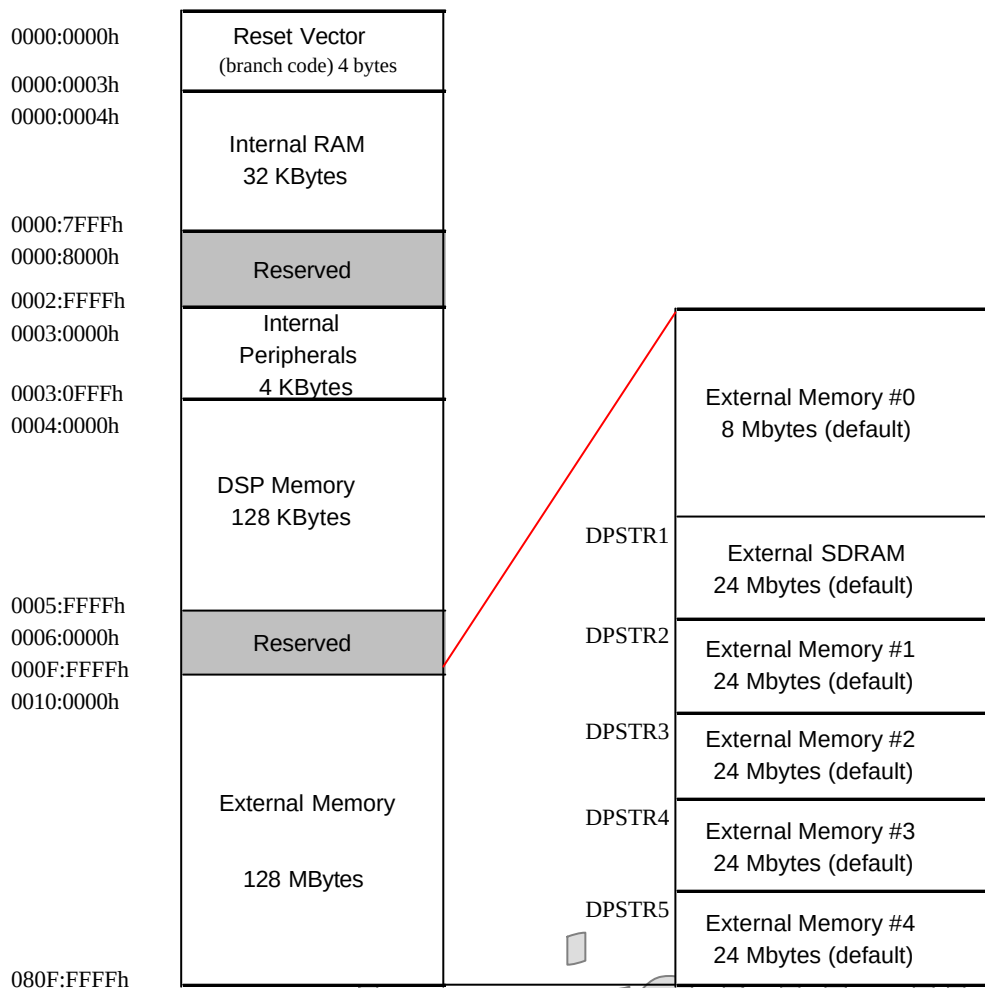


Figure 40—ARM Memory Map

6.4.1 Reset Vector ROM

Reset Vector ROM (RVR), which is defined at 0x0000:0000 of ARM internal memory space, is hard-coded in BUSC. ARM accessed this area to fetch instruction after power on reset. BUSC will provide ARM with the following code. The code means that jump to the address 0x0010:0000.

0000:0000EA 03 FF FE ; RVR: B 0010_0000h

After the reset signal is cancelled, DM270 accesses external memory address 0 (CS0). Therefore the program must be written here in advance.

6.4.2 ARM Internal RAM

This is ARM internal memory (IRAM). Since ARM can access this memory region in 1 cycle (no wait state), the RTOS (real time OS) or application programs can be operated more quickly here. Contents of this memory region can be access as byte (8 bit), half word (16 bit) or word (32 bit).

6.4.3 Peripheral Registers

BUSC can access to all internal registers with 16 bits data width. A read and a write access can be achieved basically within one cycle of ARM clock except for USB/SD/MS register access.

Table 17 Peripheral module memory map

Peripheral Name	Start address	End address
Timer 0	0x0003:0000	0x0003:007F
Timer 1	0x0003:0080	0x0003:00FF
Timer 2	0x0003:0100	0x0003:017F
Timer 3	0x0003:0180	0x0003:01FF
Serial 0	0x0003:0200	0x0003:027F
Serial 1	0x0003:0280	0x0003:02FF
UART 0	0x0003:0300	0x0003:037F
UART 1	0x0003:0380	0x0003:03FF
WDT	0x0003:0400	0x0003:047F
SD/MMC IF*	0x0003:0480	0x0003:04FF
Interrupt Controller	0x0003:0500	0x0003:057F
GIO	0x0003:0580	0x0003:05FF
DSP Controller	0x0003:0600	0x0003:067F
OSD	0x0003:0680	0x0003:06FF
CCD Controller	0x0003:0700	0x0003:077F
Preview	0x0003:0780	0x0003:07FF
NTSC/PAL Encoder	0x0003:0800	0x0003:087F
Clock Controller	0x0003:0880	0x0003:08FF
Bus Controller	0x0003:0900	0x0003:097F
SDRAM Controller	0x0003:0980	0x0003:09FF
External Memory I/F (EM IF, CFC/SSF IF)	0x0003:0A00	0x0003:0A7F
USB0	0x0003:0A80	0x0003:0AFF
USB1	0x0003:0B00	0x0003:0B7F
3A	0x0003:0B80	0x0003:0BFF
Memory Stick1	0x0003:0C00	0x0003:0C7F
Memory Stick2	0x0003:0C80	0x0003:0CFF
ATM	0x0003:0D00	0x0003:0D7F
I2C	0x0003:0D80	0x0003:0DFF
ICACHE	0x0003:0E00	0x0003:0E7F

The number of cycles of USBSD/MS register access varies depending on the internal state, and several cycles are required.

6.4.4 DSP Memory

When the ARM accesses the DSP internal memory through the HPI bridge module (HPIB), one wait-state is inserted automatically. In addition to it, extra wait cycles will be inserted depending on the wait signal that is generated in HPIB module.

ARM has no access to the image buffer memories or the expansion memory of the DSP subsystem.

6.4.5 External Memory

As external memory interfaces, DM270 has an SDRAM interface and a memory card or general-purpose memory interface. ARM can access SDRAM via the SDRAM controller module. Also, it can access memory card or flash ROM via the external memory interface module. In access from ARM to external memory, a wait state is automatically inserted, and its cycle varies depending on SDRAM frequency, external memory access time and so forth.

The SDRAM memory access is controlled by the SDRAM controller. When ARM access SDRAM memory an asynchronous transfer is achieved between SDRAMC and BUSC.

The dynamically allocated external memory, which is directly accessible by the ARM, is a maximum of 128 Mbytes, and can be divided into a maximum of 6 regions (SDRAM, and EM0 to EM4). By changing the memory map dynamically, external memory of a different capacity can be used efficiently. Each of the 6 regions can be used by setting a boundary address. Byte (8-bit), half-word (16-bit) and word (32-bit) types of accesses are possible.

6.5 Interrupt Controller

The Interrupt Controller (INTC) has 39 interrupt channels: 23 internal peripheral interrupt channels and 16 external interrupt channels. Table 18 lists each interrupt and the interrupt's corresponding function.

The INTC has the following characteristics:

- ❑ 39 interrupt channels (16 external channels)
- ❑ Interrupt mask each for channel
- ❑ Each interrupt channel can be mapped to an FIQ or IRQ type interrupt
- ❑ Configurable hardware interrupt priority
- ❑ Configurable interrupt entry table

Table 18 DM270 Interrupts

Interrupt	Interrupt Source	Description
INT38	CLKC	WAKE UP
INT37	I ² C	I ² C
INT36	WATCHDOG TIMER	WATCHDOG TIMER
INT35	PREVIEW	Occurs at the end of frame(configured by HSIZE/VSIZE registers) when SDRAM input mode is selected.
INT34	External interrupt15	IRP15 bit IRQPORT register GIO15
INT33	External interrupt14	IRP14 bit IRQPORT register GIO14
INT32	External interrupt13	IRP13 bit IRQPORT register GIO13
INT31	External interrupt12	IRP12 bit IRQPORT register GIO12
INT30	External interrupt11	IRP11 bit IRQPORT register GIO11
INT29	External interrupt10	IRP10 bit IRQPORT register GIO10
INT28	External interrupt9	IRP9 bit IRQPORT register GIO9
INT27	External interrupt8	IRP8 bit IRQPORT register GIO8
INT26	External interrupt7	IRP7 bit IRQPORT register GIO7
INT25	External interrupt6	IRP6 bit IRQPORT register GIO6
INT24	External interrupt5	IRP5 bit IRQPORT register GIO5
INT23	External interrupt4	IRP4 bit IRQPORT register GIO4
INT22	External interrupt3	IRP3 bit IRQPORT register GIO3
INT21	External interrupt2	IRP2 bit IRQPORT register GIO2
INT20	External interrupt1	IRP1 bit IRQPORT register GIO1
INT19	External interrupt0	IRP0 bit IRQPORT register GIO0
INT18	MMC/SD MS	MMC/SD Interface and MS Interface
INT17	MMC/SD MS	MMC/SD Interface and MS Interface
INT16	EMIF 1	BUSINTEN register
INT15	EMIF 0	At the end of DMA transfer DMACTL register

Interrupt	Interrupt Source	Description
INT14	USB	USB
INT13	UART1	UART1
INT12	UART0	UART0
INT11	DSP	DSP_HINT register (DSP)
INT10	External CPU I/F	External CPU I/F register
INT9	Serial I/F 1	At the end of DMA transfer
INT8	Serial I/F 0	At the end of DMA transfer
INT7	OSD	At the falling edge of VSYNC
INT6	CCD Controller	At the falling edge of C_WEN port
INT5	CCD Controller	VDINT1 register (CCD Controller)
INT4	CCD Controller	VDINT0 register (CCD Controller)
INT3	TIMER3	TIMER3
INT2	TIMER2	TIMER2
INT1	TIMER1	TIMER1
INT0	TIMER0	TIMER0

Each interrupt channel can be mapped to an FIQ or IRQ type interrupt, and each channel can be enabled or disabled. The ARM core supports two interrupt types: FIQ and IRQ. See the ARM7 User's Manual for detailed information about the ARM's FIQ and IRQ interrupts.

The INTC supports hardware configurable interrupt priority, and entry address generation. Entry address generation minimizes time spent jumping to interrupt vectors. When an interrupt occurs the corresponding interrupt vector is stored in the INTC's ENTRY register. The interrupt service routine can read the ENTRY register and jump to the interrupt vector. Thus the ARM does not have to execute any parsing software to associate the interrupt with the correct interrupt vector.

6.5.1 INTC Methodology

Figure 41 illustrates the INTC methodology. As shown in Figure 41, interrupts are mapped to FIQ or IRQ type interrupts by setting the FISEL_n register. When an interrupt occurs the status is reflected in either the FIQ_n or IRQ_n registers. Interrupts are enabled or disabled, masked, by setting the EINT_n register. Note: even if an interrupt is masked, the status of each interrupt is still reflected in the FIQ_n and IRQ_n registers. The INTC generates the entry address of the pending interrupt with the highest priority and stores the entry address in the FIQENTRY or IRQENTRY register. Interrupt priority can be configured in the INTPR_n registers, which are described below. The entry address is equal to a base address plus an offset, and the base address can be set in the EABASE_n registers. Each of the INTC reregisters is described below.

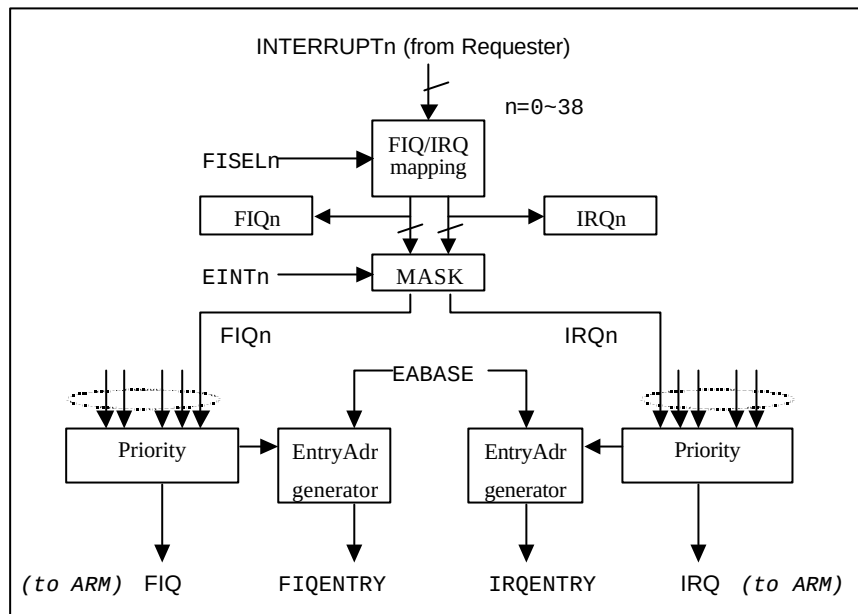


Figure 41 INTC Methodology

6.5.2 Hardware Configurable Interrupt Entry Table

The hardware configurable interrupt entry table can be used to minimize the time spent jumping to interrupt service routines. The entry table, created in software, can be stored anywhere in memory, but it must be on an 8-byte boundary. The structure of the entry table is shown in Figure 42.

Initialization:

1. Mask FIQ and IRQ
2. Mask all interrupt request by MASK register
3. Mapping the each interrupt to FIQ or IRQ by selection register
4. Setting up the each priority by priority configuration register
5. Setting-up a interrupt table base register(EABASE)
6. Enable the corresponding interrupts
7. Enable the FIQ and IRQ

Using the entry table to process interrupts works as follows:

8. ARM receives common FIQ/IRQ interrupt signal
9. ARM responds to interrupt signal and jumps to common FIQ or IRQ vector (depending on whether an FIQ or IRQ type interrupt occurred)
10. Common FIQ(0x001C) /IRQ(0x0018)vector
11. The software located in memory at the common interrupt vector reads the FIQ entry or the IRQ entry registers to get the interrupt service routine address, and then jumps to this address, the interrupt handler corresponding to INTx.
12. The interrupt is cleared by setting the bit corresponding to INTx in FIQ, IRQ status register

13. Return to the common interrupt vector routine from the interrupt handler
14. Return to the state of program execution that existed prior to when the interrupt occurred from the common interrupt vector routine

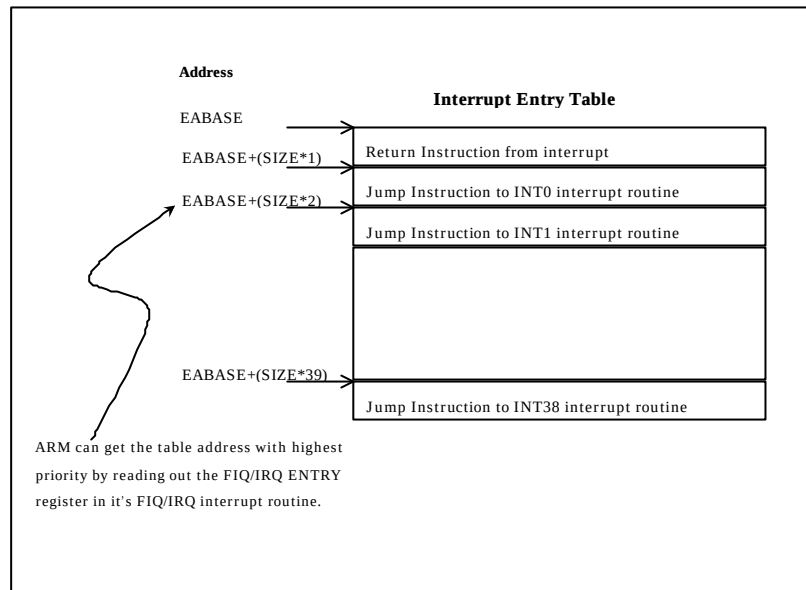


Figure 42 Interrupt Entry Table

6.6 Timers, Watch-Dog Timer

The ARM subsystem has four timers. The output of each timer is input into an ARM interrupt controller. Each of the general-purpose timers can run in one of two modes: one-shot mode and free-run mode. In one-shot mode, an interrupt only occurs once and then the timer must be explicitly reset to begin the timing operation again. In free-run mode, when the timer generates an interrupt, the timer counter is automatically reloaded to start the count operation again.

The Watch-Dog Timer (WDT) is used to determine when a system is running out-of-control and to initiate the system's recovery from an out-of-control state. If the WDT is not reset within a configurable time, the WDT automatically interrupts the ARM or resets the ARM. This ARM reset signal can be monitored externally through GIO pin (GIO25). The WDT uses the external clock connected to the M27XI pin. In normal operation a 27MHz clock is connected to M27XI.

6.7 PLL / Clock Controller

The PLL / Clock Controller module generates multiple clock frequencies from a single crystal input. This module generates an internal interim frequency as a multiplier of the input crystal, then divides this frequency down to obtain the necessary clocks to drive the ARM7, IMX, Coprocessors, USB and SDRAM. The input frequency is user definable within the allowable range of 20-35MHz, however under normal conditions would need to be 27MHz to allow for accurate NTSC/PAL video output. The internal interim frequency is the direct output from the PLL and can be a divided up to 4 (x1,1/2,1/3,1/4) of the PLL output frequency. Up to four kind of output frequencies can be generated from this interim internal frequency.

As shown in Figure 43 the CLKC has two analog PLLs built into it, and you can freely set the frequencies of the ARM clock, SDRAM clock, and DSP clock using these PLLs. To reduce system power, each clock frequency can be slowed down and each clock can be disabled.

The Clock Controller has the following external inputs:

1. PCLK: External pixel clock input
2. EXT_CLK: External system clock input(optional)
3. MXI: Externally attached X'tal oscillator input
4. M48XI: Externally attached 48MHz X'tal oscillator input
5. TEST2: ARM Clock Selection

In normal operation, a 27MHz signal is applied to MXI and a 48MHz signal is applied to M48XI. The CCD Controller (CCDC) pixel clock is connected to PCLK. The maximum pixel clock frequency is 40MHz, since the CCDC operates at a maximum of 40MHz (refer to the section titled "CCDC Controller"). The DM270 passes-through MXI and M48XI to output signals MXO and M48XO, respectively.

The Clock Controller generates the following 11 signals:

1. CLK_ARM: ARM clock for ARM core and its related modules
2. CLK_DSP: DSP clock for DSP core and its related modules
3. CLK_AXL: Accelerator clock for iMX,VLCD and Image Buffers
4. CLK_SDR: SDR clock for external SDRAM and its related modules
5. CLK_XI: X'tal clock such as TMR, WDT,UART
6. CLK_CCD: CCD Controller clock
7. CLK_ENC: Video Encoder clock
8. CLK_OSD: OSD clock
9. CLK_USB: USB clock
10. CLK_MMC: Multimedia Card / Secure Digital clock
11. CLK_MS: Memory Stick clock

As shown in Figure 43, the input signal TEST2 is used to select which clock will be used as the ARM clock prior to power-on and reset. By default, the ARM PLL is bypassed during power-on and reset, and either MXI or SYSCLK are used as the ARM clock. When the state of TEST2 is low, MXI is used and when the state of TEST2 is high, SYSCLK is used. After the ARM is rebooted, the ARM can configure several clocks by setting various registers in the Clock controller module.

Preliminary

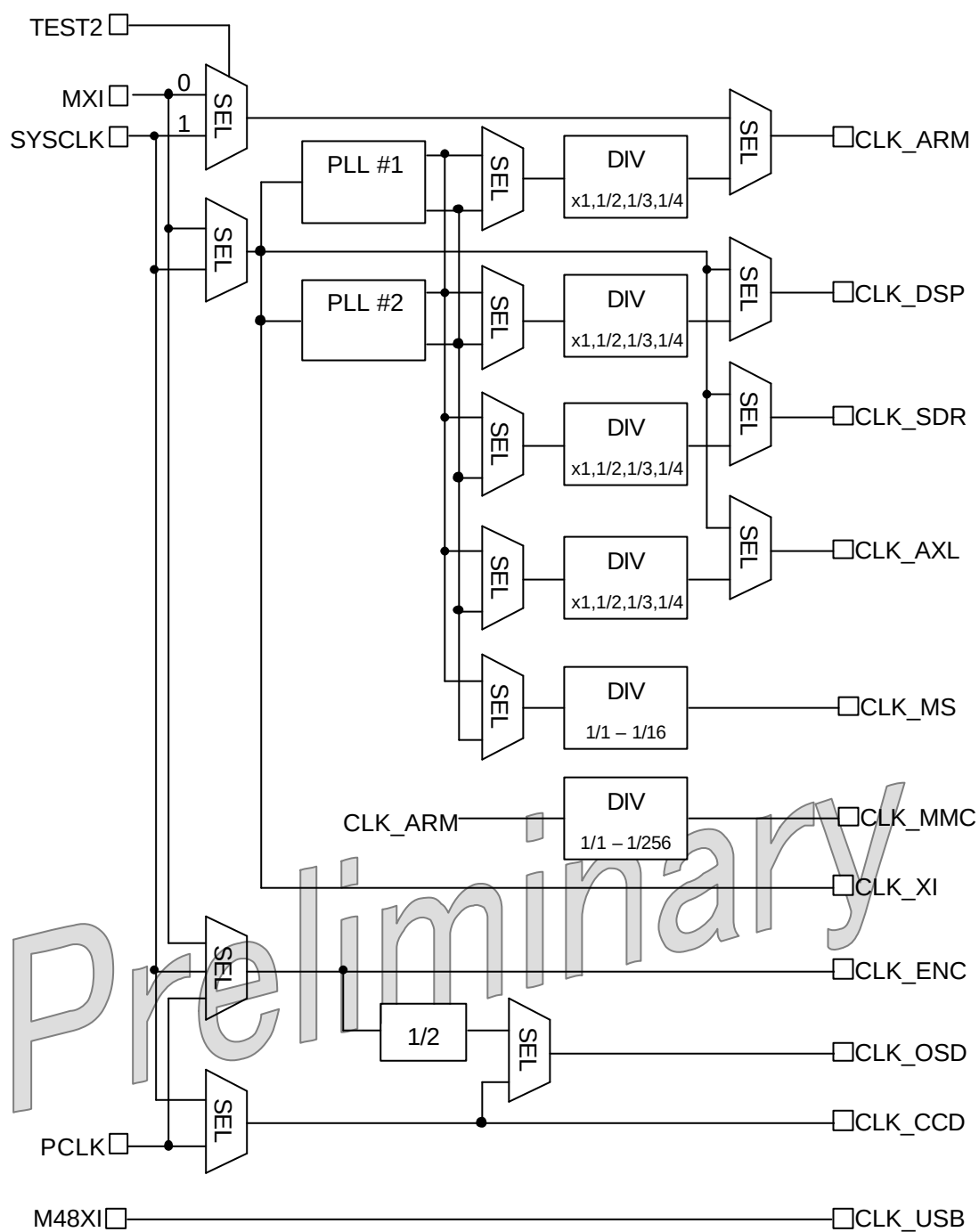


Figure 43 Clock Controller Block

6.8 ARM Boot-up Sequence

DM270 provides three different ARM boot-up sequences. There are self boot-up mode, serial boot-up mode and HOST boot-up mode.

6.8.1 Self boot-up mode

Upon power up and reset, the ARM program counter is reset to 0x000, which resides at the start of ARM Internal RAM. The ARM reset vector is hard coded in ROM at this location with an instruction to branch to 0x0010:0000 (start of the CS0 address space). The ARM should then execute a boot sequence from CS0 region to start the DM270 system. After turn-off the power on reset signal to first access to CS0(FLASH_CE) region takes about 10 ARM clock cycles. In this mode, the external pin of BTSEL should be tied to low(god) level.

6.8.2 Serial boot-up mode

DM270 is downloadable externally through serial interface(SIF0) and UART(UART0) pins. During reset, the external CPU can access to ARM internal memory by using SIF/UART. This interface consists of four signals serial clock input(SCLK0), enable signal input(SDEN0), serial data input(RXD0) and serial data output(TXD0). The external CPU can write a necessary data to ARM internal memory (AIM), for instance boot code, and read out the data from AIM.

The followings show the features:

- ❑ Full addressing from 0x0 to 0x007F:FFFF accessible
- ❑ Access mode is a byte mode only
- ❑ Address can be incremented automatically after gotten first address
- ❑ Support both of write and read
- ❑ Effectively during power on reset is low level, eg.during DM270 is on reset.
- ❑ Effectively during SDEN0 is low level

Figure 44 and Figure 45 show the timing diagram for serial interface. The first received bit indicates read/write command and it follows 15 bits address bits. When the command bit is low, being the write mode and when the command bit is high, being the read mode. Following the command and address bits, data of 8 bits length can be received in the write mode. In the read mode, data of 8-bits length read from AIM will be transferred through data out pin. The enable signal is active low. The address to be accessed is incremented automatically during enable signal is low. The minimum unit of the transfer is defined 8 bits. If the data are less than 8 bits will be discarded at the last transferring unit. The interface has below protocols.

- ❑ The data to be transferred is MSB first.
- ❑ The maximum frequency of SCLK0 is 10MHz.
- ❑ Performed while the power on reset signal is in assertion.

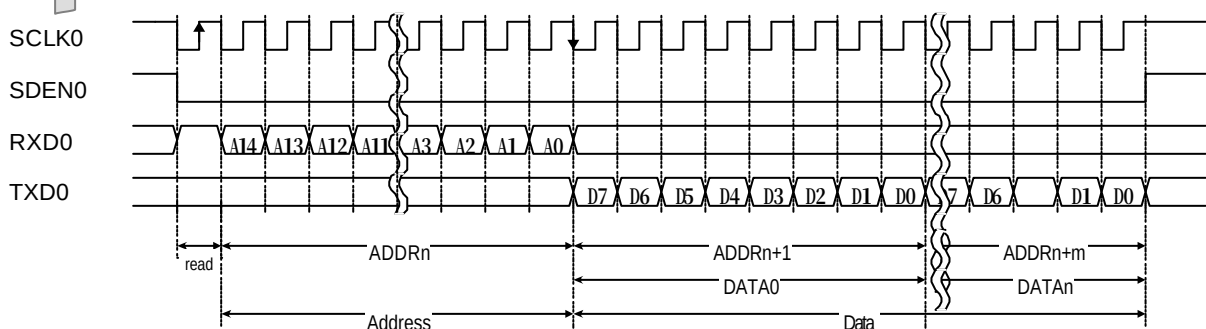


Figure 44 Serial data read timing chart

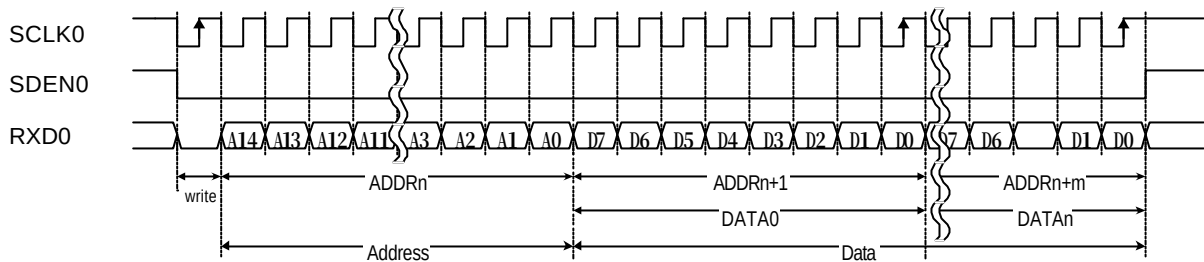


Figure 45 Serial data write timing chart

6.8.3 External CPU boot-up mode

External CPU can access to ARM internal memory directly through EMIF bus pin. It also enables downloading an ARM boot code externally.

6.8.4 Hardware configuration

The BTSEL pin defines how ARM will operate after power-on reset. Table 19 shows pin configuration.

Table 19 ARM boot-up configuration

Pin	Status	Function
BTSEL	0	Self boot up Jump to external Flash memory
	1	Serial/Host boot up Jump to ARM internal memory

7 Memory Traffic Controller

When processing data, it is often necessary to have access to large memory spaces. The internal memory of the DM270 chip is often too small to fit all the data and programs. To solve this problem, the external memory interface controller offers the possibility to access a large amount of external memory. Since the external memory interface is configurable, many different types of memories can be connected to the external interface. In fact the external memory interface includes two specialized interfaces to control and access CompactFlash and SmartMedia cards.

The memory traffic controller(MTC) can be separated into two sub-blocks: the external memory interface (EMIF) and the SDRAM controller(SDRAMC).

Another sub-block of memory traffic controller is DMA controller(DMAC). This DMA controller allows fast and efficient memory transfers between the different external memory regions.

7.1 Memory map

The complete memory map of the DM270 is represented in the Table 20. The external memory region consists of 128 Mbytes of ARM memory space that starts at address 0010:0000h.

Table 20 ARM memory address map

Region name	Start address	End address	Size (bytes)
Reset vector ROM	0x0000:0000	0x0000:0003	4
ARM internal memory ARM	0x0000:0004	0x0000:8003	32 K
Internal peripheral	0x0003:0000	0x0003:0FFF	4 K
DSP memory	0x0004:0000	0x0004:FFFF	64 K
Dynamically allocated external memory	0x0010:0000	0x080F:FFFF	128 M

After reset is canceled, the ARM program counter jumps to address 0 of the CS0 region.

The memory traffic controller is the main point of communication between the different modules of the DM270. The CCD Controller, Preview Engine, OSD, Image Buffer, HPIB, DMAC, EMIF, and ARM are connected directly to the SDRAM controller. The Image Buffer, HPIB and DMAC can be connected either to the SDRAM controller either to the EMIF. Depending on the memory location accessed, the ARM will either access the SDRAM controller, either the EMIF.

7.2 Blocks Diagram

The SDRAM controller is the main point of communication between the different modules of the DM270. The CCD Controller, Preview Engine, OSD, Image Buffer, DMAC, EMIF, and ARM are connected directly to the SDRAM controller. The CCDC can be connected either to the SDRAM controller either to the EMIF. Depending on the memory location accessed, the ARM will either access the SDRAM controller, either the EMIF.

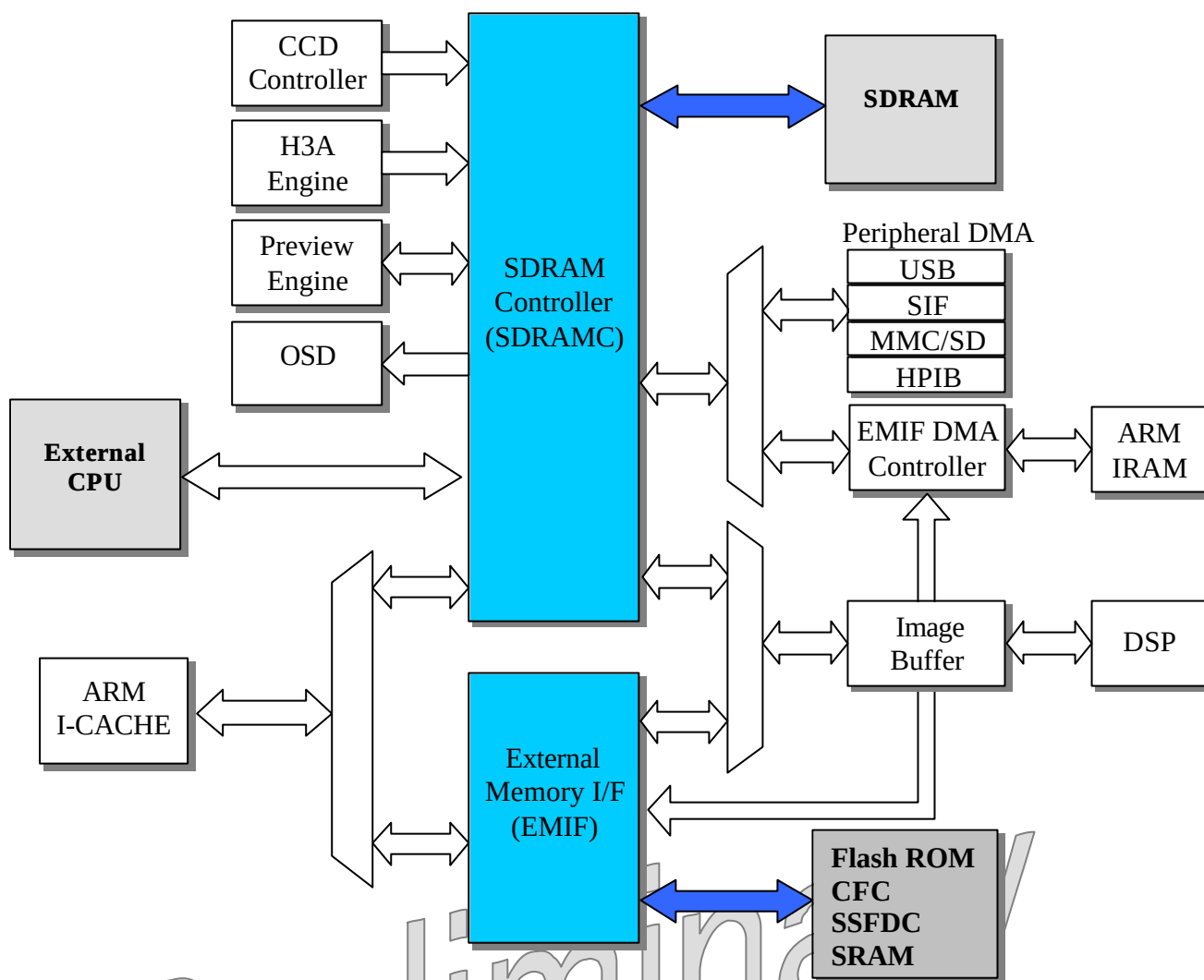


Figure 46 Memory Traffic Controller blocks diagram

7.2.1 Access to Memory Traffic Controller(MTC) Areas

The memory traffic controller can access 128Mbytes of memory separated into 6 regions:

CS0: external memory region 0 (FLASH memory interface)

CS1: external memory region 1 (CompactFlash interface)

CS2: external memory region 2 (SmartMedia interface)

CS3: external memory region 3 (general purpose device)

CS4: external memory region 4 (general purpose device)

SDRAM memory region

An SDRAM memory region is set up in the same way as the other external memory areas.

7.2.2 Boundary Addresses and Sizes of the External Memory Regions

Table 21 Initial values of offset

Variable region name	Initial value	Corresponding boundary address	Corresponding memory	Corresponding Chip Select
External memory region 0	09h	0090:0000h	FLASH memory	FLS_CE
SDRAM region	21h	0210:0000h	SDRAM	SDR_CS
External memory region 1	39h	0390:0000h	Compact Flash	CFE1,CFE2
External memory region 2	51h	0510:0000h	Smart Media	CFE1
External memory region 3	69h	0690:0000h	General	EM_CE0
External memory region 4	81h	0810:0000h	General	EM_CE1

If the regions are set so that they overlap, the chip select is valid for the region of higher priority ranking. If possible, try not to overlap memory areas.

Variable region name	Priority
External memory region 0	Low
SDRAM region	
External memory region 1	
External memory region 2	
External memory region 3	
External memory region 4	High

7.3 External Memory Interface

7.3.1 Hardware Connection

As external memory interfaces, DM270 has a general-purpose memory interface. The ARM can access SDRAM via the SDRAM controller module. The ARM can access general-purpose SRAM or flash ROM, a CompactFlash card, or a SmartMedia card via the external memory interface module. In access from ARM to external memory, a wait is automatically inserted, and its cycle varies depending on SDRAM frequency, external memory access time and so forth.

The functions of several pins in the of the external memory bus are shown in Table 23.

Table 23 External Memory Bus Pins

Pin Name	I/O	Explanation
ARM_A[22:0]	O	ADDRESS bus Byte address for 8 bit device and half word address for 16 bit device is outputted. ARM_A[22]: -REG for CF/CF+
ARM_D[15:0]	I/O	DATA bus
EM_WE	O	Write enable code (FLSH_CE, CFE1, CFE2, EM_CS0, EM_CS1)
EM_RDWR	O	READ/WRITE Strobe code
EM_BEH	O	Upper byte enable code
EM_BEL	O	Lower byte enable code
EM_OE	O	Read enable code (FLSH_SE, EM_CS0, EM_CS1)
EM_WAIT	I	External wait code (EM_CS0 or EM_CS1)
CFE1	O	CF/CF+ [-CFE1] Smart Media [-CE]
CFE2	O	CF/CF+ [-CFE2] Smart Media [ALE]
CFOE	O	CF/CF+ [-OE] Smart Media [CLE]
CFIORD	O	CF/CF+ [-IORD] Smart Media [-RE]
CFIOWR	O	CF/CF+ [-IOWR] Smart Media [-WE]
CFRDY	I	CF/CF+ [READY/-BUSY] Smart Media [READY/-BUSY]
CFWAIT	I	CF/CF+ [-WAIT]
IOIS16	I	CF/CF+ [-IOIS16]

7.3.2 Timing control

The timing for each external memory region can be configured by various register settings. This allows many different types of devices, which have different interface timing requirements, to be connected to each of the external memory regions. Also, an idle state can be inserted in order to prevent bus collision when performing continuous memory accesses. Table 24, Table 25 and Table 26 show the configurable timing characteristics for each external memory region, and Figure 47 shows the corresponding signal timing diagrams.

Table 24 Configurable Timing Parameters for CS0/CS3/CS4

Parameter	Explanation	clock variable	Variable unit
CYCLE	Cycle no.	2 ~16 clock variable	1 CLK unit

CS_SU	Setup no. of CS	0~15 clock variable	1 CLK unit
WE_SU	Setup no. of WE	0~15 clock variable	1 CLK unit
OE_SU	Setup no. of OE	0~15 clock variable	1 CLK unit
CS_WAIT	Wait no. of. CS	1~16 clock variable	1 CLK unit
WE_WAIT	Wait no. of. WE	1~16 clock variable	1 CLK unit
OE_WAIT	Wait no. of. OE	1~16 clock variable	1 CLK unit
IDLE	Address no.	0~3 clock variable	1 CLK unit

Table 25 Configurable Timing Parameters for CS1

Parameter	Explanation	clock variable	Variable unit
CYCLE	Cycle no.	2~32 clock variable	1 CLK unit
CS_SU	Setup no. of CS	0~15 clock variable	1 CLK unit
WE_SU	Setup no. of WE	0~15 clock variable	1 CLK unit
OE_SU	Setup no. of OE	0~15 clock variable	1 CLK unit
CS_WAIT	Wait no. of. CS	1~32 clock variable	1 CLK unit
WE_WAIT	Wait no. of. WE	1~32 clock variable	1 CLK unit
OE_WAIT	Wait no. of. OE	1~32 clock variable	1 CLK unit
IDLE	Address no.	0~3 clock variable	1 CLK unit

Table 26 Configurable Timing Parameters for CS2

Parameter	Explanation	clock variable	Variable unit
CYCLE	Cycle no.	2~16 clock variable	1 CLK unit
WE_SU	Wait no. of WE	0~15 clock variable	1 CLK unit
OE_SU	Wait no. of OE	0~15 clock variable	1 CLK unit
WE_WAIT	Wait no. of. WE	1~16 clock variable	1 CLK unit
OE_WAIT	Wait no. of. OE	1~16 clock variable	1 CLK unit
READ_SU	Set up no. of read	3~18 clock variable	1 CLK unit
IDLE	Address no.	0~3 clock variable	1 CLK unit

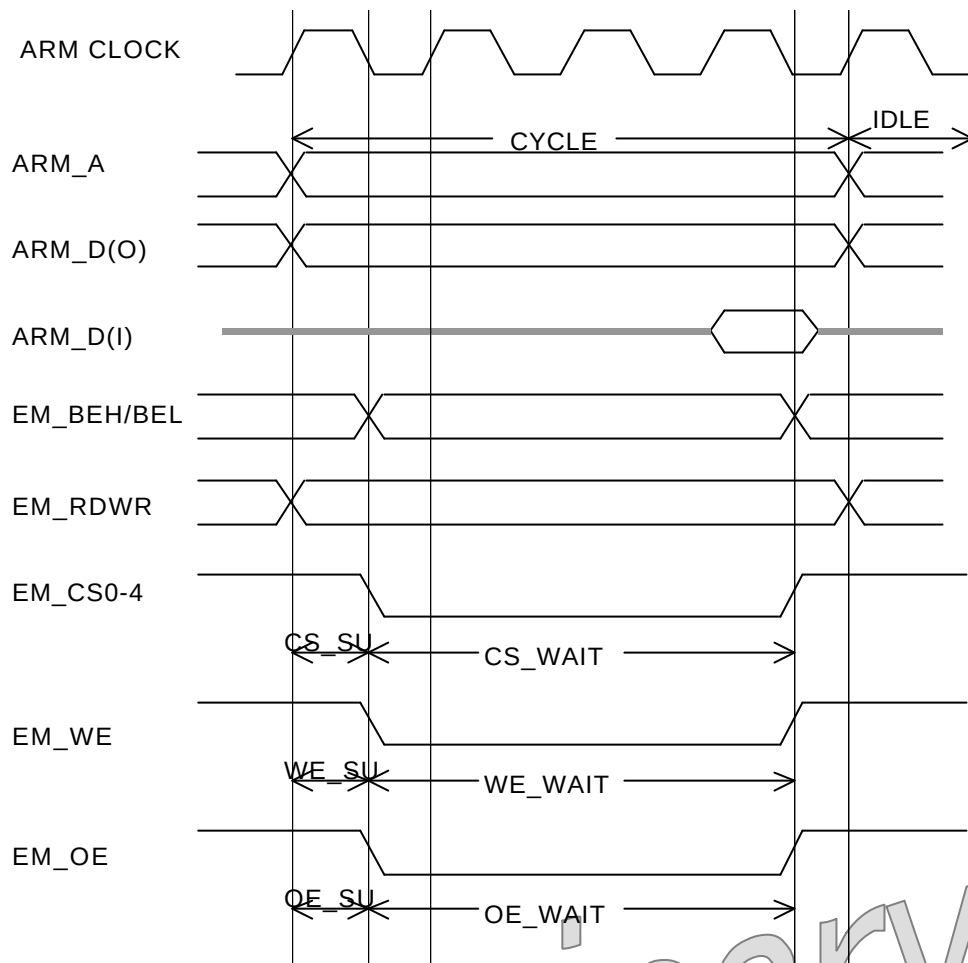
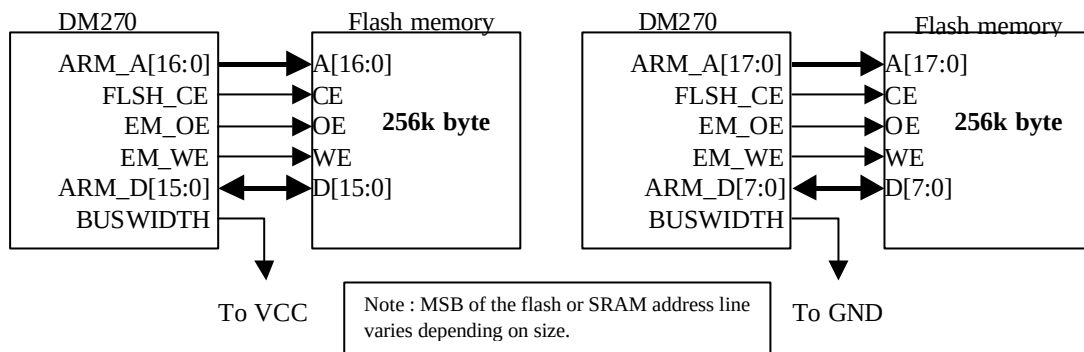


Figure 47 External Memory Bus Timings

7.3.3 External Memory Region 0: The FLASH Memory Interface

Systems that use asynchronous FLASH ROM for program memory should attach the FLASH ROM to external memory region 0, since the ARM reset vector includes an instruction to jump to the start address of external memory region 0. The start address of external memory region 0 is 0x10:0000, and this address cannot be changed.

Figure 48 shows the FLASH memory interface. The FLASH interface uses ARM_A, FLSH_CE, EM_OE, EM_WE, ARM_D, and BUSWIDTH. The write enable signal (EM_WE) is used commonly by all devices connected to the external memory interface. When the ARM accesses the flash region, FLSH_CE becomes active. In addition to FLASH memory, it is also possible to connect SRAM or PROM devices to the FLASH interface.

**Figure 48 Flash Memory Interface Data Bus**

External memory region 0 supports 8-bit and 16-bit wide FLASH. As shown in Table 27, the BUSWIDTH pin is used to set the data bus width of the region 0 to 8 or 16 bits.

Table 27 FLASH_CE Bus Width

BUSWIDTH state	FLASH_CE Bus Width
Low	8-bit
High	16-bit

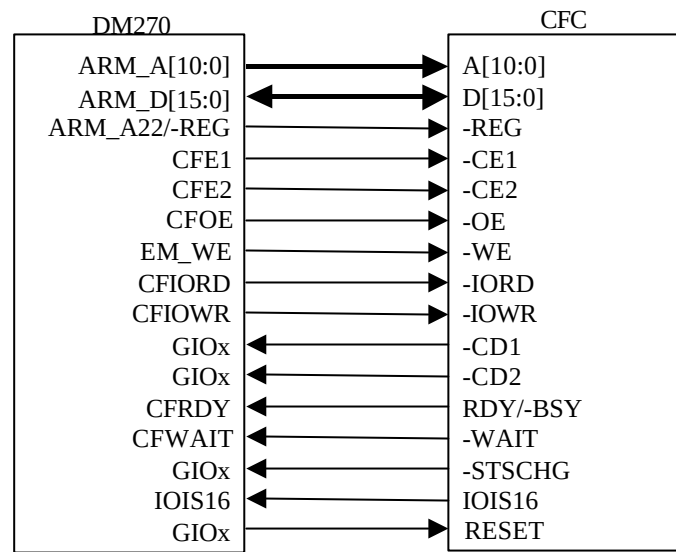
Although the FLASH data bus width can be set to 8 or 16-bits, a system may need to access 8, 16, and/or 32-bit data from the FLASH memory. As shown in Table 28, the external memory interface will automatically perform the correct number of accesses for an 8, 16, or 32-bit accesses, given an 8 or 16-bit wide FLASH memory. When using an 8-bit wide FLASH memory, 1 access is required for an 8-bit access, 2 accesses are required for a 16-bit access, and 4 accesses are required for a 32-bit access. When using a 16-bit wide FLASH memory, 1 access is required for an 8-bit access, 1 accesses are required for a 16-bit access, and 2 accesses are required for a 32-bit access. To perform an 8-bit read from a 16-bit wide FLASH memory, the required 8-bit portion is loaded and the remaining 8-bits are.

Table 28 Relationship Between Access Size and Flash Memory Data Bus Width

Access Size	8 bits wide	16 bits wide
8-bit access	1 time	1 time
16-bit access	2 times	1 time
32-bit access	4 times	2 times

7.3.4 External Memory Region 1: The CompactFlash Interface

External memory region 1 can be used as to interface CompatFlash cards. The CompactFlash interface is shown in Figure 49. The interface supports memory mode and IO mode, however, true IDE mode is not supported.

**Figure 49 CompactFlash Interface**

The CompactFlash interface supports dynamic bus sizing. The data bus width will be 8-bits or 16-bits depending on the level of the IOIS16 pin. When IOIS16 is low, the CompactFlash interface operates with an 8-bit bus, and when IOIS16 is high, the interface operates with a 16-bit bus. Similar to the FLASH interface, the correct number accesses are automatically performed when 32-bit, 16-bit, and 8-bit data is transferred, given that the data bus is set to be 8 or 16-bits.

The CompactFlash Interface Memory Map is shown in Table 29. The table shows the offset address for the CFC memory space and the CFC IO space. The ARM software accesses the memory space and IO space starting from the offsets at 00:0000 and 00:0800 respectively. The offset addresses are relative to the start address of external memory 1.

Table 29 CompactFlash Interface Memory Map

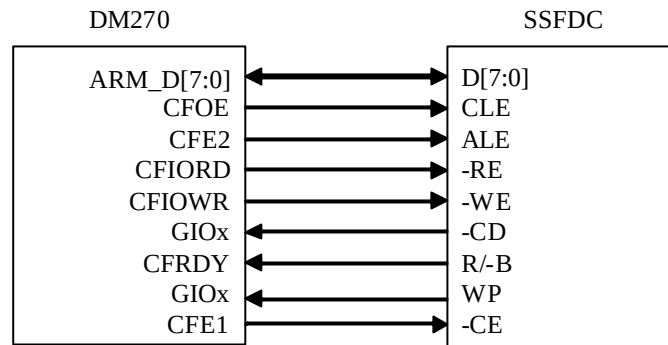
External Memory Area	Offset Address	Size	CompactFlash Interface Function
CS1	00:0000 – 00:07FF	2-Kbytes	CFC Memory Space (Attribute and Common)
	00:0800 – 00:0FFF	2-Kbytes	CFC IO Space

Accessing the CompactFlash memory space (attribute and common memory region)

When the CompactFlash card memory space is accessed (when operating in memory mode), the REG signal controls which region is accessed. The REG signal is output on pin ARM_A22. When REG is low in memory mode, the attribute memory region can be accessed. When REG is high in memory mode, the common memory region can be accessed. When operating in I/O mode, the REG signal is always output.

7.3.5 External Memory Region 2: The SmartMedia Interface

External memory interface has a built-in SmartMedia interface. Figure 50 shows the SmartMedia interface. The interface uses ARM-D[7:0], ARM_A20, ARM_A21, CFIORD, CFIOWR, CFRDY, and any available external GIO pin. See the TMS320DM270 for more detail on the SmartMedia interface.

**Figure 50 SmartMedia Interface**

Since SmartMedia uses one 8-bit data bus for command, address, and data, external memory interface 2 is an 8-bit wide interface. Similar to the FLASH interface, when 32-bit, 16-bit, and 8-bit data is transferred, an 8-bit access is automatically generated 4 times, 2 times, and 1 time respectively.

The SmartMedia Interface Memory Map is shown in Table 30. This table shows the offset address for command, address, and data. The ARM software should write SmartMedia commands to the address offset at 00:0000, write SmartMedia addresses to offset 00:0004, and should read and write SmartMedia data from offset 00:0008. The offset addresses are relative to the start address of external memory region 2.

Table 30 SmartMedia Interface Memory Map

External Memory Area	Offset Address	Size	Smart Media Interface Function
CS2	00:0000	4-bytes	Smart Media Command Write
	00:0004	4-bytes	Smart Media Address Write
	00:0008	4-bytes	Smart Media Data Read/Write

The DM270 supports SmartMedia error detection and correction.

The SmartMedia interface supports hardware error detection. During a read, hardware error correction code (ECC) computation is performed every 256-bytes. The result of the computation can be read from ARM register in EMIF. Although the hardware performs ECC computation, the ARM software must perform ECC data correction. If 512+16 byte page smart media is used, 256 bytes of data is read, then the ECC data is evacuated all at once, and after the ECC data is cleared.

7.3.6 External Memory Regions 3 and 4

The width of external memory areas 3 and 4 can be either 8 or 16 bits.

The EM_WAIT signal is available for region 3 and region 4, which enables asynchronous device interface capability. This feature is not available in any other external memory region. When EM_WAIT is enabled for CS3 or CS 4, each time one of the regions is accessed. If EM_WAIT is driven low during a bus cycle by an external device, an extra wait state (hardware wait-state) is added to the programmed wait states (software wait-states).

For instance, if CS_WAIT = 5 and EM_WAIT is held low for 5 clock cycles, the total number of wait states will be 10.

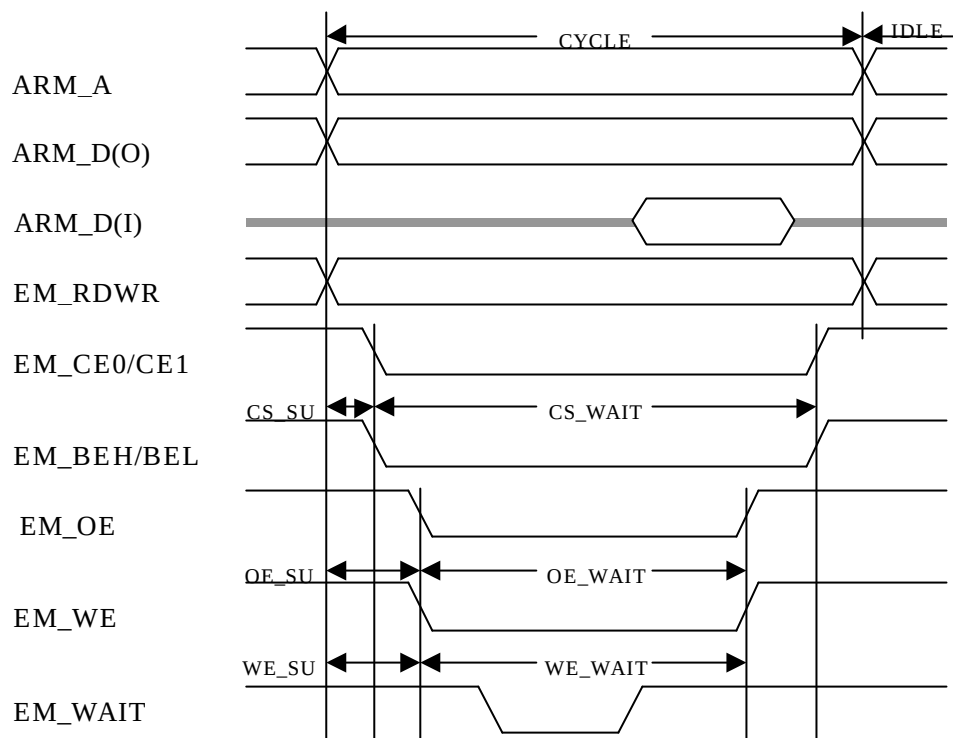


Figure 51 External memory bus timing with hardware wait-states

The EMIF bus can be opened for an external device by software. By setting the external memory bus open register in the external memory interface module, all I/F signals (ADDRESS/DATA/CS*/WE/OE) are opened (output Hi-Z). By using this function, it is possible to share memory among external memories. In addition, in DM270, the SDRAM bus can be opened to an external device. All SDRAM buses (ADDRESS/DATA/RAS/CAS/CS/WE/DQM/CKE/CLK) can be opened (output Hi-Z) by setting the SDRAM bus open registers in the external memory interface module. By using this function, SDRAM can be shared between external devices or can be tuned off the power of SDRAM.

7.4 SDRAM controller

The SDRAM controller, shown in Figure 52, acts as the primary interface between SDRAM and all functional blocks such as the processors (ARM, DSP), CCDC, Preview Engine, OSD and DMA controller. This block supports SDRAM timing of a maximum of 100, and provides continuous data access with low overhead. The start address and size of the SDRAM memory region are configurable within the external memory region of the ARM memory map. The SDRAM controller supports a DMA controller interface. The SDRAM access priority can be modified. The SDRAM controller supports refresh commands and can place the device in a self-refresh mode prior to go in a power down mode. The refresh interval is programmable. The latency for the column access strobe (CAS) is programmable 2 or 3.

7.4.1 Features:

- ❑ Supports 16/64/128/256/512M SDRAM
- ❑ Supports both 32-bit-wide SDRAM or 16-bit-wide SDRAM
- ❑ Maximum 80 MHz operation
- ❑ Access in WORD, HALF WORD, and BYTE units
- ❑ Mode setting, power down and self-refresh commands
- ❑ Programmable auto-refresh interval
- ❑ CAS latency 2 or 3 cycles can be selected
- ❑ Maximum SDRAM size 64 Mbytes
- ❑ DMA transfer with peripheral modules
- ❑ Variable SDRAM access priority
- ❑ SDRAM Controller Access

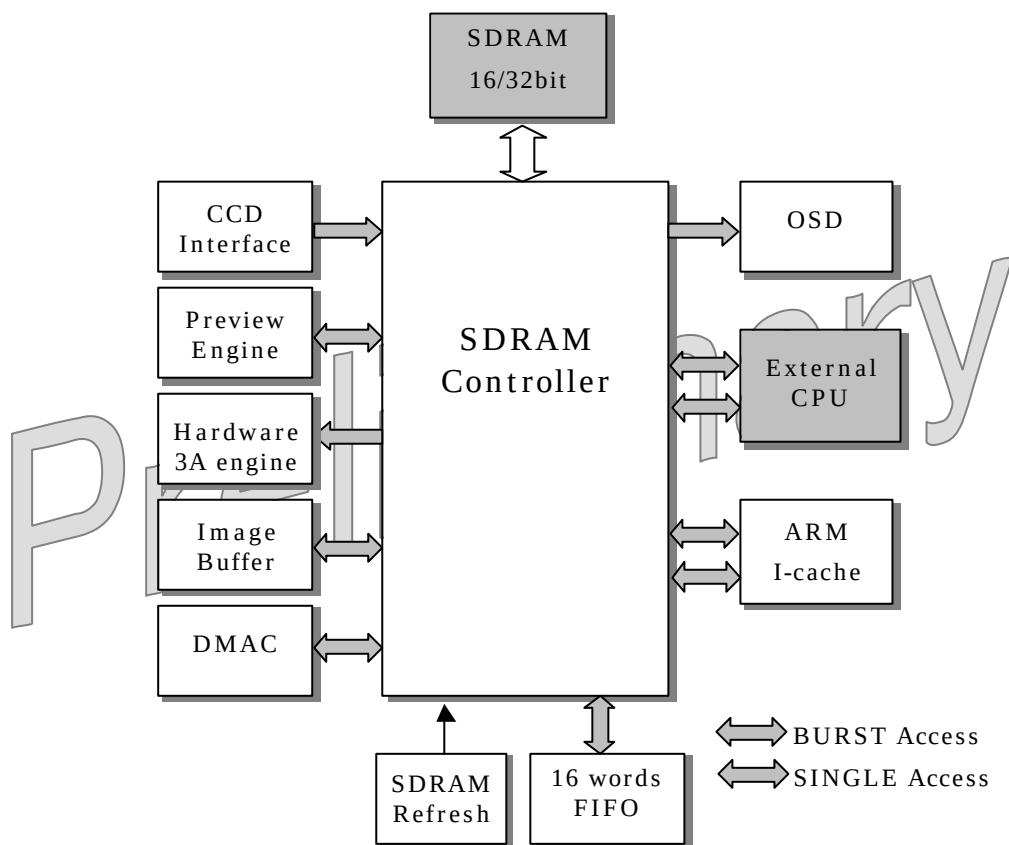


Figure 52 SDRAM Controller Block

The SDRAM region can be set within the external memory region, similar to the other external memory regions 0-4. The region is set by specifying the physical address of the external memory region in the start address register. After reset, it becomes 0x900000h.

The SDRMA controller can access 512-Mbits (64-Mbytes) of SDRAM and supports both 16 and 32-bit SDRAM devices. Each access can be done in word, half, word or byte units from the ARM side.

7.4.2 Hardware Connection

The functions of several pins in the of the SDRAM bus are shown in Table 31.

Table 31 SDRAM Bus Pins

Pin Name	I/O	Explanation
SDR_DQ[31-16]	I/O	SDRAM data bus (32bit mode)
SDR_DQ[15-0]	I/O	SDRAM data bus (16/32bit mode)
SDR_A[12-0]	O	SDRAM address bus
SDR_A[14-13]	O	SDRAM bank address
SDR_RAS	O	SDRAM Row address strobe
SDR_CAS	O	SDRAM Column address strobe
SDR_WE	O	SDRAM Write enable
SDR_CS0	O	SDRAM Support chip select
SDR_CKE	O	SDRAM clock enable
SDR_DQMHH	O	SDRAM mask for DQ[31:24]
SDR_DQMHL	O	SDRAM mask for DQ[23:16]
SDR_DQMLH	O	SDRAM mask for DQ[15:8]
SDR_DQMLL	O	SDRAM mask for DQ[7:0]
SDR_CLK	O	SDRAM clock

7.4.3 Memory Structure Setting

The DM270 can connect to SDRAM of a maximum of 512Mbits (64-Mbytes). The address that is connected differs depending on the memory structure of the SDRAM, as follows.

Table 32 Correspondence between memory structure and address

Memory structure	ADDRESS	Number of banks	BANK
2k x 256 WORD	SDR_A10-SDR_A0	2	SDR_A13
		4	SDR_A14:13
4k x 256 WORD	SDR_A11-SDR_A0	2	SDR_A13
		4	SDR_A14:13
4k x 512 WORD	SDR_A11-SDR_A0	2	SDR_A13
		4	SDR_A14:13
8k x 512 WORD	SDR_A12-SDR_A0	2	SDR_A13
		4	SDR_A14:13

7.4.4 SDRAM command

The ARM can issue the following commands to SDRAM by settings in the registers in the SDRAM controller :

MRS (mode register set)

PREA (pre-charge all)

REF (auto refresh)

SELF (self refresh entry)

SELFC (self refresh end)

PDN (power down mode)

When an MRS command is issued, the following data settings are performed in the SDRAM mode register.

MRS Value

SDRAM Address pin	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
Meaning	Reserved				0	0	Read Latency			S/I	Burst length		
DM270 output value	0	0	0	0	0	0	0	CL	0	0	0	BST 1	BST 0

7.4.5 SDRAM Initialization

SDRAM has a power on procedure. Since the procedure differs depending on the SDRAM that is used, SDRAM must be initialized by software using the SDRAM command. Also, since the DQM (input/output mask) signal is high output as default, after the mode register set command (MRS) is finished, DQM must be set to low level. These SDRAM settings are performed by setting in registers in the SDRAM controller.

A typical sequence is given below:

Apply power and start the clock. Attempt to maintain the NOP command at the input.

Maintain stable power, stable clock, and the NOP condition for a minimum of 100us or 200us. (Minimum time depends on devices. Please refer to the datasheet.)

Pre-charge all banks by the Pre-charge All (PREA) command.

Assert a minimum of 2 or 8 Auto-refresh commands. (Assert times depend on the devices. Please refer to the datasheet)

Program the mode register by the Mode Register Set command (MRS).

7.5 EMIF DMA Controller

The External Memory Interface has a built-in DMA controller. As shown by the solid arrows in Figure 53 EMIF DMA Controller, the EMIF's DMA controller can transfer data between SDRAM and external memory regions 0-4. This diagram also shows that data can be transferred between external memory and the DSP's image buffer or HPIB. However, data is transferred between the image buffer and external memory by the Image Buffer's own built-in DMA controller not the DMA controller built into the EMIF. Refer to the chapter on the DSP Subsystem for information on the Image Buffer's DMA controller.

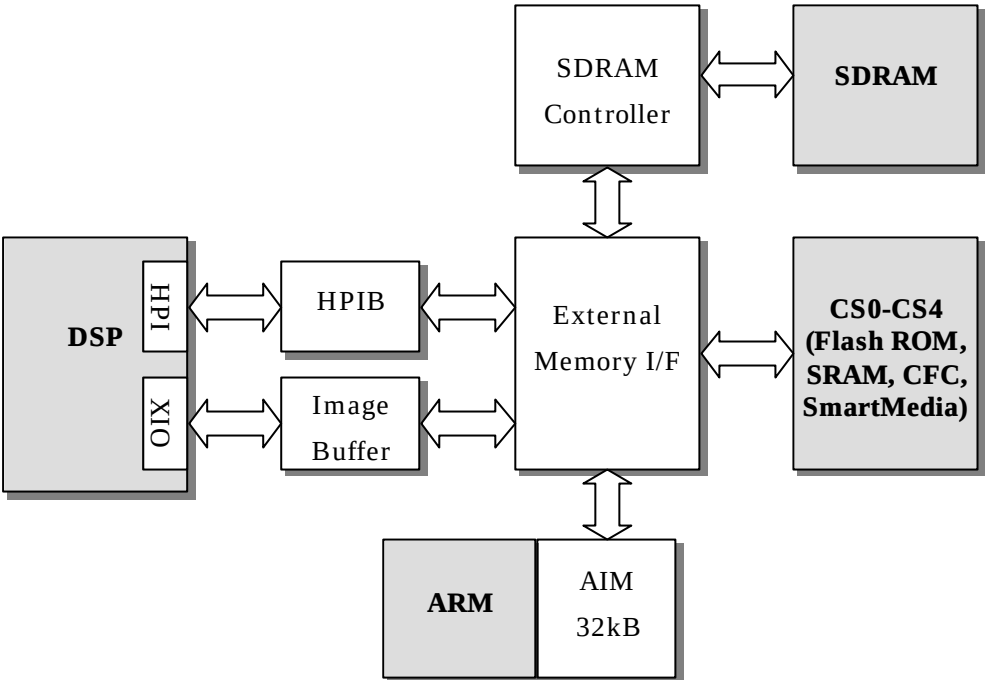


Figure 53 EMIF DMA Controller

7.5.1 Sources and destinations devices

To setup a DMA transfer using the EMIF’s DMA controller you must configure the destination device, the destination and source addresses, and the transfer size.

Table 33 DMA transfer source, transfer destination devices

Transfer source device	Transfer destination device
External general-purpose memory CS0	External general-purpose memory CS0
External general-purpose memory CS1	External general-purpose memory CS1
External general-purpose memory CS2	External general-purpose memory CS2
External general-purpose memory CS3	External general-purpose memory CS3
External general-purpose memory CS4	External general-purpose memory CS4
SDRAM	SDRAM
ARM internal memory	ARM internal memory

7.5.2 Sources/Destinations Addresses and Transfer Size

The registers to configure the DMA source and destination addresses as well the transfer size are shown in the following figures and diagram. The addresses are relative to the start of the each region used, and the addresses must be a multiple of four. Also, the Transfer size must always be specified in a multiple of 4.

DSP’s Image Buffer and HPIB have a built-in DMA controller that can transfer data to the EMIF in addition to SDRAM the SDRAM controller.

The sharing of pins between SDRAM and flash memory is carried out on bus cycle by bus cycle basis. Each of SDRAM interface and flash memory interface has a separate chip select. When a bus cycle falls into the SDRAM's address space, the chip select of the SDRAM is asserted and the shared pins generate the SDRAM-related timing control signals to perform the cycle. Similarly for flash memory, if the cycle is its address space, its chip select is asserted and the shared pins generate the signals to perform the cycle. Note that it is possible that while ARM is accessing the flash memory (for example, to fetch an instruction), there may be concurrent DMA activities on the SDRAM. In such case, flash memory and the DMA will have to arbitrate for the control for the memory interface.

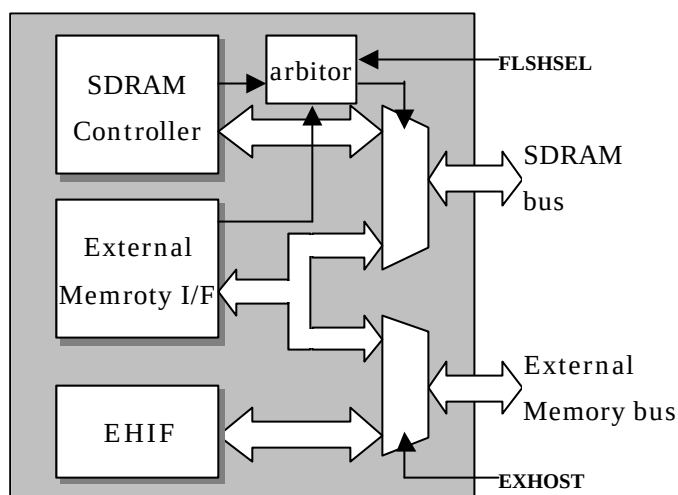


Table 34 shows DM270 pins that are shared between SDRAM and the External Bus (EXBUS). All the logic for multiplexing SDRAM interface signals and the EXBUS interface signals to the I/O pad is done by internal register and FLHSSEL/EXHOST pins. Table 36 shows the pin list which shared between SDRAM bus and external memory bus.

There are two types of usage for memory resource in DM270. The first type is the usage for the SDRAM memory, which is the same as DSC25. The masters of the SDRAM memory is the SDRAM controller(SDRC). The second type is the usage of SDRAM interface pins but not only the SDRAM memory. SDRC or External Memory Interface(EMIF) are the master that uses SDRAM bus pins and they are arbitrated. With this, a scheme can be implemented to allow the pins to be shared by the SDRAM external and EXBUS interface.

Pin sharing is required to support the optional media storage card hot plug within the constraint of pin count. The basic idea of hot plug is to have two separate buses for the media storage card (for example Compact Flash Card) and the flash memory so that plugs and unplug of the media card does not interfere with the normal operation of flash memory.

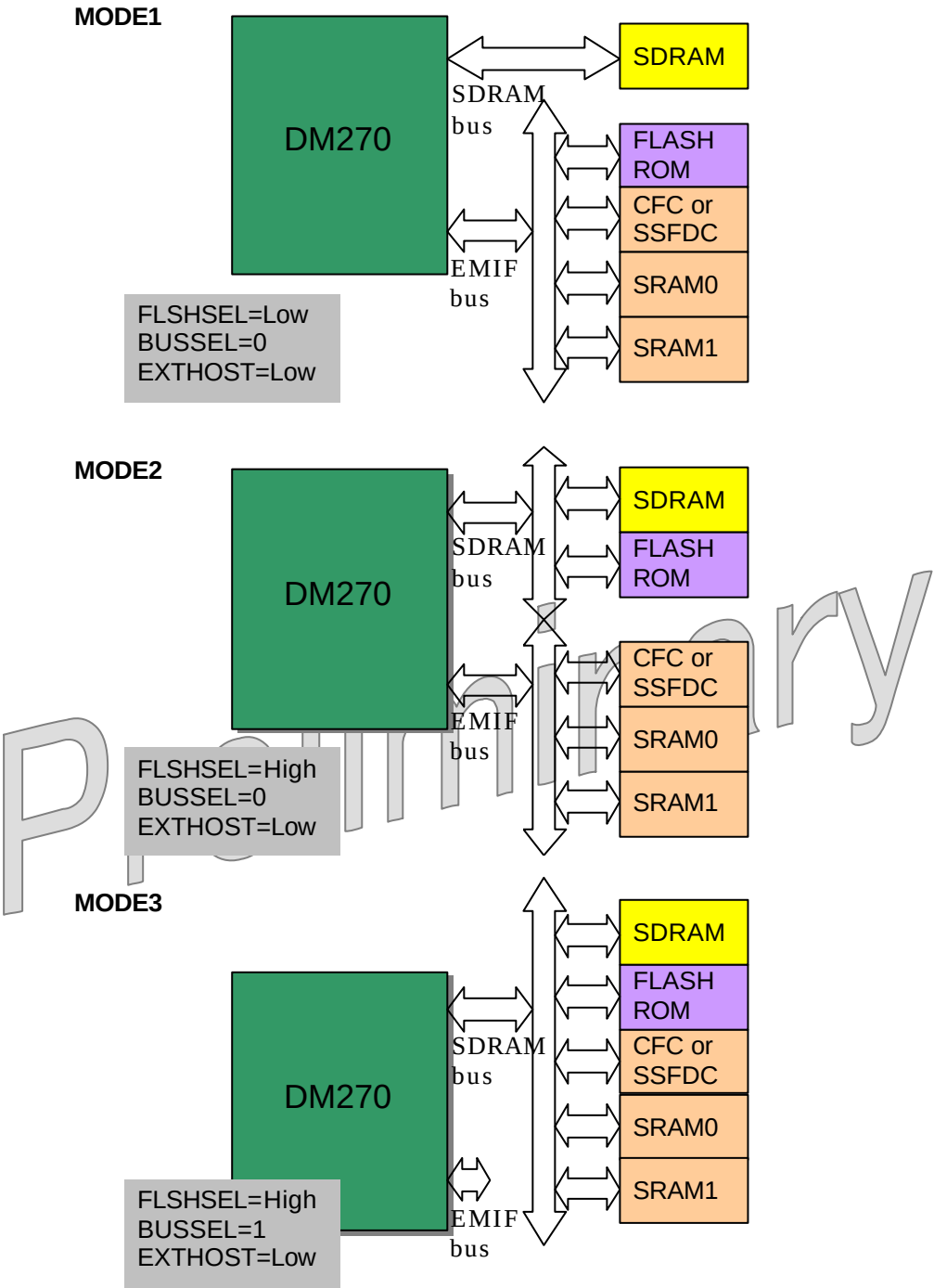
Another pin sharing method is required to support external HOST CPU connection with external memory bus. The external CPU can be a bus master by releasing all the external memory bus to it.

	FLSHSEL*	BUSSEL*	SDRAM	Flash ROM	CFC/SSFDC/ SRAM
MODE1	Low	0	SDRAM bus	Ex. Memory bus	Ex. Memory bus
MODE2	High	0	SDRAM bus	SDRAM bus	Ex. Memory bus
MODE3	High	1	SDRAM bus	SDRAM bus	SDRAM bus

Table 35 External HOST CPU Configuration

EXTHOST*	External Host CPU
Low	Not available
High	Available

*EXTHOST: pin name



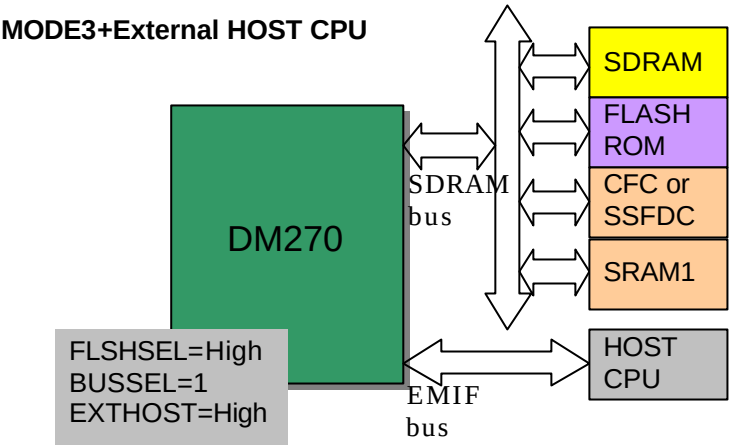


Figure 55 Meomry connecion on DM270

Table 36 Memory Bus Connection

Pin name	External Memory bus
SDR_DQ23	ARM_A22
SDR_DQ22	ARM_A21
SDR_DQ21	ARM_A20
SDR_DQ20	ARM_A19
SDR_DQ19	ARM_A18
SDR_DQ18	ARM_A17
SDR_DQ17	ARM_A16
SDR_DQ16	ARM_A15
SDR_A14	ARM_A14
SDR_A13	ARM_A13
SDR_A12	ARM_A12
SDR_A11	ARM_A11
SDR_A10	ARM_A10
SDR_A9	ARM_A9
SDR_A8	ARM_A8
SDR_A7	ARM_A7
SDR_A6	ARM_A6
SDR_A5	ARM_A5
SDR_A4	ARM_A4
SDR_A3	ARM_A3
SDR_A2	ARM_A2
SDR_A1	ARM_A1
SDR_A0	ARM_A0
SDR_DQ15	ARM_D15
SDR_DQ14	ARM_D14
SDR_DQ13	ARM_D13
SDR_DQ12	ARM_D12
SDR_DQ11	ARM_D11
SDR_DQ10	ARM_D10
SDR_DQ9	ARM_D9

SDR_DQ8	ARM_D8
SDR_DQ7	ARM_D7
SDR_DQ6	ARM_D6
SDR_DQ5	ARM_D5
SDR_DQ4	ARM_D4
SDR_DQ3	ARM_D3
SDR_DQ2	ARM_D2
SDR_DQ1	ARM_D1
SDR_DQ0	ARM_D0
SDR_RAS	EM_OE
SDR_CAS	EM_WE
SDR_DQ26	EM_RDWR
SDR_DQ25	EM_BEH
SDR_DQ24	EM_BEL

Preliminary

8 Communication Interface

8.1 USB Device Controller

DM270 incorporates a USB 1.1 compliant device controller.

8.1.1 Features

- ❑ Compliant with USB Specification Revision 1.1
- ❑ Involve UDC (USB Device Controller), the Protocol Engine
- ❑ Support only for Full Speed Device
- ❑ Built-in USB Xceiver
- ❑ Built-in 2K RAM for Endpoint FIFO
- ❑ Programmable MaxPktSize
- ❑ Understanding and Decoding of Standard Requests Except For GetDescriptor, SetDescriptor and SynchFrame
- ❑ Support of Device Remote Wakeup Feature
- ❑ DMA transferring capability

8.1.2 USB Functional Description

Figure 56 shows the block diagram of USB module. USB module has a built-in xceiver and 2K bytes RAM. USB module has 7 FIFOs and each FIFO is associated each Endpoint. The structure of each FIFO is same except for the direction and the buffer size. The maximum buffer size is 256 bytes for EP0 and EP2 while it is 2048 bytes for EP1, EP3 and EP4. Each FIFO's buffer is allocated onto a unified RAM. The base address and buffer size can be programmed by user.

ARM accesses FIFO read/write port registers to read from or write to FIFOs. DMA interface can be used to transfer data to or from a single FIFO. During a DMA transfer, ARM register access to the same endpoint as the DMA target is not allowed.

ARM clock frequency must be faster than 6MHz. No upper limit.

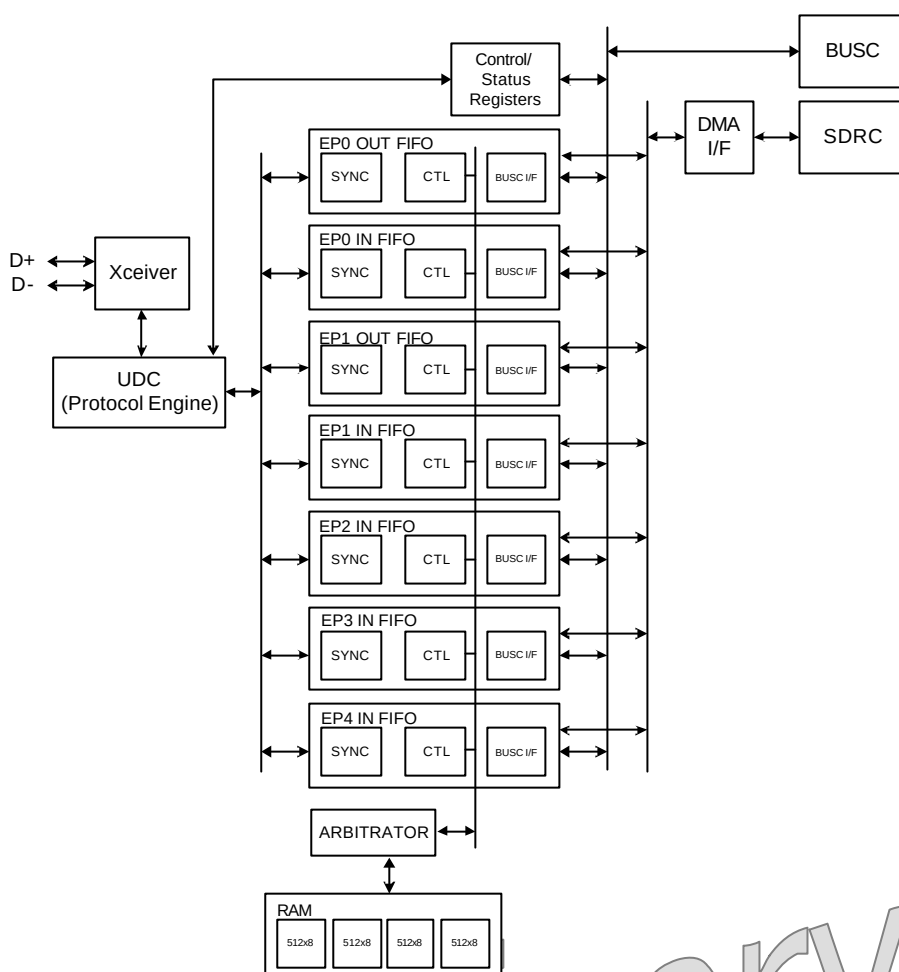


Figure 56 Block Diagram of USB

8.1.3 USB Device Configuration

Figure 57 shows the configuration which USB module supports. There are two configurations which are supported in this module. CfgA supports CBI (Control-Bulk-Interrupt) Transport defined in Mass Storage Class. CfgB is optionally provided to support the configuration defined in "Still Image Capture Device Definition".

User can configure the following property at device initialization;

Whether or not to use each Configuration

Each configuration number

Whether or not to use each Endpoint for each AlternateSetting

MaxPktSize of each Endpoint for each AlternateSetting

The above device initialization is done by writing the initialization data to USBCFG register. It must be done whenever the cable is attached. The format of the actual initialization data will be described in the next subsection.

User cannot change the number of Interface, Interface value, the maximum number of AlternateSettings of each Interface, Endpoint number and Endpoint type.

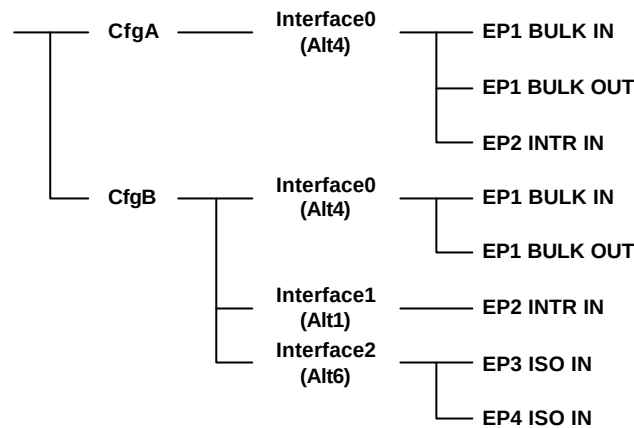


Figure 57 Device Configuration

8.1.4 USB Power Management

When USB bus is in suspend, all the clocks used in USB module except a clock used for resume detection can be cut-off by register for power saving. The clocks will be automatically activated as soon as the resume signal is detected.

If USB is not used, user can completely disable all the USB clocks by a register in CLK Controller module. In that case, more power saving can be achieved by connecting VDD pins for USB I/O to GND.

8.1.5 DMA Data Transfer

Internal DMA controller can be used to transfer data between SDRAM and USB endpoints without ARM intervention.

A DMA transmission from/to one endpoint FIFO is allowed in one time. User has to choose one endpoint as DMA target endpoint preceding DMA transfer. User should also specify SDRAM base address and transfer length. Then writing '1' to DON bit in DMATRG register starts DMA transmission. After the transmission of the specified amount of data is completed, ARM is notified the DMA completion by an interrupt.

For OUT endpoints, data stored in OUT FIFO can be transferred to SDRAM. If no data is received in FIFO, setting DON register does not start DMA. Also the number of data in OUT FIFO is less than DMA transfer length specified by user, the actual DMA transfer length is automatically clipped to actual length.

For IN endpoints, data to be sent to HOST can be transferred from SDRAM. If there is no space for the IN FIFO, setting DON register has no effect. Also the room for the FIFO is less than the specified DMA length, the actual DMA transfer is clipped by the actual room length.

User can also change DMA Endean. Options are LSB byte first, MSB byte first or byte swap in half-word.

8.2 SD/MMC Interface

As shown in Figure 58, the MMC passes data between the ARM and the DMA controller on one side and a memory card or cards on the other side. The ARM or the DMA controller can read from or write to the registers in the MMC Controller. As necessary, the ARM and/or the DMA controller can store or retrieve data in memory.

The MMC Controller initiates transfers between itself and the memory card(s). The communication follows either the controller's native MMC protocol or a protocol that is based on the SPI standard. The native protocol can use one bi-directional data line (for MMC) or four parallel data lines (for SD cards). The SPI protocol uses two serial data lines: one for storing data to the card and one for retrieving data from the card.

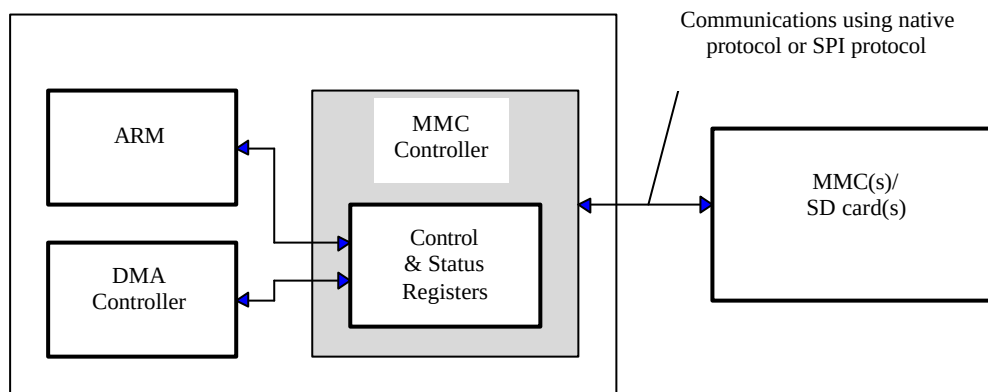


Figure 58 Role of the MMC Controller

The MMC Controller includes the following features:

- ❑ Support for a Multimedia Card (an MMC) or a Secure Digital Memory Card (an SD card)
- ❑ The capability to use either the MMC/SD protocol or the SPI protocol
- ❑ Software-oriented implementation for future extensions
- ❑ A programmable frequency for the clock that controls the timing of transfers between the MMC Controller and the memory card.
- ❑ Support for DMA transfer between MMC Controller and SDRAM

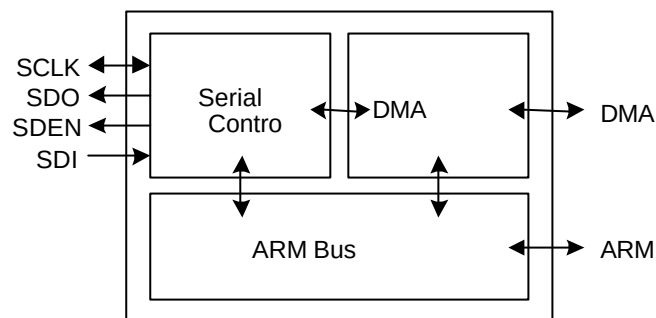
8.3 Memory Stick Interface

The DM270 supports Memory Stick communications(ver1.3/Pro); it contains a Memory Stick controller. With the Memory Stick controller, the DM270's ARM core can read from or write to a Memory Stick.

8.4 Serial Port

The DM270 has a two-channel clock synchronized serial port interface (SPI). The transfer unit is 8 bits each time, and it is a simple three-line serial interface consisting of a clock line, output data line, and input data line. Serial port 0 has a DMA transfer function with SDRAM for transfer of large quantities of data. DM270 SPI has master mode and slave mode and which is selectable. The SPI can supply the serial clock when it is used in a master mode. The SPI also can be synchronized by the external serial clock input when it is used in a slave mode.

Both of the serial ports and the DMA channel are controlled by ARM registers.



8.5 UART

DM270 supports a two-channel two-line (RX, TX) start-stop synchronization serial port (UART). A 32-byte FIFO is built in for each the transmitting side and receiving side, and interrupts can be generated due to causes such as errors or various states of FIFO. Data is 8 or 7 bits. Even, odd or no parity bit can be selected, and 1 or 2 stop bits can be selected. Transmission/reception timing uses the ARM or X'tal clock. The maximum baud rate achievable is a 1/16 of the ARM or X'tal clock (in bps).

The UART module detects parity errors and overrun errors. Interrupt to ARM is generated when any of the following events occur:

- ☐ Amount of data in receiving-side FIFO exceeds set value
- ☐ Amount of data in transmitting-side FIFO exceeds set value
- ☐ Word received has an error
- ☐ Change occurs in DSR or CTS input(one channel only)
- ☐ The timeout period is reached

ARM controls the UART module by accessing the following internal registers.

- * Data transmitting-side/receiving-side registers (FIFO)
- * Bit rate register
- * Mode register
- * Receiving-side FIFO control register
- * Transmitting-side FIFO control register
- * Line control register

* Status register

8.6 I²C Interface

The DM270 includes an Inter Integrated Circuit(I²C) serial us interface that can act as single master. Both 100kbps and 400kbps I²C-bus system is implemented. The interface uses 7bit/10bit addressing. The DM270 initiates and terminates transfers and generates clock signals.

8.7 GIO

General-purpose IOs(GIOs) consists of 34 pins. GIOs serve two purpose. One is to provide users a general-purpose programmable I/O which gives a convenient way of performing simple control to external devices. The other is to provide additional dedicated interface signals to external devices. With the additional signals, DM270 is able to either increase the performance or the functionality of the overall system. Since the GIOs can not perform both functions at the same time, it is up to the users to trade off between general-purpose programming IOs and dedicated feature and performance enhanced functions.

8.7.1 Parallel IO mode

Each GIO port can be set to input or output. If a GIO port is set to input, the ARM can read the status of that GIO port by GIO port registers. Also can control the polarity of input signal. If a GIO port is set to input and set to be inverted, then the reverse polarity will be read from the GIO port registers.

In the GIO module, output is controlled using a set register and a clear register. When a GIO port is set to output, 1 is output when 1 is written in the relevant bit in the set register, and 0 is output when 1 is written in the relevant bit of the clear register. It is possible to control only the bits you wish to control, without affecting the output status of other GIO ports.

8.7.2 External Interrupt Settings

A GIO port can be used as an external ARM interrupt port. GIO0 through GIO15 can be used as external interrupt ports. A maximum of 16 external interrupts can be specified by settings in registers in the GIO module. These interrupt signals are supplied to the interrupt controller. Either rise or fall can be selected as the interrupt trigger, but the pulse width must be at least two clocks of the DM270 internal ARM clock.

8.7.3 GIO Functional Settings

GIO ports (GIO0-GIO33) can be used as dedicated function ports rather than general-purpose input/output ports. Dedicated ports can be selected by setting the proper bits in the GIO registers. A table of GIO ports and signals that can be selected is shown in Table 37.

After a reset all GIO pins are defined as non-inverted, general-purpose input.

Table 37 GIO port selection output

Pin name (primary)	Pin name (secondary)	I/O	Pin description
GIO0	INT0	I	External interrupt #0
GIO1	INT1	I	External interrupt #1
GIO2	INT2	I	External interrupt #2
GIO3	INT3	I	External interrupt #3
GIO4	INT4	I	External interrupt #4
GIO5	INT5	I	External interrupt #5
GIO6	INT6	I	External interrupt #6

GIO7	INT7	I	External interrupt #7
GIO8	DSP_BFSR1	I/O	DSP frame sync pulse input #1
GIO9	DSP_BCLKR1	I/O	DSP serial transfer clock input #1
GIO10	DSP_BDR1	I	DSP serial data input #1
GIO11	DSP_BFSX1	I/O	DSP frame sync pulse #1
GIO12	DSP_BCLKX1	I/O	DSP serial transfer clock #1
GIO13	DSP_BDX1	O	DSP serial data output #1
GIO14	DSP_CLKS1	I	DSP external clock input #1
GIO15	XF	O	DSP External flag output
GIO16	CLKOUT0	O	Clock output #0
GIO17	CLKOUT1	O	Clock output #1
GIO18	CLKOUT2	O	Clock output #2
GIO19	SDO1	O	Serial I/F#1 data output
GIO20	SDI1	O	Serial I/F#1 data input
GIO21	SCLK1	I/O	Serial I/F#1 clock input/output
GIO22	SDEN1	I/O	Serial I/F#1 transfer enable
GIO23	RXD1	I	UART #1 data input
GIO24	TXD1	O	UART #1 data output
GIO25	MIRQ MRST	O	ARM interrupt(IRQ) output WDT reset signal output
GIO26	C_WEN	I	CCD controller write enable
GIO27	CCDC_SHUT	O	CCD Controller shutter trigger
GIO28	FIELD_ID	O	Video Encoder Field ID output
GIO29	PWM1	O	PWM output #1
GIO30	PWM2	O	PWM output #2
GIO31	SCL	O	I2C clock output
GIO32	SDA	I/O	I2C transmit data
GIO33	EXT_LINE_ID	I	EVEN/ODD line control signal for Digital LCD I/F

Preliminary

9 External CPU Interface

This module allows a host device to access AIM, DSP Internal memory and SDRAM connected to DM270.

By setting the pin EXTHST to high level, a host device can access some of the memory areas of DM270. When in external CPU mode, ARM_A22-ARM_A0, EM_CE0, EM_WE and EM_OE become input pins. The data bus is fixed at 16-bits and uses ARM_D15-ARM_D0. Access from a host device can be performed dynamically by enabling EM_WAIT output for the host device to monitor the status of the DM270. DM270 can interrupt the host device through GIO25, after GIO25 has been configured to function as the MIRQ (interrupt) signal.

9.1 Features

- ❑ The data bus is fixed at 16-bits and uses ARM_D15-ARM_D0. Both read and write are half-word (16 bit) access.
- ❑ Access from a host device can be performed dynamically by enabling EM_WAIT output for the host device to monitor the status of the DM270 ('0':Busy, '1':ready). WAITEN register of the External CPU Interface must be set in advance to enable the EM_WAIT output.
- ❑ The access time seen from the host device varies depending on ARM, DSP and SDRAM clock frequency settings.
- ❑ 32byte Burst/Continuous Burst access to SDRAM is supported via sixteen 16bit data registers.
- ❑ Host device can issue an interrupt signal to the ARM by setting the register INT.
- ❑ If a read is performed continuously after read, EM_OE or EM_CE0 must once return to high level. RTRG register must be set in case EM_OE becomes high level and EM_CE0 stays low.

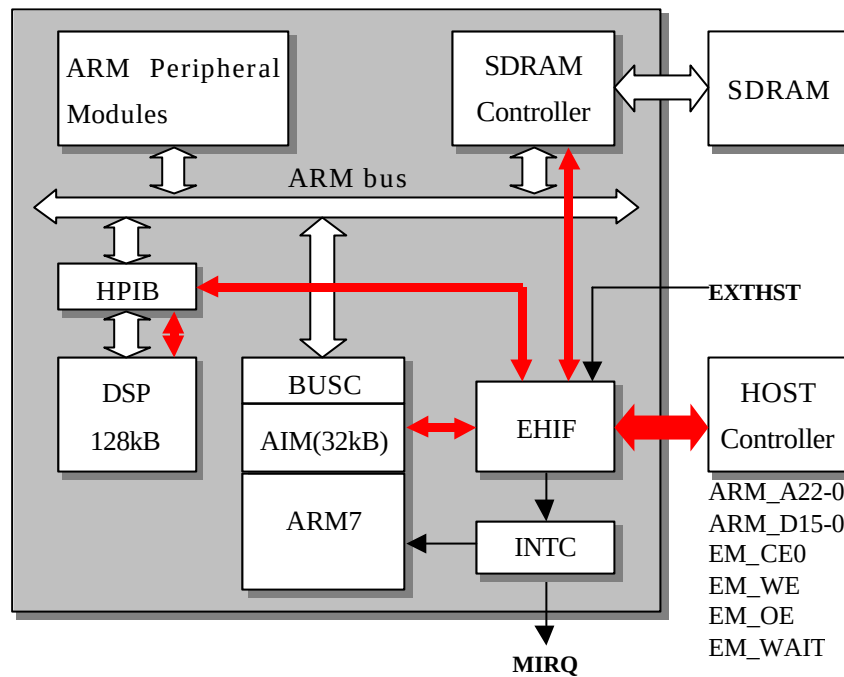


Figure 59 Host Interface mode

9.2 Memory Space

The memory areas that can be accessed from an external host controller are ARM internal memory (32 Kbytes), SDRAM (16,128Kbytes), and DSP internal memory (128 Kbytes). However, the memory map addresses differ from the usual ARM memory map. Table 38 shows the memory map of DM270 as seen from the external CPU.

Table 38 Memory Space

Memory Area	Start address	End address	size(byte)
External CPU registers	0x00:0000	0x00:007F	128
ARM Internal Memory	0x00:8000	0x00:FFFF	32 K
DSP Internal Memory	0x02:0000	0x03:FFFF	128 K
SDRAM	0x04:0000	0xFF:FFFF	16,128 K

9.3 Access Time

The External Host I/F(EHIF) module operates by the ARM clock of the DM270. The access time seen from the host device varies depending on ARM, DSP and SDRAM clock frequency settings. The number of cycles of the ARM clock is shown below.

- EHIF register/SDRAM burst access/AIM write : 2 + α cycles (α :Input and/or Output delay)
- AIM read : 3 + α cycles
- SDRAM single access/DSP memory : 3 + n + α cycles (n:Peripheral wait cycles)

9.4 SDRAM access

In DM270, there are two types of access to SDRAM: burst access via a buffer, and ordinary memory access. Transfer to and from SDRAM is performed in 16-word units. There is a control register in the SDRAM controller for burst access as well as sixteen 16-bit data registers to store the data from 16-word burst transfer. To write into the SDRAM, data needs to be set in these data registers. After the write destination SDRAM address is set in the buffer address registers, a write command can be written in the buffer transfer control register. When the write command in this register is written, writing to the SDRAM is performed. To read from SDRAM, the read origin from SDRAM address must be set first. This information is stored in the buffer address registers. When a read command is issued by writing the control register, the data is read from SDRAM and stored in the data registers. You need to check the status bit to know if a transfer is completed or not.

Continuous Burst access to SDRAM is also supported. When in Continuous Burst transfer mode, data of the specified length in 32 byte unit are transferred to and from SDRAM. There are 2 modes for Continuous Burst access.

Mode1 : Address decode enabled. Register, AIM, HPIB and SDRAM single accesses are allowed during Continuous Burst access. There are some registers to monitor the status of EHIF during Continuous Burst access which are:

Burst transfer wait flag : To check the Burst transfer status every 16 accesses. This is for a host device which does not have a WAIT input.

SDRAM access wait flag : To check the access status between EHIF and SDRC before you start access to AIM/HPIB/SDRAM(single) during Continuous Burst transfer, or before you end the Continuous Burst transfer.

Mode2 : No address decodes. No other access is allowed during DMA.

Preliminary

10 ARM-DSP Communication

This section describes the hardware and software interface between the ARM subsystem and the DSP subsystem. The Host Port Interface Bridge (HPIB) facilitates communication between the ARM and DSP.

10.1 Introduction

The DSP Controller in the DM270 controls data transfer and interrupts between the host and DSP, via the Host Port Interface Bridge (HPIB). DM270 has the Texas Instruments TMS320VC5409-equivalent DSP built into it. This DSP has an Host Port Interface (HPI), which allows communication between a host CPU, such as the ARM7 core used in the DM270.

As shown in Figure 60, the ARM and DSP can communicate through the HPI. The ARM (or external host) can access the DSP memory through a 128-Kbyte window into the DSP memory map. A software designer can define a communication protocol and data structures understood by the ARM and DSP to pass command requests, acknowledgements, and datagram between the ARM and DSP. HPIB module also has DMA controller inside, it enables data transfer from/to DSP internal memory to/from external memory (SDRAM, Flash ROM etc.) through memory traffic controller (MTC) module. This DMA controller is managed by DSP through XIO bus of DSP.

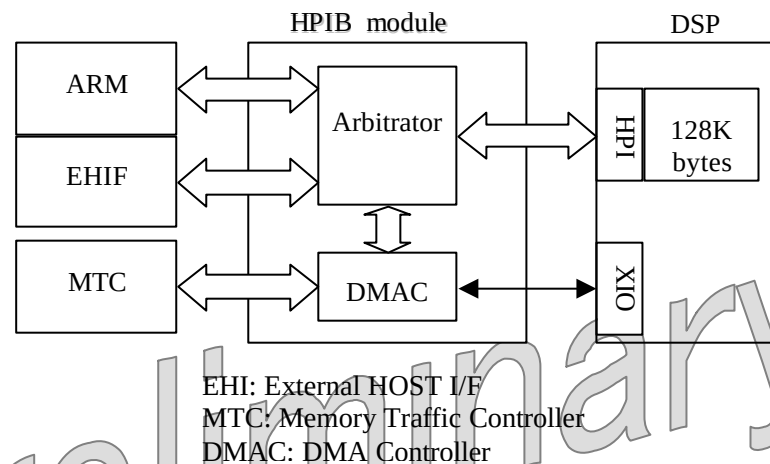


Figure 60 ARM-DSP Communication

10.2 HPI-16 mode

Normally, the HPI in the TMS320VC5409 DSP can be used in two modes, HPI-8 mode and HPI-16 mode. However the TMS320VC5409-equivalent DSP built into the DM270 can be used only in non-multiplexed HPI16 mode. In non-multiplexed HPI-16 mode, the ARM can read and write the DSP memory through a 16-bit bus. However, in this mode handshaking is not possible like it is in HPI-8 mode. Although handshaking is not possible, the ARM can still use the INT0 interrupt to interrupt the DSP, and the DSP can use an I/O port mapped to DSP address 0xFFFF to interrupt the ARM. For additional information on non-multiplexed HPI16 mode, refer to *TMS320C54x DSP Reference Set, Volume 5: Enhanced Peripherals*.

10.3 Memory Access

The DSP internal memory (32-Kwords of DARAM, 32-Kwords of SARAM) can be read and written by the ARM or an external host through the HPI-Bridge. Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM(data) and 0xC000-0xFFFFh of DSP SARAM(PAGE1:program) correspond to addresses 0004:0000-

0005:FFFF in the ARM memory map. Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM(data) and 0xC000-0xFFFFh of DSP SARAM(PAGE1:program) correspond to addresses 0002:0000-0003:FFFF in the external host memory map. The memory maps of the ARM and external HOST are shown in Table 39 and Table 40, respectively. When the endianness of the ARM and DSP do not match, EXCHG in HPIBCTL (0x00) can be used to swap the upper and lower 8-bits of data transferred between the ARM and DSP.

Table 39 ARM Memory Map

ARM address(byte)	DSP address(word)	DSP memory	Size(byte)
0x40000-0x4FFFF	0x0-0x7FFF	Data & Program(Page0)	64k
0x50000-0x57FFF	0x8000-0xBFFF	Data	32k
0x58000-0x5FFFF	0xC000-0xFFFF	Program(Page1)	32k

Table 40 External HOST Memory Map

ARM address(byte)	DSP address(word)	DSP memory	Size(byte)
0x20000-0x2FFFF	0x0000-0x7FFF	Data & Program(Page0)	64k
0x30000-0x37FFF	0x8000-0xBFFF	Data	32k
0x38000-0x3FFFF	0xC000-0xFFFF	Program(Page1)	32k

10.4 HPI Interrupt

The sections that follow explain how the host and the DSP interrupt each other.

10.4.1 Host Device Using DSPINT0 to Interrupt the DSP

The host (ARM) can interrupt the DSP by writing a 0 to the CPU register in DSP controller(DSPC) module.

10.4.2 DSP Using Bit 0 of 0xFFFF to Interrupt the Host Device

The DSP can interrupt the ARM through the memory mapped I/O port, 0xFFFF. The DSP's HINT pin is not used to interrupt the ARM, since the HPI is operating in a mode that cannot control the HINT pin (non-multiplexed HPI16 mode). The memory map of the DSP's I/O space is shown in Table 13, and as shown in the table, address 0xFFFF is used to interrupt the ARM from the DSP. This interrupt signal is connected to the ARM's interrupt controller and corresponds to INT11 in the interrupt controller.

10.5 Boot loader

10.5.1 ARM boot sequence

Upon power up and reset, the ARM program counter is reset to 0x000, which resides at the start of ARM Internal RAM. The ARM reset vector is hard coded in ROM at this location with an instruction to branch to 0x0100:0000 (start of the CS0 address space). The ARM should then execute a boot sequence from CS0 region to start the DM270 system.

The ARM boot sequence should initialize the different peripheral registers, such as the register for the clock controller module, the SDRAM controller, etc.

Also, the ARM boot phase should reset the DSP via the DSP Controller register bit. When done, the ARM should take the DSP out of reset. At this stage the DSP begins execution at 0xFF80, a hard coded address in ROM, which places the DSP into IDLE 1 mode (a power down mode). This gives the ARM the opportunity to download the DSP code by using the HPI-16 interface. After it completes downloading the DSP image, the ARM can send an interrupt to the DSP, which wakes it up from IDLE 1 mode and jump to address 0x7F80 where it starts running its application code loaded by the ARM.

10.5.2 DSP Boot Sequence

On the 'C54x DSP, the reset vector is located at address 0x0FF80. Within the DM270, this memory location has been ROM'ed with a small boot loader. The Figure 61 describes this process. It is achieved in 6 steps:

The DSP goes out of reset and load its program counter (PC) register with 0x0FF80

The ROM code found at this location branches the DSP to address 0x0F800 where the initialization routine resides.

At address 0x0F800, the DSP status register PMST is initialized to mode the vector table to 0x07F80, the interrupts is disabled except for INT0 and finally, the DSP is sent to a power down mode (Idle 1).

While in this mode, the ARM loads the DSP internal 64k words of Program/Data memory with the DSP code. The code being loading from the arm side is not predefined; refer to the DM270 library and application note for more information.

When the ARM is done downloading the DSP code, it wakes it up from IDLE mode by asserting INT0.

The DSP then branches to address 0x7F80 where the new interrupt vector table is located. The ARM should have loaded this location with at least a branch to the start of your code.

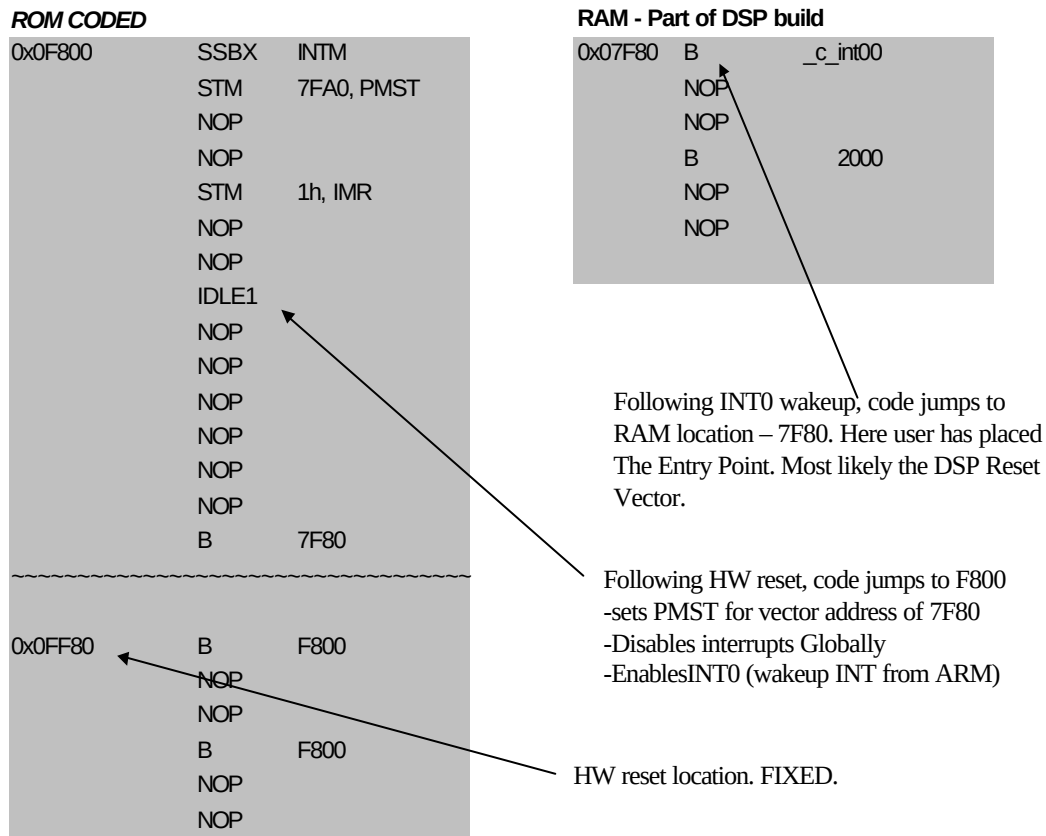


Figure 61 Host Interface mode

11 Power Saving Modes

The DM270 has been designed as a low power device, however, in some cases, it could be necessary to reduce even more the power consumption.

The following describes different modes that the DSP or ARM cores can be placed into to save power.

11.1 DSP Idle modes

The C54x DSP embedded in the DM270 has power-down modes in which it enters a dormant state and dissipates less power than normal operation while maintaining the CPU contents. This allows operations to continue unaltered when the power-down mode is terminated.

You can invoke one of the power-down modes either by executing the IDLE 1 IDLE 2 instructions.

11.1.1 IDLE1 Mode

The IDLE1 mode halts all CPU activities except the system clock. Because the system clock remains applied to the peripheral modules, the peripheral circuits continue operating and the CLKOUT pin remains active. Thus, peripherals such as McBSP and timers can take the CPU out of its power-down state.

Use the IDLE 1 instruction to enter the IDLE1 mode. To terminate IDLE1, use a wake-up interrupt. If INTM = 0 when the wake-up interrupt takes place, the C54x DSP enters the ISR when IDLE1 is terminated. If INTM = 1, the C54x DSP continues with the instruction following the IDLE 1 instruction. All wake-up interrupts must be set to enable the corresponding bits in the IMR register regardless of the INTM value. The only exceptions are the non-maskable interrupts, RS and NMI.

11.1.2 IDLE2 Mode

The IDLE2 mode halts the on-chip peripherals as well as the CPU. Because the on-chip peripherals are stopped in this mode, they cannot be used to generate the interrupt to wake up the DSP as with IDLE1. However, power is significantly reduced because the device is completely stopped.

Use the IDLE 2 instruction to enter the IDLE2 mode. To terminate IDLE2, activate any of the external interrupt signal (RS, NMI, and INT0) with a 10-ns minimum pulse. If INTM = 0 when the wake-up interrupt takes place, the C54x DSP enters the ISR when IDLE2 is terminated. If INTM = 1, the C54x DSP continues with the instruction following IDLE 2 instruction. All wake-up interrupts must be set to enable the corresponding bits in the IMR register regardless of the INTM value. Reset all peripherals when IDLE2 terminates, especially the McBSP when the CLKS pin is used.

When RS is the wake-up interrupt in IDLE2, a 10-ns minimum pulse of RS can activate the reset sequence.

11.2 ARM Power Saving Modes

The ARM7 subsystem used in the DM270 features several power-saving modes to lengthen the operating time on batteries powered systems. The content of the CPU and the registers settings are maintained when the device is in one of these modes.

Depending on the power down mode chosen, there is different ways to enter and exit it. Each one is described below.

11.2.1 Sleep Mode:

Clock supply to each module can be stopped by setting the module input clock off. This is done by setting the proper bits in the MODx control registers. If clock supply to the ARM core, ARM internal memory and bus controller is stopped, DM270 is completely stopped. Therefore, these clocks must be stopped last.

Even if all clocks in DM270 are stopped, RAM contents are saved as long as power is not lost, and it is not necessary to reload ARM and DSP programs. If you wish to save SDRAM contents, the SDRAM must be set to self-refresh mode before stopping the SDRAM clock.

Recovery from sleep is performed by GIO0. You need to apply a pulse signal on the GIO0 pin (active low) at least the width of two clocks of SYSCLK or X'tal clock. Since the PLLs still operate, the full recovery is quick.

11.2.2 Sleep by sleep mode register setting

Clock supply to all modules and the built-in PLLs can be stopped by setting the sleep mode register in the CLOCK controller. Recovery from sleep mode is performed by GIO0, but since it takes 8192 clock cycles until the internal PLLs are stable, this recovery time is slow.

Recovery using GIO0 is performed, but the recovery times are progressively slower for the three methods Sleep Mode:

When you write '1' into the sleep mode bit of the clock controller register (SLPMD bit of CLKC) the DSC enters into sleep mode. When in this mode, all the internal PLLs and internal clocks are stopped. The SLPMD is cleared by pulling the GIO0 pin to the ground for at least the width of the input clock.

11.2.3 Power-down mode

When the device is placed in a sleep mode by pulling the PWDN pin low, the oscillation of the external oscillator cell stops when entering the power down mode.

If X'tal (MXI, MXO) is used as the clock source to ARM, a more uniform power-down can be implemented by stopping its oscillation. The PWRDN terminal is used in power down of the oscillator. When this terminal is HIGH, oscillation is stopped. To recover, first set the PWRDN terminal to LOW, and wait for stable oscillation (about same time as power reset). After that, recovery using GIO0 is performed. Compared to the power down mode, the recovery time with respect to the crystal oscillation stabilizing time is longer.

Stopping of 48 MHz oscillators (M48XI, M48XO) can also be performed by setting registers in the CLOCK controller.

11.2.4 Wake up Method

It returns from above-mentioned powering down * mode, state of one time device it is necessary to make sleep * mode. The difference of powering down * mode and sleep * mode is whether the PWDN terminal '1' or '0' just. First, the PWDN terminal is reset to '0', oscillation of the input clock is made to put in stable state. Because (the same time as power on * reset) this state already has become sleep * mode, when low * pulse is inputted from the GIO0 terminal as an external interrupt, the edge is detected inside, the sleep * mode * bit is cleared, returns to normal state. Because as for on & off of the clock immediately before entering into sleep * mode, you follow setting, the ARM and the other things, as for the module whose operation is necessary the occasion where it returns unless it sets to on by all means, after the returning being not to operate note is necessary.

11.2.5 PLL Bypass Mode

The BPSMD (the CLKC [13]) by the fact that it sets to '1', the PLL becomes bypass * mode. Bypass * mode, discharge doing the VCO of all PLL, it points to the state of the input clock = output clock of the PLL. (It does not influence the M/N) it is mode just in order to make the electric power consumption of the PLL minimum.

11.3 DAC Power

If you want to tune off the power to unused DACs, you must do as follows:

The following conditions must always be met:

- Connect VSSA and VSS3V to ground
- Connect VDD3V to power (3.3V)

The voltage of pins IREF, DAOOUT, BIAS, and VREF_RGB must satisfy following formula:

$$\text{Voltage (VDDA)} \geq \text{Voltage (IREF,DAOOUT,BIAS,VREF_RGB)}$$

Thus, IREF, BIAS, and VREF_RGB must be LOW.

Because of terminal resistance connected to VSSA, DAOOUT goes to GND. DAOOUT is an output pin, so you (the user) don't have to worry about DAOOUT.

Preliminary

12 PIN Assignment

No	Name	I/O	Description
VIDEO INPUT			
105	CI7	I	Chroma(MSB) data input
106	CI6	I	Chroma data input
107	CI5/CCD13	I	Chroma data input CCD(MSB) data input
108	CI4/CCD12	I	Chroma data input CCD data input
109	CI3/CCD11	I	Chroma data input CCD data input
110	CI2/CCD10	I	Chroma data input CCD data input
111	CI1/CCD9	I	Chroma data input CCD data input
112	CI0/CCD8	I	Chroma(LSB) data input CCD data input
91	YI7/CCD7	I	Luminance(MSB) data input CCD data input
92	YI6/CCD6	I	Luminance data input CCD data input
93	YI5/CCD5	I	Luminance data input CCD data input
94	YI4/CCD4	I	Luminance data input CCD data input
95	YI3/CCD3	I	Luminance data input CCD data input
96	YI2/CCD2	I	Luminance data input CCD data input
97	YI1/CCD1	I	Luminance data input CCD data input
98	YI0/CCD0	I	Luminance(LSB) data input CCD(LSB) data input
102	HD	I/O	H-Sync
103	VD	I/O	V-Sync
104	FID	I/O	Field Indicator
100	PCLK	I	Pixel Clock
VIDEO OUTPUT			
71	YOUT7	I/O	Y OUT Signal
72	YOUT6	I/O	Y OUT Signal
73	YOUT5	I/O	Y OUT Signal
74	YOUT4	I/O	Y OUT Signal
75	YOUT3	I/O	Y OUT Signal
76	YOUT2	I/O	Y OUT Signal
77	YOUT1	I/O	Y OUT Signal
78	YOUT0	I/O	Y OUT Signal
81	COUT7	I/O	C OUT Signal
82	COUT6	I/O	C OUT Signal
83	COUT5	I/O	C OUT Signal
84	COUT4	I/O	C OUT Signal
85	COUT3	I/O	C OUT Signal
86	COUT2	I/O	C OUT Signal
87	COUT1	I/O	C OUT Signal
88	COUT0	I/O	C OUT Signal
69	HSYNC	O	H-Sync for RGB output
70	VSNC	O	V-Sync/Composite-Sync output
68	VCLK	O	Video Out Clock
ANALOG VIDEO OUTPUT			
125	IREF	I	DAC Current Control
126	DAOUT	O	DAC Composite Output
129	BIAS	I	DAC Bias with Compensation Capacitor

130	VREF	I	DAC Reference Voltage
GENERAL I/O			
16	GIO33	I/O	General I/O
17	GIO32	I/O	General I/O
18	GIO31	I/O	General I/O
19	GIO30	I/O	General I/O
20	GIO29	I/O	General I/O
23	GIO28	I/O	General I/O
24	GIO27	I/O	General I/O
25	GIO26	I/O	General I/O
26	GIO25	I/O	General I/O
27	GIO24	I/O	General I/O
28	GIO23	I/O	General I/O
29	GIO22	I/O	General I/O
30	GIO21	I/O	General I/O
31	GIO20	I/O	General I/O
32	GIO19	I/O	General I/O
33	GIO18	I/O	General I/O
34	GIO17	I/O	General I/O
35	GIO16	I/O	General I/O
36	GIO15	I/O	General I/O
38	GIO14	I/O	General I/O
40	GIO13	I/O	General I/O
41	GIO12	I/O	General I/O
42	GIO11	I/O	General I/O
43	GIO10	I/O	General I/O
44	GIO9	I/O	General I/O
45	GIO8	I/O	General I/O
46	GIO7	I/O	General I/O
47	GIO6	I/O	General I/O
48	GIO5	I/O	General I/O
49	GIO4	I/O	General I/O
50	GIO3	I/O	General I/O
51	GIO2	I/O	General I/O
52	GIO1	I/O	General I/O
53	GIO0	I/O	General I/O
SDRAM INTERFACE			
196	SDR_CLK	I/O	SDRAM I/F
193	SDR_RAS	O	SDRAM RAS Strobe
194	SDR_CAS	O	SDRAM CAS Strobe
195	SDR_WE	O	SDRAM WE
197	SDR_CS	O	SDRAM CS
198	SDR_CKE	O	SDRAM CKE
199	SDR_DQMHH	O	DQMH for SDR_DQ[31:24]
200	SDR_DQMHL	O	DQMH for SDR_DQ[23:16]
201	SDR_DQMLH	O	DQMH for SDR_DQ[15:8]
202	SDR_DQMLL	O	DQMH for SDR_DQ[7:0]
138	SDR_A14	O	SDRAM Bank 0
139	SDR_A13	O	SDRAM Bank 1
140	SDR_A12	O	SDRAM Address(MSB)
141	SDR_A11	O	SDRAM Address

142	SDR_A10	O	SDRAM Address
143	SDR_A9	O	SDRAM Address
144	SDR_A8	O	SDRAM Address
145	SDR_A7	O	SDRAM Address
146	SDR_A6	O	SDRAM Address
147	SDR_A5	O	SDRAM Address
148	SDR_A4	O	SDRAM Address
151	SDR_A3	O	SDRAM Address
152	SDR_A2	O	SDRAM Address
153	SDR_A1	O	SDRAM Address
154	SDR_A0	O	SDRAM Address(LSB)
155	SDR_DQ31	I/O	SDRAM Data (MSB)
156	SDR_DQ30	I/O	SDRAM Data
157	SDR_DQ29	I/O	SDRAM Data
158	SDR_DQ28	I/O	SDRAM Data
159	SDR_DQ27	I/O	SDRAM Data
160	SDR_DQ26	I/O	SDRAM Data
161	SDR_DQ25	I/O	SDRAM Data
162	SDR_DQ24	I/O	SDRAM Data
165	SDR_DQ23	I/O	SDRAM Data
166	SDR_DQ22	I/O	SDRAM Data
167	SDR_DQ21	I/O	SDRAM Data
168	SDR_DQ20	I/O	SDRAM Data
169	SDR_DQ19	I/O	SDRAM Data
170	SDR_DQ18	I/O	SDRAM Data
171	SDR_DQ17	I/O	SDRAM Data
172	SDR_DQ16	I/O	SDRAM Data
173	SDR_DQ15	I/O	SDRAM Data
174	SDR_DQ14	I/O	SDRAM Data
175	SDR_DQ13	I/O	SDRAM Data
176	SDR_DQ12	I/O	SDRAM Data
179	SDR_DQ11	I/O	SDRAM Data
180	SDR_DQ10	I/O	SDRAM Data
181	SDR_DQ9	I/O	SDRAM Data
182	SDR_DQ8	I/O	SDRAM Data
183	SDR_DQ7	I/O	SDRAM Data
184	SDR_DQ6	I/O	SDRAM Data
185	SDR_DQ5	I/O	SDRAM Data
186	SDR_DQ4	I/O	SDRAM Data
187	SDR_DQ3	I/O	SDRAM Data
188	SDR_DQ2	I/O	SDRAM Data
189	SDR_DQ1	I/O	SDRAM Data
190	SDR_DQ0	I/O	SDRAM Data (LSB)
EXTERNAL MEMORY INTERFACE			
260	ARM_A22	I/O	External Memory Address (MSB)
261	ARM_A21	I/O	External Memory Address
262	ARM_A20	I/O	External Memory Address
263	ARM_A19	I/O	External Memory Address
264	ARM_A18	I/O	External Memory Address
265	ARM_A17	I/O	External Memory Address
266	ARM_A16	I/O	External Memory Address

267	ARM_A15	I/O	External Memory Address
268	ARM_A14	I/O	External Memory Address
271	ARM_A13	I/O	External Memory Address
272	ARM_A12	I/O	External Memory Address
273	ARM_A11	I/O	External Memory Address
274	ARM_A10	I/O	External Memory Address
275	ARM_A9	I/O	External Memory Address
276	ARM_A8	I/O	External Memory Address
277	ARM_A7	I/O	External Memory Address
278	ARM_A6	I/O	External Memory Address
281	ARM_A5	I/O	External Memory Address
282	ARM_A4	I/O	External Memory Address
283	ARM_A3	I/O	External Memory Address
284	ARM_A2	I/O	External Memory Address
285	ARM_A1	I/O	External Memory Address
286	ARM_A0	I/O	External Memory Address (LSB)
240	ARM_D15	I/O	External Memory Data (MSB)
241	ARM_D14	I/O	External Memory Data
242	ARM_D13	I/O	External Memory Data
243	ARM_D12	I/O	External Memory Data
244	ARM_D11	I/O	External Memory Data
245	ARM_D10	I/O	External Memory Data
246	ARM_D9	I/O	External Memory Data
247	ARM_D8	I/O	External Memory Data
248	ARM_D7	I/O	External Memory Data
251	ARM_D6	I/O	External Memory Data
252	ARM_D5	I/O	External Memory Data
253	ARM_D4	I/O	External Memory Data
254	ARM_D3	I/O	External Memory Data
255	ARM_D2	I/O	External Memory Data
256	ARM_D1	I/O	External Memory Data
257	ARM_D0	I/O	External Memory Data (LSB)
236	FLSH_CE	O	Flash ROM Chip Enable
222	EM_OE	I/O	External Memory Output Enable
221	EM_WE	I/O	External Memory Write Enable
226	EM_CE0	I/O	External Memory Chip Enable #1
227	EM_CE1	O	External Memory Chip Enable #2
228	EM_RDWR	O	External Memory Read/Write
225	EM_BEH	O	External Memory Byte Enable High
224	EM_BEL	O	External Memory Byte Enable Low
223	EM_WAIT	I/O	External Memory Wait
204	CFE1	O	CFC Enable 1
205	CFE2	O	CFC Enable 2
203	CFOE	O	CFC Output Enable
206	CFWAIT	I	CFC Wait Signal Input
207	CFRDY	I	CFC Ready
209	CFIORD	O	CFC I/O Read Strobe
210	CFIOWR	O	CFC I/O Write Strobe
208	IOIS16	I	CFC I/O Select
237	BUSWIDTH	I	8/16 bit select for FLSH_CE
COMMUNICATION INTERFACE			

134	USB_DP	I/O	USB Data Plus
135	USB_DM	I/O	USB Data Minus
10	RXD0	I	UART RXD 0
11	TXD0	O	UART TXD 0
12	SDEN0	I/O	Serial Data Enable 0
13	SCLK0	I/O	Serial Clock 0
14	SDI0	I	Serial Data input 0
15	SDO0	O	Serial Data Output 0
SD/MS CARD INTERFACE			
215	SD_CLK	O	SD Clock
216	SD_CMD	I/O	SD Command
217	SD_DATA3	I/O	SD Data 3
218	SD_DATA2	I/O	SD Data 2
213	SD_DATA1	I/O	SD Data 1
214	SD_DATA0	I/O	SD Data 0
AUDIO INTERFACE			
8	DSP_CLKS0	I	DSP Serial Clock 0
1	DSP_BDX0	O	DSP Serial-port Transmit 0
2	DSP_BCLKX0	I/O	DSP Transmit Clock 0
3	DSP_BFSX0	I/O	DSP Frame Sync Pulse Transmit 0
4	DSP_BDR0	I	DSP Serial Data Receive 0
5	DSP_BCLKR0	I/O	DSP Receive Clock 0
6	DSP_BFSR0	I/O	DSP Frame Sync Pulse Receive 0
MISCELLANEOUS			
120	SYSCLK	I	External Clock Input
55	RSN	I	Reset for Main Land
56	PWDWN	I	Power Down Mode
231	MXI	I	X'tal Input
230	MXO	O	X'tal Output
117	M48XI	I	48MHz X'tal Input
116	M48XO	O	48MHz X'tal output
123	FLSHSEL	I	Flash ROM Bus Selector
124	EXTHOST	I	EMIF/EHIF Selector
122	BTSEL	I	ARM Boot Selector
58	TCK	I	JTAG Clock
59	TDI	I	JTAG Data Input
67	TDO	O	JTAG Data Output
60	TMS	I	JTAG Test Mode Select
61	TRST	I	JTAG Test Reset
62	TEST0	I	Test Input0
63	TEST1	I	Test Input1
64	TEST2	I	Test Input2
65	TEST3	I	Test Input3
66	SCANEN	I	Test Scan Enable
287	EMU0	I/O	Emulator Interrupt0
288	EMU1	I/O	Emulator Interrupt1
POWER PINS			
21	CVDD1	-	Core VDD
54	CVDD2	-	Core VDD
89	CVDD3	-	Core VDD
118	CVDD4	-	Core VDD

136	CVDD5	-	Core VDD
163	CVDD6	-	Core VDD
191	CVDD7	-	Core VDD
219	CVDD8	-	Core VDD
238	CVDD9	-	Core VDD
258	CVDD10	-	Core VDD
279	CVDD11	-	Core VDD
22	CGNG1	-	Core GND
57	CGNG2	-	Core GND
90	CGNG3	-	Core GND
115	CGNG4	-	Core GND
137	CGNG5	-	Core GND
164	CGNG6	-	Core GND
192	CGNG7	-	Core GND
220	CGNG8	-	Core GND
239	CGNG9	-	Core GND
259	CGNG10	-	Core GND
280	CGNG11	-	Core GND
7	VDD1	-	I/O VDD
37	VDD2	-	I/O VDD
79	VDD3	-	I/O VDD
101	VDD4	-	I/O VDD
121	VDD5	-	I/O VDD
211	VDD6	-	I/O VDD
229	VDD7	-	I/O VDD
249	VDD8	-	I/O VDD
269	VDD9	-	I/O VDD
9	GND1	-	I/O GND
39	GND2	-	I/O GND
80	GND3	-	I/O GND
99	GND4	-	I/O GND
119	GND5	-	I/O GND
212	GND6	-	I/O GND
232	GND7	-	I/O GND
250	GND8	-	I/O GND
270	GND9	-	I/O GND
149	SDVDD1	-	SDRAM I/O VDD
177	SDVDD2	-	SDRAM I/O VDD
150	SDGND1	-	SDRAM I/O GND
178	SDGND2	-	SDRAM I/O GND
113	PLLVDD1	-	PLL VDD
233	PLLVDD2	-	PLL VDD
114	PLLGND1	-	PLL GND
234	PLLGND2	-	PLL GND
133	USBVDD	-	USB I/O VDD
128	DAAVDD	-	DAC analog VDD
132	DADVDD	-	DAC digital VDD
127	DAAGND	-	DAC analogGND
131	DADGND	-	DAC digitalGND